



Escuela Técnica Superior  
de Ingeniería Informática

Grado en Ingeniería Informática

Curso 2021-2022

Trabajo Fin de Grado

**INTERFAZ GRÁFICA ASOCIADA A UN  
ALGORITMO DE CLUSTERING JERÁRQUICO,  
INTERACTIVO E INTERPRETABLE**

Autor: Marcos Vázquez Martín

Tutor: Manuel Rubio Sánchez



# Agradecimientos

Este trabajo ha sido posible gracias al tutor Manuel Rubio Sánchez que me ha permitido trabajar con tecnologías punteras en el campo del clustering inventadas por él. También quiero agradecer el apoyo de mi familia durante la realización de este Trabajo Fin de Grado.



# Resumen

El agrupamiento es una técnica usada en multitud de disciplinas que van desde la biología hasta la economía. Cuando el tamaño de los elementos a separar crece es de utilidad usar técnicas automáticas las cuales se engloban dentro del clustering, uno de los campos más antiguos del aprendizaje automático.

Uno de los cometidos de este TFG es el desarrollo de una herramienta que permita ejecutar un algoritmo de clustering interactivo e interpretable desarrollado por el Dr. Manuel Rubio. Este algoritmo nace con el mismo propósito que persigue el algoritmo de Maximum Marging Clustering (MMC): separar dos grupos de datos mediante hiperplanos maximizando el margen entre ellos. Para entender el algoritmo, se comenzará con la comprensión de qué es el clustering. Luego se hará un repaso por el estado del arte introduciendo técnicas modernas en el campo de clustering como los algoritmos basados en MMC o las ventajas de la interactividad.

Al ser, en un principio, necesaria la interacción se debe crear una interfaz que permita su ejecución, para ello se desarrollará una herramienta utilizando tecnologías web como React y Flask. Por último, se analizará el rendimiento de la herramienta con varios experimentos en los que se utilizarán bases de datos de distintos tamaños y se comparará con algoritmos, tradicionales y modernos, ampliamente utilizados en la actualidad. El algoritmo ha logrado alcanzar el estado del arte en bases de datos de distintos tamaños y dimensiones, aunque, sobre todo, en las de mayor tamaño. Además, se sugiere un método para automatizar el algoritmo.

## Palabras clave:

- Clustering interactivo
- React
- Flask
- Análisis de rendimiento



# Índice de contenidos

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| Índice de tablas                                                              | IX        |
| Índice de figuras                                                             | XIII      |
| <b>1. Introducción</b>                                                        | <b>1</b>  |
| 1.1. Contexto . . . . .                                                       | 1         |
| 1.1.1. Objetivos . . . . .                                                    | 2         |
| 1.2. Estructura del documento . . . . .                                       | 2         |
| <b>2. Marco Teórico</b>                                                       | <b>5</b>  |
| 2.1. Machine Learning . . . . .                                               | 5         |
| 2.2. Clustering . . . . .                                                     | 7         |
| 2.2.1. Distancias . . . . .                                                   | 8         |
| 2.2.2. Tipos de algoritmos de Clustering . . . . .                            | 9         |
| 2.2.3. Elección del algoritmo . . . . .                                       | 12        |
| 2.2.4. Aplicaciones del Clustering . . . . .                                  | 14        |
| 2.2.5. ¿Cómo evaluar lo buena que es una partición? . . . . .                 | 14        |
| 2.3. Problemas . . . . .                                                      | 17        |
| 2.3.1. Problemas derivados de la distancias . . . . .                         | 18        |
| 2.3.2. Datos de alta dimensionalidad . . . . .                                | 18        |
| 2.4. Ejemplos de Algoritmos de Clustering . . . . .                           | 20        |
| 2.4.1. K-Means . . . . .                                                      | 20        |
| 2.4.2. Otros . . . . .                                                        | 21        |
| <b>3. Estado del Arte</b>                                                     | <b>23</b> |
| 3.1. Clustering basado en separación máxima . . . . .                         | 23        |
| 3.1.1. Maquinas Vectores Soporte . . . . .                                    | 23        |
| 3.1.2. Maximum Margin Clustering . . . . .                                    | 25        |
| 3.2. Clustering mediante cortes con hiperplanos . . . . .                     | 26        |
| 3.3. Clustering Interactivo . . . . .                                         | 28        |
| 3.4. Algoritmo Interactivo de Clustering Jerárquico de Manuel Rubio . . . . . | 29        |
| 3.4.1. Medidas de Bimodalidad . . . . .                                       | 29        |
| 3.4.2. Algoritmo . . . . .                                                    | 29        |

|           |                                                                                    |           |
|-----------|------------------------------------------------------------------------------------|-----------|
| 3.4.3.    | Comparación con otros algoritmos . . . . .                                         | 31        |
| 3.4.4.    | Varianza . . . . .                                                                 | 31        |
| 3.4.5.    | Interactividad . . . . .                                                           | 31        |
| 3.4.6.    | Desventajas . . . . .                                                              | 32        |
| 3.5.      | Tecnologías . . . . .                                                              | 32        |
| <b>4.</b> | <b>Descripción Informática</b>                                                     | <b>35</b> |
| 4.1.      | Estudio Previo . . . . .                                                           | 36        |
| 4.2.      | Análisis de Requisitos . . . . .                                                   | 37        |
| 4.2.1.    | Identificación de Requisitos Funcionales . . . . .                                 | 37        |
| 4.2.2.    | Casos de uso . . . . .                                                             | 38        |
| 4.2.3.    | Prototipos . . . . .                                                               | 41        |
| 4.3.      | Diseño . . . . .                                                                   | 43        |
| 4.3.1.    | Diagrama de Actividad . . . . .                                                    | 44        |
| 4.3.2.    | Diagrama de Componentes . . . . .                                                  | 45        |
| 4.3.3.    | Representación de Interacción con la API mediante Diagramas de Secuencia . . . . . | 46        |
| 4.4.      | Prototipo Final: Experiencia de usuario . . . . .                                  | 48        |
| <b>5.</b> | <b>Experimentos</b>                                                                | <b>55</b> |
| 5.1.      | Prueba Inicial: Separación de Aceites de Oliva . . . . .                           | 56        |
| 5.2.      | Experimento 1: Comparar las distintas medidas . . . . .                            | 58        |
| 5.3.      | Experimento 2: Conjuntos 2D . . . . .                                              | 59        |
| 5.4.      | Experimento 3: Bases de Datos Reales . . . . .                                     | 60        |
| 5.5.      | Experimento 4: Algoritmos MMC . . . . .                                            | 64        |
| 5.6.      | Experimento 5: Automatización del Algoritmo . . . . .                              | 66        |
| <b>6.</b> | <b>Conclusiones y trabajos futuros</b>                                             | <b>71</b> |
|           | <b>Bibliografía</b>                                                                | <b>75</b> |
|           | <b>Apéndices</b>                                                                   | <b>77</b> |
| <b>A.</b> | <b>Conceptos Matemáticos</b>                                                       | <b>79</b> |
| A.1.      | Conceptos de Álgebra . . . . .                                                     | 79        |
| A.2.      | Algoritmos de optimización . . . . .                                               | 80        |
| A.2.1.    | Descenso de Gradiente . . . . .                                                    | 80        |
| A.2.2.    | Frontera de Pareto . . . . .                                                       | 81        |
| A.3.      | Exploración de Datos . . . . .                                                     | 82        |
| A.3.1.    | Métricas de Variabilidad . . . . .                                                 | 82        |
| A.3.2.    | Exploración de una Distribución . . . . .                                          | 82        |
| <b>B.</b> | <b>Gráficas y Tablas Grandes</b>                                                   | <b>83</b> |



# Índice de tablas

|      |                                                                                                                                                                                                                                                                       |    |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 5.1. | Porcentaje de acierto en olives con el algoritmo utilizando las distintas medidas de bimodalidad. . . . .                                                                                                                                                             | 59 |
| 5.2. | Características de las Bases de Datos utilizadas. . . . .                                                                                                                                                                                                             | 60 |
| 5.3. | Comparación entre el algoritmo propuesto, Hierarchical Interactive Clustering (HIC) y la implementación del algoritmo MMC (Los resultados marcados con (*) son los mejores en esa base de datos con la comparación hecha en el artículo de Li et co. (2009) [1]). . . | 65 |
| 5.4. | Resultados de ejecutar el algoritmo de MMC con sus dos kernel en bases de datos, frente a los resultados obtenidos por el propuesto en este TFG, Hierarchical Interactive Clustering (HIC). . . . .                                                                   | 66 |
| 5.5. | Generación automática de la separación con los métodos de Otsu y (en verde dentro de los márgenes y en rojo fuera), frente a los elegidos por el usuario. . . . .                                                                                                     | 69 |
| B.1. | Comparación de algoritmos de Clustering con el algoritmo propuesto en el TFG, Hierarchical Interactive Clustering (HIC). . . .                                                                                                                                        | 85 |



# Índice de figuras

|      |                                                                                                                                                                                                                                               |    |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. | (a) distribución (clases representadas por colores, nombre de los elementos representados por números en cada punto) (b) Ejemplo de dendograma de la distribución (nombre de los elementos representados por números en el eje x).            | 10 |
| 2.2. | Dendogramas de la misma distribución con distintos representantes: (a) distribución (b) dendograma con single-linked; (c) dendograma con Ward.                                                                                                | 11 |
| 2.3. | Comparación de métodos de Linkage en clustering aglomerativo.                                                                                                                                                                                 | 13 |
| 2.4. | Resultado de aplicar VAT a las bases de datos de (a) Iris y (b) Olivas.                                                                                                                                                                       | 15 |
| 2.5. | Ejemplo de la utilidad del Subspace Clustering.                                                                                                                                                                                               | 19 |
| 2.6. | Ejemplo de Ejecución del Algoritmo K-Means en unos datos con 3 nubes de puntos con un ruido diferente. Las marcas rocas corresponden a los centroides. Las divisiones están marcadas con líneas negras y los clústeres con distintos colores. | 20 |
| 3.1. | Definición de los márgenes en un problema de SVM. (AnAj, 2006) [2].                                                                                                                                                                           | 24 |
| 3.2. | Ejemplo de aplicación de kernel en SVM (Elisfm, n.d.) [3].                                                                                                                                                                                    | 25 |
| 3.3. | Ejemplo de Maximum Margin Clustering.                                                                                                                                                                                                         | 26 |
| 3.4. | Ejemplo de separación mediante corte con hiperplano en 2 dimensiones.                                                                                                                                                                         | 27 |
| 3.5. | Representación del Algoritmo.                                                                                                                                                                                                                 | 30 |
| 4.1. | Diagrama de caso de uso de preparación de los datos.                                                                                                                                                                                          | 39 |
| 4.2. | Diagrama de caso de uso ejecución del algoritmo.                                                                                                                                                                                              | 39 |
| 4.3. | Diagrama de casos de uso de crear nuevo histograma y realizar separación.                                                                                                                                                                     | 40 |
| 4.4. | Otros casos de uso.                                                                                                                                                                                                                           | 40 |
| 4.5. | Mocks de la herramienta (Fase de diseño).                                                                                                                                                                                                     | 41 |
| 4.6. | Prototipo en TKinter.                                                                                                                                                                                                                         | 42 |
| 4.7. | Prototipo en Bokeh.                                                                                                                                                                                                                           | 43 |
| 4.8. | Prototipo Plotly.                                                                                                                                                                                                                             | 43 |
| 4.9. | Diagrama de Actividad del algoritmo interactivo.                                                                                                                                                                                              | 44 |

---

|                                                                                                                                                                                                                                                                                                    |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.10. Diagrama de Árbol de Componentes de la Ventana del Algoritmo. . . . .                                                                                                                                                                                                                        | 45 |
| 4.11. Componentes de la Ventana del Algoritmo. . . . .                                                                                                                                                                                                                                             | 46 |
| 4.12. Diagrama de secuencia de la interacción entre frontend y backend<br>al inicio de la aplicación (cargaDatos, uploadDB y iniciar). . . . .                                                                                                                                                     | 47 |
| 4.13. Diagrama de secuencia de la interacción entre frontend y backend:<br>(a) ejecución del algoritmo (nuevoPunto, CambioDeMedida, nuevoHistograma y separación); (b) funciones auxiliares de la API<br>(juntar, divisorAutomatico, resultadoNodoSolucion, resultadoSolucion y download). . . . . | 48 |
| 4.14. Pantalla de carga de la herramienta. . . . .                                                                                                                                                                                                                                                 | 49 |
| 4.15. Carga de Archivos de la herramienta. . . . .                                                                                                                                                                                                                                                 | 49 |
| 4.16. Ventana principal del algoritmo. . . . .                                                                                                                                                                                                                                                     | 50 |
| 4.17. Información sobre las divisiones: (a) Carrusel, (b) Grid. . . . .                                                                                                                                                                                                                            | 51 |
| 4.18. Árbol en estado de partición (nodo amarillo), con tres particiones<br>ya hechas (nodos azules). . . . .                                                                                                                                                                                      | 52 |
| 4.19. Frontera de Pareto al terminar de ejecutar el algoritmo. . . . .                                                                                                                                                                                                                             | 52 |
| 4.20. Prototipo en el estado de selección de los histogramas. . . . .                                                                                                                                                                                                                              | 53 |
| 4.21. Separación del histograma. . . . .                                                                                                                                                                                                                                                           | 53 |
| 4.22. Información de las particiones en los nodos hoja. . . . .                                                                                                                                                                                                                                    | 54 |
| 4.23. Información de la separación en los nodos internos. . . . .                                                                                                                                                                                                                                  | 54 |
| 5.1. Estudio de los datos: (a) Información de las propiedades, (b) Información de la columna solución. . . . .                                                                                                                                                                                     | 56 |
| 5.2. Ejemplos de histogramas de la primera separación. . . . .                                                                                                                                                                                                                                     | 57 |
| 5.3. Estado del componente árbol cuando la separación ha sido realizada. . . . .                                                                                                                                                                                                                   | 57 |
| 5.4. Gráfica de <i>eiconsenoic</i> frente a las clases. . . . .                                                                                                                                                                                                                                    | 58 |
| 5.5. Gráficos con la separación de los datos: (a) $\phi$ , (b) $\psi$ , (c) $\beta$ . . . . .                                                                                                                                                                                                      | 59 |
| 5.6. VAT de la base de datos de Glass. . . . .                                                                                                                                                                                                                                                     | 60 |
| 5.7. Resultado de la primera división de Glass. . . . .                                                                                                                                                                                                                                            | 61 |
| 5.8. Resultado de la segunda división de Glass. . . . .                                                                                                                                                                                                                                            | 62 |
| 5.9. Resultado de la tercera división de Glass. . . . .                                                                                                                                                                                                                                            | 62 |
| 5.10. (a) Árbol de la herramienta (b) Dendrograma con Ward-Linkage. . . . .                                                                                                                                                                                                                        | 63 |
| 5.11. (a) Árbol de la herramienta (b) Dendrograma con Ward y Average-Linkage, (c) Dendrograma con Single-Linkage, (d) Dendrograma con Complete-Linkage. . . . .                                                                                                                                    | 64 |
| 5.12. Salida del programa de Li (n.d.) [4] donde se implementa la versión descrita en esta sección del algoritmo MMC en Matlab. . . . .                                                                                                                                                            | 65 |
| 5.13. Concepto de separación automática. . . . .                                                                                                                                                                                                                                                   | 68 |
| A.1. Ejemplo de espacio vectorial y un vector de ese espacio. . . . .                                                                                                                                                                                                                              | 80 |
| A.2. Simulación de la ejecución del Descenso de Gradiente para un Parámetro. . . . .                                                                                                                                                                                                               | 81 |
| A.3. Ejemplo de Histograma. . . . .                                                                                                                                                                                                                                                                | 82 |

---

|                                                                                                   |    |
|---------------------------------------------------------------------------------------------------|----|
| B.1. Resultados de HIC en distribuciones 2D comparado con otros algoritmos de clustering. . . . . | 84 |
|---------------------------------------------------------------------------------------------------|----|





# 1

## Introducción

Los humanos tienden a agrupar las cosas parecidas según un criterio. Por ejemplo, a los animales que vuelan se les denomina aves y a los que viven en el mar peces. O a los objetos que se utilizan para comer les se les llama cubiertos. Esto tiene multitud implicaciones sociológicas y permite a los seres humanos usar en el día a día habilidades tan útiles como agrupar, jerarquizar, sintetizar, categorizar... Pero ¿puede una máquina simular esta manera de pensar? Eso es lo que tratan los algoritmos de agrupamiento. El objetivo es sencillo: crear grupos de datos similares mientras se diferencian lo máximo posible entre ellos. El clustering es uno de los campos con mayor recorrido dentro del campo del aprendizaje automático. Siendo numerosos los algoritmos desarrollados y los campos en los que se ha aplicado entre los que se encuentran la genética, la astronomía o la psicología.

### 1.1. Contexto

Debido al gran número de algoritmos de clustering que existen se hace complicado buscar cual es el más apropiado para cada problema. Muchos algoritmos tradicionales se estancan cuando las bases de datos aumentan su tamaño (algo que es cada vez más normal con el actual nivel de acaparamiento de datos). Esto hace que proliferen soluciones que tengan en cuenta estos problemas. Una de ellas es el Maximum Marging Clustering (MMC), una solución que busca el máximo margen de separación entre dos grupos de datos. Además, durante el desarrollo de las técnicas de clustering, se han mezclado con herramientas interactivas que permiten obtener resultados más entendibles y que se ajusten mejor a los deseos



del usuario.

En este contexto nace el algoritmo del Dr. Manuel Rubio, un método de clustering interactivo que tiene el mismo propósito que el MMC. Para conocer el potencial de este algoritmo será necesario saber si es capaz de superar a otros algoritmos de clustering como el MMC, si es dependiente de la interactividad o se puede automatizar y en el caso de que lo sea, si los resultados siguen siendo lo suficientemente buenos. Para responder a estas cuestiones será necesario desarrollar una herramienta que permita ejecutar el algoritmo.

### 1.1.1. Objetivos

El objetivo principal del TFG es desarrollar una herramienta que permita ejecutar el algoritmo de agrupamiento del Dr. Manuel Rubio. Además, esta interfaz debe permitir seleccionar los parámetros del algoritmo de forma intuitiva y observar la interpretabilidad del algoritmo visualmente.

Otros objetivos son:

- Realizar un estudio de las tecnologías con las que se desarrollará la herramienta. Y realizar un desarrollo incremental con evaluaciones periódicas del tutor.
- Añadir varias herramientas interactivas que permitan obtener soluciones más entendibles y ajustables.
- Evaluar el rendimiento del algoritmo frente a otros algoritmos en bases de datos de distintas dimensiones.
- Evaluar la posibilidad de automatizar ciertas partes del algoritmo.

## 1.2. Estructura del documento

En el **Capítulo 2: Marco Teórico** se comenzará introduciendo al lector en el campo del aprendizaje automático y explicando donde encaja el clustering dentro de él. Para seguir, se hará un repaso al problema del agrupamiento, donde se contará su origen, se explicarán sus bases y algunos de los tipos de algoritmos. Se enumerarán distintas formas de medir los rendimientos de estos algoritmos y los problemas que acarrearán algunos métodos de clustering. Luego se describirán varios algoritmos como el K-means.

En el **Capítulo 3: Estado del Arte** se explicarán alternativas al algoritmo que se utilizará en la parte experimental como el MMC y el corte con hiperplanos. Se

seguirá mostrando las ventajas de la interactividad en los algoritmos de clustering y explicando el método inventado por el Dr. Manuel Rubio. Por último, se describirán las herramientas utilizadas para desarrollar la herramienta.

En el **Capítulo 4: Descripción Informática**, una vez descrito el contexto en el que se encuentra el algoritmo, se explicará cómo se ha realizado la herramienta que permite ejecutarlo. Empezando por la toma de requisitos, el diseño y el prototipado, para terminar, explicando cómo es el prototipo final.

En el **Capítulo 5: Experimentos** se realizarán varios experimentos en los que se probará la efectividad de la herramienta y del algoritmo del Dr. Manuel Rubio.

En el **Capítulo 6: Conclusiones y trabajos futuros**, se reflexionará sobre los resultados obtenidos y se hará un repaso a los objetivos listados en la sección anterior. Se terminará hablando sobre posibles trabajos futuros.

Por último en los Apéndices se explicarán conceptos útiles para el entendimiento del trabajo y se representarán tablas e imágenes grandes (**Apéndice B: Gráficas y Tablas Grandes**). En el **Apéndice A: Conceptos Matemáticos** se explicarán conceptos matemáticos de álgebra, optimización y estadísticas que se usan a través del TFG.



# 2

## Marco Teórico

### 2.1. Machine Learning

La Inteligencia Artificial es una rama de la informática poco definida y con acepciones distintas según el autor. Unos piensan que se trata del diseño de programas que se asemejen al comportamiento humano., como, por ejemplo, Suchman y Trigg (2000) [5] que afirman que “El objetivo de la investigación en Inteligencia Artificial es el de diseñar programas de ordenador que repliquen un aspecto del comportamiento humano” (p. 34). Mientras otros investigadores piensan como McCarthy (1998) [6] que afirma que la inteligencia es “la capacidad de alcanzar metas en el mundo” (p. 2). A la pregunta “¿La IA trata de simular la inteligencia humana?”, responde: “A veces, pero no siempre”(p. 3). Este TFG se centra en el aprendizaje automático, uno de los campos de la Inteligencia Artificial que se dedica a crear sistemas capaces de aprender.

De manera más formal, se puede definir el Machine Learning (*aprendizaje automático* en inglés) como la habilidad de un programa de realizar una tarea y mejorarla a través de la experiencia, desde el punto de vista de cierta medida.

En contraposición al aprendizaje automático, existe la automatización, la cual utiliza técnicas tradicionales de programación, como condicionales o bucles, para definir reglas que determinan como procederá el programa. Las técnicas de automatización son deterministas: bajo una misma entrada tendrá siempre los mismos resultados. Además, la solución al problema se devuelve de manera instantánea al terminar el algoritmo.

Los algoritmos de aprendizaje automático requieren una computación inicial para dar resultados que se acerquen lo máximo a una solución correcta. Estos algoritmos permiten solucionar problemas complejos, no deterministas y dinámicos (que pueden cambiar con el tiempo), los cuales serían muy costosos o imposibles de solucionar con programación tradicional.

Existen multitud de algoritmos de aprendizaje automático que resuelven problemas muy diferentes, pero existen ciertos conceptos transversales. Algunos de ellos que se usarán en el resto del actual trabajo son:

- **Problema:** Para utilizar un algoritmo de inteligencia artificial primero se necesitan un problema o grupo de problemas que resolver, por ejemplo, diferenciar las imágenes en las que hay un perro y en las que no.
- **Atributos:** Son un tipo de dato que describe una característica de la entrada. De vuelta al ejemplo anterior de detección de perros, un atributo podría ser el nivel de verde del primer píxel de una imagen.
- **Etiqueta:** Es la medida que se desea predecir. En el caso del clasificador de imágenes de perros sería si en cierta imagen hay un perro o no.
- **Modelo:** Será el método que se usará para encontrar la etiqueta que represente a los datos. Una vez el problema haya sido resuelto el modelo que se utilice será único y no sirve para resolver ningún otro problema. Lo cual no significa que no se pueda utilizar el mismo algoritmo e incluso el conocimiento o parte del conocimiento adquirido durante la resolución de otro problema para solucionarlo. Normalmente, los algoritmos cuentan con multitud de parámetros para moldearlos al problema en el que se utilicen.

Además, las etapas que se suelen seguir para resolver un problema con aprendizaje automático son:

1. **Comprensión del problema, análisis de los datos y elección del algoritmo:** Es la primera etapa de cualquier problema de aprendizaje automático. Entender el contexto y analizar los datos ayudan para prepararlos y elegir un algoritmo o grupo de ellos para solucionar el problema. Es recomendable probar con varios métodos ya que la eficacia de los algoritmos es diferente en cada problema, a veces, incluso, en cada ejecución del mismo algoritmo.
2. **Aprendizaje:** Es el proceso en el que el modelo aprende a hacer la tarea para la que ha sido entrenada con la experiencia que le dan los datos de entrenamiento.
3. **Evaluación:** En esta etapa se mide el rendimiento del modelo a través de cierta medida. Dependiendo del problema y de la medida escogida, la

evaluación dará información más o menos fiable sobre el comportamiento que tendrá el modelo con nuevos datos.

Según contenga el problema etiquetas o no, se pueden dividir los algoritmos de Machine Learning en:

- Los **modelos supervisados**: Para problemas en los que los datos están etiquetados. Los dos principales tipos de problemas de este tipo de algoritmos son los de clasificación, en los que se averigua la clase a la que pertenece cierta instancia; y el de regresión el cual asigna una medida a cada instancia. Un ejemplo del primero sería decidir que imágenes contienen perros y cuales no, mientras que un ejemplo de la segunda sería determinar la edad de un vehículo a través de los datos de su estado. Algunos modelos supervisados son k-vecinos más cercanos, máquinas de vector soporte o redes neuronales.
- Los **modelos no supervisados**: Se utilizan con datos sin etiquetar. Estos algoritmos tienen multitud de usos como el análisis o la segmentación de datos, la reducción de la dimensionalidad, o la detección de anomalías. Algunos ejemplos de modelos no supervisados son: K-means(clustering), PCA (reducción de la dimensionalidad), SVM de una clase (detección de anomalías).

## 2.2. Clustering

El clustering aglutina un conjunto de algoritmos que permiten encontrar patrones o agrupaciones en conjuntos sin etiquetar. Lo que quiere decir que, mientras que los algoritmos supervisados necesitan etiquetas, en el clustering no se utiliza información de la clase. Esto permite descubrir nuevas tendencias en los datos, agrupar los elementos parecidos o descubrir clasificaciones de datos. Los resultados no suelen tener la precisión de los algoritmos supervisados, pero las soluciones pueden dar mucha más información.

La primera vez que se habló del clustering en un artículo científico no fue en el campo de la informática, sino que sucedió en el campo de la antropología, donde Driver y Kroeber (1932) [7] la utilizaron en relación a grupos de tribus que separaron por la zona geográfica a la que pertenecían (p. 249). Le siguieron artículos en multitud de otros campos como psicología, astronomía o biología, quedando claro, que la agrupación es un problema común en muchas disciplinas. Era cuestión de tiempo que se descubriese un algoritmo para automatizar la solución de estos problemas, y fue Steinhaus (1956) [8] quien dio, por primera vez, las bases del algoritmo de k-medias, aunque no sería publicado hasta 1982. Tiempo en el que otros autores como J. MacQueen (1967)[9], llegaron a la misma conclusión. A partir de aquí hasta el día de hoy se sucedieron multitud de familias

de algoritmos de agrupamientos muy variadas entre sí, que hacen complicado su comparación.

Formalmente se puede definir clustering como la partición de datos en grupos según una medida que maximice la similitud entre los datos del mismo grupo y la minimice con los datos de otros grupos. La medida de similitud es similar a lo que hacen los humanos: cuando se dice que dos cosas se parecen es que tienen propiedades similares. Lo mismo hacen las máquinas, las cuales utilizan estas medidas de similitud sobre los atributos de cada dato. Esto permite saber cómo de iguales son 2 elementos. Normalmente estas medidas se basan en distancias, aunque también se pueden utilizar medidas estadísticas o de densidad.

### 2.2.1. Distancias

Una distancia asociada a un conjunto formado por puntos es la asignación a cada par de estos puntos de un valor real, siguiendo las siguientes normas:

*Definición 2.2.1* Sea  $d(x,y)$  la distancia entre  $x$  e  $y$ ,  $d$  debe cumplir:

1. *Propiedad reflexiva:*  $d(i, j) = 0$  solo si  $i = j$
2. *Propiedad simétrica:*  $d(i, j) = d(j, i)$
3. *Propiedad triangular:*  $d(i, j) \leq d(i, k) + d(k, j)$

Con estas normas se puede ver que existen infinitas maneras de medir, y se podría utilizar cualquiera de ellas en un algoritmo de clustering, será tarea del diseñador del algoritmo el escoger la más conveniente. Entre las más famosas se encuentran las distancias de Minkowsky:

$$d_p(x, y) = \sqrt[p]{\sum_{k=1}^n |x_k - y_k|^p}$$

Donde  $x$  e  $y$  son dos datos con  $n$  parámetros. Algunas de estas distancias tienen un nombre especial:

- Distancia de Manhattan para  $p = 1$ .
- Distancia Euclídea para  $p = 2$ .
- Distancia Chebyshev para  $p = \infty$ , donde se elige el máximo de las restas de cada componente:

$$d_\infty(x, y) = \max(|x_0 - y_0|, |x_1 - y_1|, \dots, |x_n - y_n|)$$

Estas distancias solo sirven para números. Para otros tipos de datos como cadenas de caracteres o datos binarios existen otras medidas como, por ejemplo, la distancia de Hamming, que cuenta los caracteres necesarios que hay que cambiar para convertir una palabra en otra.

### 2.2.2. Tipos de algoritmos de Clustering

Existen multitud de familias de clustering, las cuales engloban a su vez varios algoritmos. Entre las más importantes se encuentran:

**Agrupamiento basado en particiones:** Se eligen un número de representantes de cada grupo, que serán los que lo definan. El resto de los puntos se asigna al grupo con los representantes más cercanos. Estos representantes se actualizarán para encontrar los que minimicen la distancia de los puntos a sus representantes. Estos algoritmos son sencillos de implementar, rápidos y funcionan bien separando conjuntos convexos, es decir, que el camino mínimo de un punto a otro del clúster este por dentro de la silueta de la agrupación. Algunas de las desventajas de este tipo de agrupamiento, que remarcan en un manual publicado por Google (n.d.) [10] sobre clustering, son que si no se fija el número de particiones se convierten en problemas NP-Duros, que son dependientes de los valores iniciales, su efectividad disminuye cuando los clústeres tienen distintos tamaños o densidades y son dependientes de outsiders (elementos extremos). Un ejemplo de estos algoritmos es K-Means.

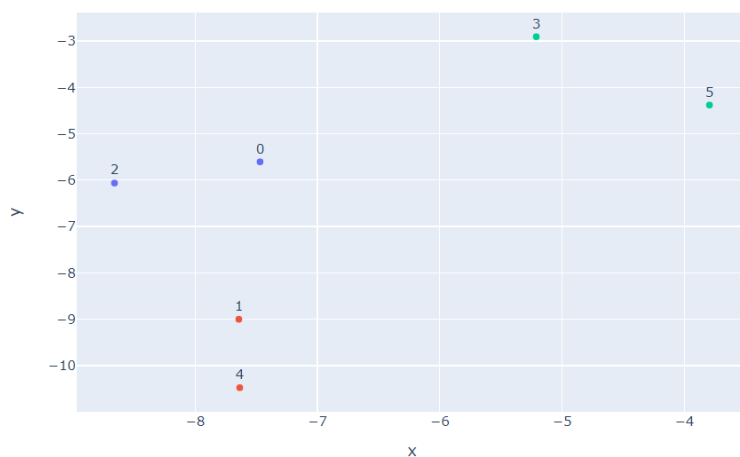
**Agrupamiento basado en modelos estadísticos:** Se asume un modelo estadístico, y se utilizan los datos para adaptar los parámetros del modelo al problema. Con estos modelos se puede sacar las probabilidades de cada punto de pertenecer a un clúster. Su efectividad depende de lo bien que se adapten los datos al modelo. Un ejemplo de estos algoritmos es la Gaussian Mixture Model.

**Agrupamiento basado en densidad:** Buscan encontrar regiones con muchos puntos juntos separados por regiones vacías. El algoritmo trata de agrandar grupos uniendo a los puntos vecinos. Cuando no encuentre vecinos lo suficientemente cercanos, delimitará un vecindario o grupo. Estos algoritmos permiten agrupar datos con formas indeterminadas pero su efectividad cae cuando aumentan las dimensiones. Un ejemplo es el algoritmo DBSCAN.

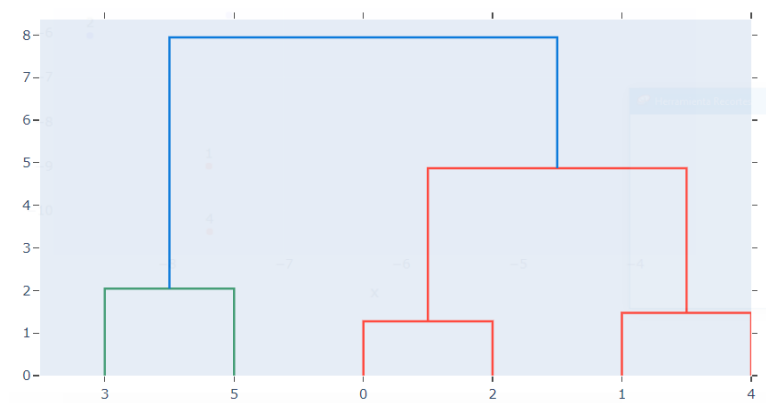
**Clustering jerárquicos:** Crean clústeres con relaciones de pertenencia en forma de árbol. Existen dos tipos de algoritmos jerárquicos: los aglomerativos, que agrupa elementos pequeños en conjuntos más grandes; y los divisivos que dividen los grupos más grandes en grupos más pequeños. El proceso que siguen los algoritmos aglomerativos es el de juntar los dos elementos más parecidos en base a una distancia repetidamente hasta que solo quede un grupo con todos los elementos. Estos se pueden representar con los dendogramas, un gráfico en forma de árbol cuyas hojas son los elementos separados y sus padres son las uniones de



estos puntos. Según se asciende en los niveles del árbol se generan grupos más grandes hasta que en la cabeza se encuentran todos los datos. La suma de la longitud de los vértices de cada nodo representa la distancia entre los grupos. En la Figura 2.1 (b) se puede observar un ejemplo de dendograma de la distribución representada en (a).



(a)



(b)

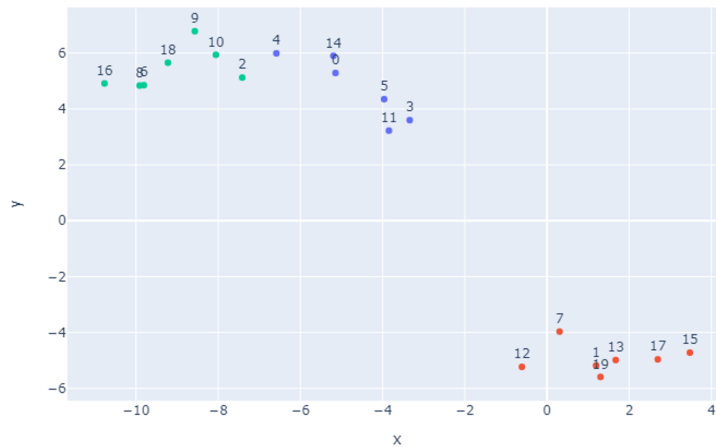
Figura 2.1: (a) distribución (clases representadas por colores, nombre de los elementos representados por números en cada punto) (b) Ejemplo de dendograma de la distribución (nombre de los elementos representados por números en el eje x).

Para comparar la similitud de dos grupos se debe escoger un representante de cada uno y calcular la distancia como si fuesen elementos individuales. Existen distintas formas de escoger un representante:

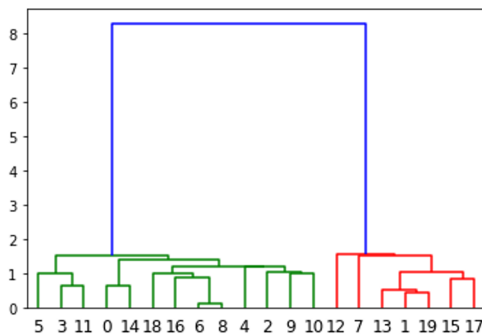
- **Single-Linkage:** escoge como representante al punto medio entre los dos elementos más similares del grupo.

- **Complete-Linkage:** escoge como representante al punto medio entre los dos elementos más lejanos del grupo.
- **Average-Linkage:** escoge como representante al punto medio entre todos los elementos del grupo.
- **Single-Linkage:** escoge como representante con la varianza de todos los elementos del grupo.

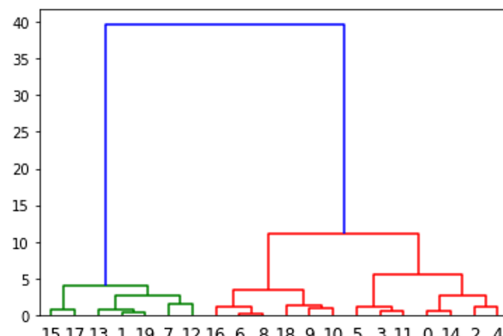
El método usado para escoger los representantes dependerá de la base de datos utilizada. Cuando los datos son simples los resultados son idénticos como en el caso de la Figura 2.1, pero cuando las separaciones son más complicadas la elección puede ser decisiva como en el caso de la Figura 2.2 donde se puede ver en los resultados que Single-Linked (*b*) ha realizado una peor partición de las clases que el resto de métodos, como, por ejemplo, Ward-linkage (*c*). Un ejemplo de algoritmo jerárquico es el clustering aglomerativo.



(a)



(b)



(c)

Figura 2.2: Dendrogramas de la misma distribución con distintos representantes: (a) distribución (b) dendrograma con single-linked; (c) dendrograma con Ward.

En el artículo de Jain et al. (1999) [11] afirman que el método Complete-Linkage produce clústeres con delimitaciones estrechas. Mientras, los algoritmos que utilizan Single-Linkage pueden provocar encadenamientos de clústeres cercanos y tienen tendencia a ser desordenados y alargados. En la Figura 2.3 se puede observar esto. El método Single-Linkage tienden a unir clústeres cercanos como en los casos tres y cuatro. En el resto hay clústeres distintos que pueden estar unidos. Parece que para distribuciones complejas Single-Linkage funciona mejor y en el resto es mejor el método Ward-Linkage.

### 2.2.3. Elección del algoritmo

Las familias de algoritmos de la sección anterior son solo algunas de las más importantes. Por lo que, con lo visto hasta ahora, parece complicado elegir tanto el algoritmo como de la distancia que encajen mejor con el problema. A pesar, de ello, cada algoritmo suele tener multitud de parámetros para ajustarle a las necesidades del problema. Los dos más usados suelen ser el número de clústeres y el número mínimo de elementos por clúster.

Como recuerda Jain (2010) [12] “el análisis de clústeres es una herramienta exploratoria, [...] solo sugiere hipótesis”. Y afirma que a pesar de que existan multitud de métodos de clustering “no existe un algoritmo de agrupación que haya demostrado dominar a otros algoritmos en todos los dominios”. Concluyendo con que casi cualquier método de clustering se debe tener en cuenta para hacer una separación, incluidos los más simples.

Aunque para elegir un algoritmo para un problema determinado, se pueden observar que propiedades ha mostrado en los ejemplos en los que se ha usado. Algunas de estas propiedades son:

- **Escalabilidad:** Poder utilizar los algoritmos con más parámetros y elementos en tiempos proporcionales a los que tienen con menos.
- Poder procesar agrupaciones de **formas arbitrarias:** Poder hacer cortes a los datos de cualquier forma. Por ejemplo, que se puedan dividir dos aros de puntos concéntricos en 2D.
- **Tolerancia contra outsiders:** Que el modelo no se rompa por tener elementos extremadamente alejados del resto.
- **Interpretabilidad** y ser capaces de **adaptarse a las restricciones** del usuario.

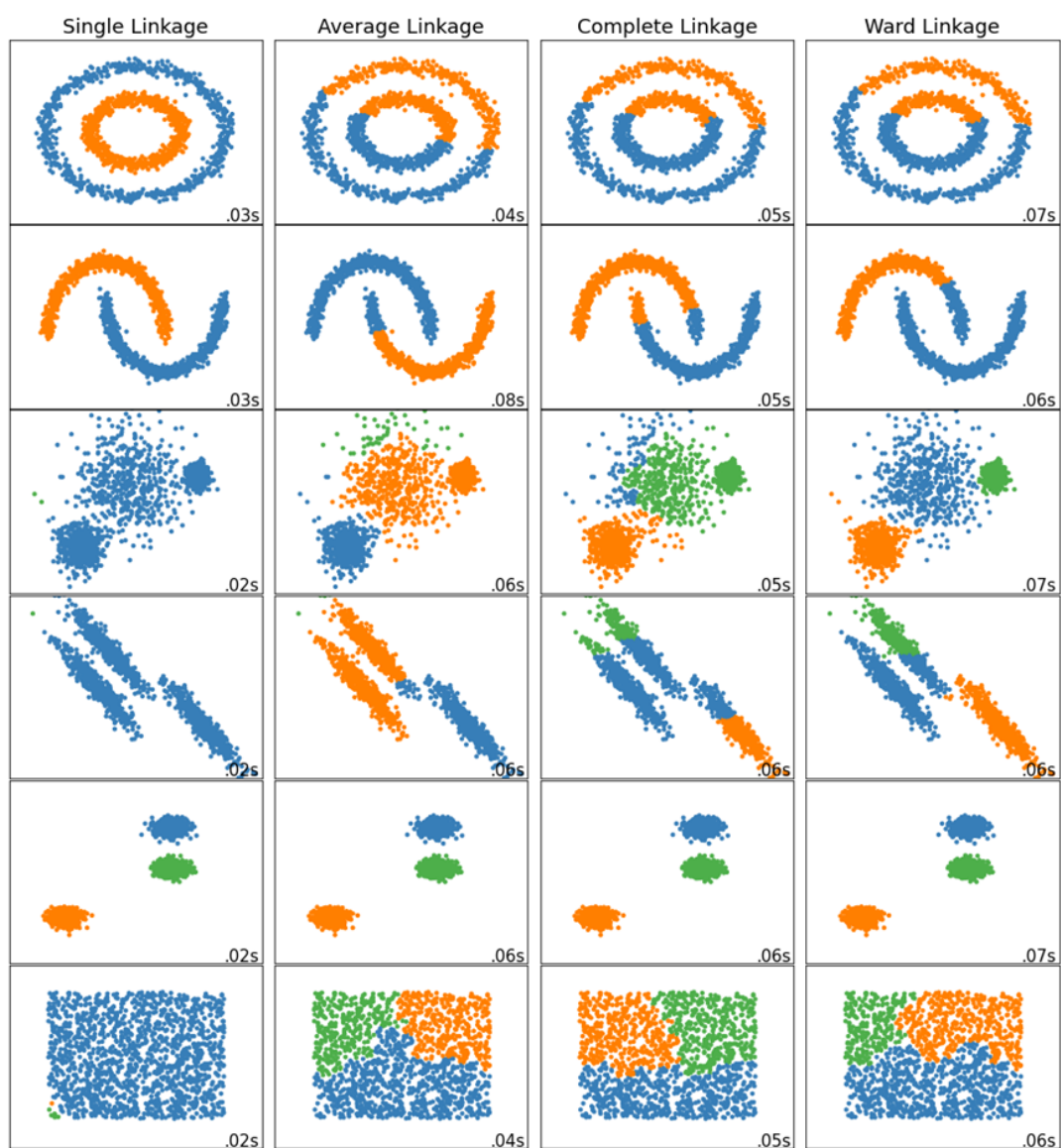


Figura 2.3: Comparación de métodos de Linkage en clustering aglomerativo.

### 2.2.4. Aplicaciones del Clustering

El clustering se utiliza en multitud de sectores (medico, genómico, financiero...) y entre sus aplicaciones se encuentran:

- Clasificación: es la tarea más común, dividir los datos en grupos. Esto se puede utilizar para etiquetar datos sin información de clase, segmentar los datos (juntar elementos similares como clientes con los mismos gustos o grupos de genes con estructuras parecidas), conocer densidades (encontrar aglomeraciones o detectar anomalías) ...
- Análisis de datos: La división y el por qué se ha realizado esa separación permite conocer mejor los datos. Existen técnicas que representan los datos en formatos gráficos más entendibles.
- Segmentación de imágenes: se trata de reducir los colores de una imagen juntando las partes similares. Se suele utilizar para detección o seguimiento de objetos.
- Sistemas de Búsquedas: La separación de grupos y la jerarquización de los datos son dos de los principales cometidos de los sistemas de búsqueda. Lo cual se parece a la definición dada d la familia de algoritmos jerárquicos de clustering en la Sección [2.2.2](#).
- Reducción de la Dimensión: Algunos algoritmos de clustering permiten conocer cuáles son los atributos en los que más se ha fijado para hacer la división. Otra forma de reducir la dimensionalidad es sustituir los parámetros por la medida de similitud de cada punto a cada clúster. Se reducirá del número de parámetros al número de clústeres que suele ser menor.

### 2.2.5. ¿Cómo evaluar lo buena que es una partición?

Para realizar una buena partición es necesario tener en cuenta tres aspectos fundamentales: que los datos sean separables, la elección correcta del número de clústeres y la correcta separación de los datos.

Como los algoritmos de Clustering pueden separar datos haya o no una evidencia de que existe una buena partición, son necesarios métodos que permitan saber si un conjunto es separable o no. Estos métodos evalúan lo que se denomina *tendencia a la separación*. Existen tanto métodos estadísticos (p.e. Método Hopkins), como métodos visuales (p.e. VAT). PyClusterEnd es paquete para Python que contiene métodos para evaluar la tendencia a la separación de bases de datos. En su documentación, Pycluster (n.d.) [\[13\]](#) se ofrece más información sobre los dos métodos mencionados como ejemplos: El **coeficiente Hopkins** permite

conocer la uniformidad de los datos, cuanto más cercano sea 0.5 más uniforme será la base de datos y los grupos encontrados no serán representativos.

**VAT** o visualización de la tendencia es un algoritmo que crea una matriz de distancias entre todo par de datos y se ordena para que los datos similares queden cerca. Una forma de interpretar el gráfico es contar el número de cuadrados negros en la diagonal que debería representar el número de grupos.

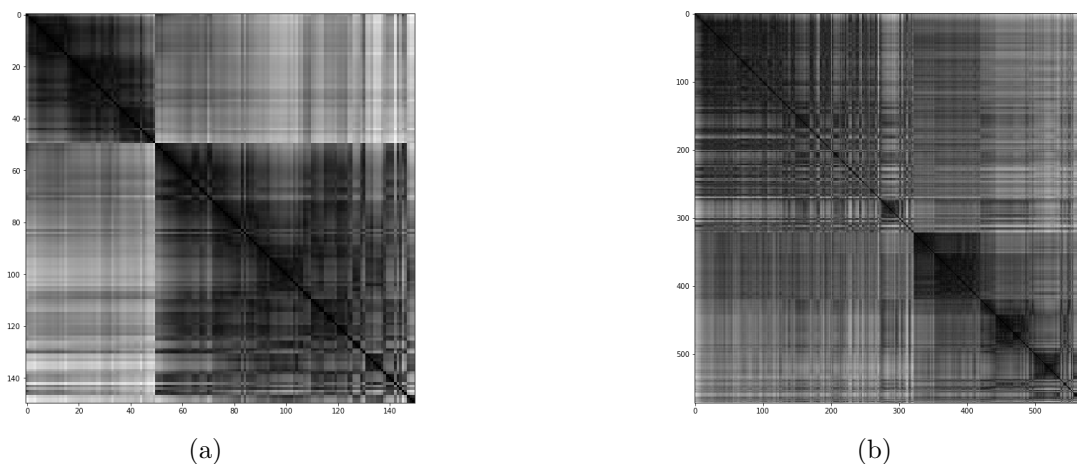


Figura 2.4: Resultado de aplicar VAT a las bases de datos de (a) Iris y (b) Olivas.

Para escoger el número de clústeres se suele probar el algoritmo y medir su rendimiento para averiguar cuál es el número de agrupaciones que da mejores resultados. Hay algoritmos que, en vez de utilizar el número de clúster, utilizan otro parámetro como condición de parada, por ejemplo, el número mínimo de elementos por clúster. Incluso, a veces, no necesitan ninguna condición para parar de hacer divisiones.

En cuanto a la correcta separación de los clústeres se puede utilizar distintas evaluaciones, aunque es una labor complicada ya que la tarea de agrupamiento es muy subjetiva. Por ejemplo, si se tienen las etiquetas, puede que la separación no se ajuste a ellas. Es por esto por lo que no se puede afirmar que un algoritmo sea mejor que otro en general, sino que es el mejor en cierto problema con ciertos parámetros.

Cuando se cuenta con las etiquetas se pueden usar las medidas utilizadas en aprendizaje supervisado. Aunque hay que tener en cuenta que las etiquetas no tienen por qué coincidir con el identificador que el algoritmo le haya dado a un clúster. Para solucionar esto se puede catalogar como correcto a la clase que más elementos tenga en un clúster y mal al resto. Pudiendo incluso crear una matriz de confusión.

Si no se tiene las etiquetas se puede utilizar otras medidas como la distancia entre distintos clústeres y entre los elementos dentro de cada clúster. Amat (2017)

[14] reúne distintas fórmulas que miden estas distancias. Para la distancia **entre elementos de un clúster** se puede calcular la distancia de cada par de valores de cada clúster en el conjunto de agrupamientos  $S$ :

$$\text{in-clúster} = \sum_{C \in S} \frac{\sum_{c_i, c_j \in C / c_i \neq c_j} d(c_i, c_j)}{\|C\| \cdot (\|C\| - 1)}$$

Si el algoritmo proporciona representantes de cada clúster se puede calcular la distancia a estos representantes y reducir el computo. La **distancia entre clústeres** se puede calcular como la suma de todas las distancias entre todos los pares de cada par de clústeres:

$$\text{inter-clúster} = \sum_{C_1, C_2 \in S / C_1 \neq C_2} \frac{\sum_{c_i \in C_1, c_j \in C_2} d(c_i, c_j)}{\|C_1\| \cdot \|C_2\|}$$

También se puede calcular con la distancia mínima en vez de la suma de distancias:

$$\text{inter-clúster} = \sum_{C_1, C_2 \in S / C_1 \neq C_2} \min_{c_i \in C_1, c_j \in C_2} d(c_i, c_j)$$

Por último, si se tienen los representantes de cada clúster, se puede calcular la distancia entre estos. Normalmente cuando una de estas medidas baja la otra sube, por lo que se busca un compromiso entre ambas.

Otras medidas que se pueden utilizar sin datos externos son:

Índice Dunn: Junta la distancias entre clústeres y dentro de clúster y el objetivo es maximizarla haciendo los grupos más compactos o maximizando la distancia entre clústeres. Su fórmula es:

$$D = \frac{\text{Distancia Mínima entre clusteres}}{\text{Distancia Máximo intra clúster}}$$

Hay que tener en cuenta que al tomar los máximos y mínimos quedará influenciado por los clústeres con peores resultados.

Coefficiente Silhoutte: mide la distancia de todo punto a las agrupaciones cercanas respecto a su propio clúster. Los valores de esta medida van de  $-1$  a  $1$ . Si es alto está alejado de cualquier distribución. Si es cercano a cero significa que, el punto, está en el borde entre dos grupos. Si es negativo es probable que se haya equivocado de clúster. En la Ecuación 2.1,  $i$  es el punto en el que se mide,  $a(i)$  es la distancia media de todos los puntos del clúster al punto medido y  $b(i)$  es la distancia media a los puntos del otro clúster. Basándose en distintos artículos y pruebas, Rodriguez et al. (2016) [15] afirman que Silhouette es la mejor medida para evaluar particiones sin información externa en la mayoría de los casos.

$$S_i = \frac{b(i) - a(i)}{\max(b(i), a(i))} \quad (2.1)$$

Otra forma de evaluar el rendimiento de una separación es mediante mediciones subjetivas. Una parte importante de este grupo es la interpretación de las separaciones, donde el usuario experto, intuye como se han separado los datos y el por qué. Según los resultados que ofrezca el algoritmo, este tipo de mediciones, resultan más o menos fiables. Aunque, estos análisis tengan gran valor y a veces es la única manera de medir el rendimiento de un algoritmo de clustering es preferible combinarlo con las anteriores formas de medición.

La tarea de comparar algoritmos de clustering no es sencilla. Al haber tantos y ser tan diferentes entre sí se hace muy complicado su comparación. Estivill-Castro (2002) [16] da unas recomendaciones para comparar algoritmos de clustering entre las que destacan:

- No se puede olvidar que el usuario es el que, en gran medida, da sentido al agrupamiento.
- La primera comparación entre algoritmos de clustering se debe hacer mediante la calidad de una solución, bajo la misma función objetivo y utilizando el mismo conjunto de datos.
- Que un algoritmo tenga una mayor puntuación en una medida determinada de comparación, no quiere decir que su solución sea la más válida.
- La calidad de los clústeres, para una tarea determinada, se puede medir con información externa, por ejemplo, utilizando bases de datos para aprendizaje supervisado

## 2.3. Problemas

Ya se ha hablado sobre algunos problemas de los algoritmos de clustering como la dificultad de su validación o el gran número de posibilidades que existen a la hora de elegir un algoritmo y sus parámetros. También se ha hablado de algunas características buscadas en los algoritmos de clustering, como la tolerancia a outsiders o su escalabilidad, pero ningún algoritmo es perfecto en todas a la vez, sino que buscan un consenso entre ellas. En esta sección se hablará de manera más extendida de una de estas características, poder procesar los datos de alta dimensionalidad. Aunque primero se hablará de los problemas que surgen de la utilización de distancias.



### 2.3.1. Problemas derivados de la distancias

Los algoritmos que utilizan distancias se enfrentan a problemas que se derivan del propio funcionamiento de las mediciones. Por ejemplo, se puede tener más presente un parámetro que otro solo porque sus distancias son más grandes, eliminando así información que puede ser útil para la separación.

Una solución es escalar y centrar los datos, con distintas técnicas de normalización. La elección de un método puede determinar el éxito o fracaso del algoritmo. Una mala elección puede distorsionar los datos o hacerlos aún más dependiente a los datos de los outsiders.

Algunos métodos de normalización son:

- **MinMax:** Con rango de 0 a 1]. El problema de esta normalización es su dependencia del ruido.

$$X_n = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Standard Scaler o Zscore:** Utiliza la media y la desviación típica de los datos. Su rango no está definido, y aunque deja de ser dependiente del ruido, tiene problemas con los outsiders.

$$X_n = \frac{X - X_{media}}{X_{SD}}$$

- **Escalado Robusto:** El escalador robusto es menos dependiente a los outsiders y escala los datos según la diferencia entre el primer y el tercer cuartil.

De nuevo, no existe un método perfecto de normalización y se deben probar varios o conocer bien los datos para seleccionar el método que distorsione los datos lo menos posible.

### 2.3.2. Datos de alta dimensionalidad

Los algoritmos de agrupamiento pueden enfrentarse a varios problemas cuando se utilizan con bases de datos de altas dimensiones (en el Apéndice A.1 se explica el término *dimensión*). Algunos de los problemas que causan este tipo de bases de datos son:

1. Cuantas más dimensiones más complicado es entender cómo se dividen los datos, y hacen imposible su visualización.
2. Los datos se encuentran más separados y las medidas de distancia son cada vez menos precisas.

3. No todos los atributos tienen por qué ser útiles para dividir los datos, y el tener más atributos puede hacer que no se encuentren esas soluciones fácilmente.

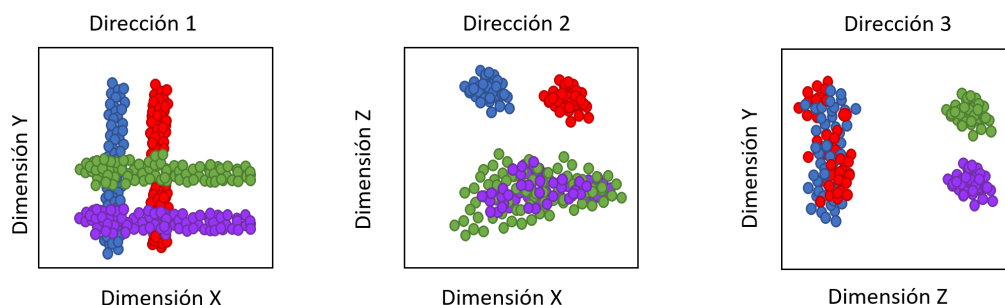


Figura 2.5: Ejemplo de la utilidad del Subspace Clustering.

En la Figura 2.5 representa un ejemplo de la importancia de tener en cuenta las dimensiones en la separación. Los datos están sobre un espacio de 3 dimensiones (X, Y y Z). Las imágenes muestran perspectivas formadas por cada par de ejes. Se observa como en la gráfica referente al par de ejes X e Y, no se pueden separar los datos. Además, que los elementos que forman el cuadrado estén tan cerca puede confundir a varios algoritmos de clustering. En cambio, en la perspectiva de X y Z, se pueden separar los elementos azules y rojos. Mientras que en la dimensión formada por los ejes Z e Y, se puede separar los elementos morados de los verdes. Si se utilizan los algoritmos tradicionales sobre todo el espacio, podrán confundirse al tener una alta densidad de datos de clases distintas en las 3 perspectivas.

Para solucionar los problemas que causa la alta dimensionalidad existen varias familias de algoritmos, una de ellas es el clustering de subespacios (en el Apéndice A.1 se explica el término *subespacio*). Esta familia de algoritmos busca encontrar un conjunto de dimensiones adecuado para dividir los datos. Se pueden dividir entre los que buscan subespacios paralelos a los ejes y los que están orientados arbitrariamente. También entre los que empiezan a buscar en las dimensiones más pequeñas (1-D, 2-D...) y los que parten desde la dimensión de los datos. El algoritmo con el que se experimentará en este trabajo y que se explica en la Sección 3.4 coge ideas de esta familia de algoritmos. El algoritmo se basa en la búsqueda de direcciones, que, en realidad, son subespacios.

## 2.4. Ejemplos de Algoritmos de Clustering

### 2.4.1. K-Means

K-Means es un algoritmo que se basa en la búsqueda de centroides. Un centroide es un punto que representa un clúster, los elementos pertenecerán al clúster que tenga un centroide más cercano. El algoritmo comienza determinando de forma aleatoria estos centroides en el plano y agrupando a cada elemento con el más cercano. Luego, se mueven los centroides a la posición media de los elementos que pertenecen al clúster definido por ese centroide. Se repetirá hasta que las posiciones de los centroides converjan. K-Means es sensible a la elección inicial de los centroides, pudiendo converger en un mínimo local, por eso se suele ejecutar varias veces el algoritmo. El objetivo del algoritmo es, entonces, minimizar la distancia de los ejemplos a los centroides:

$$\min \sum_{C_i} \sum_{x_j \in C_i} \|x_j - \mu_i\|_p^p$$

Donde  $\mu_i$  es el centroide. La norma  $\|\cdot\|$  está relacionada con la distancia para espacios con propiedades vectoriales (se puede ver más información sobre estos espacios en el Apéndice A). Normalmente se utiliza la distancia Euclídea minimizándola con la media aritmética:

$$\min \sum_{C_i} \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

Donde esta vez  $\mu_i$  es esta vez la media aritmética de cada grupo.

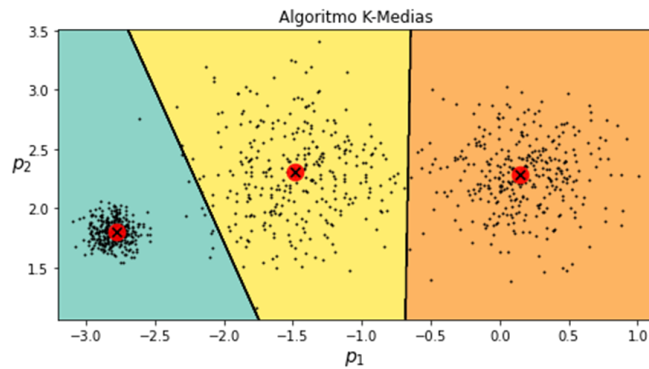


Figura 2.6: Ejemplo de Ejecución del Algoritmo K-Means en unos datos con 3 nubes de puntos con un ruido diferente. Las marcas rocas corresponden a los centroides. Las divisiones están marcadas con líneas negras y los clústeres con distintos colores.

La complejidad del algoritmo de K-Means depende de: N (el número de pun-

tos),  $K$  (el número de clústeres),  $I$  (el número de iteraciones hasta alcanzar la convergencia o parar) y  $D$  (el número de atributos):

$$O(nkId)$$

Si  $k$  no se fija, es decir, que el algoritmo debe buscar el número de clústeres que mejor separen los datos, provoca que el problema se convierte en NP. Esto hace que uno de los parámetros del K-Means deba ser el número de clústeres.

### 2.4.2. Otros

En la comparación de nuestra herramienta con otros algoritmos se utilizan los proporcionados por Sklearn (n.d) [17] en sus paquetes `cluster` y `mixture`:

- **Mini Batch K-Means:** Utiliza pequeños subconjuntos de datos escogidos aleatoriamente en el K-Means original. Es mucho más eficiente por lo que puede hacer más ejecuciones en el mismo tiempo, aunque los resultados obtenidos, al utilizar menos datos pueden ser algo peores.
- **Affinity Propagation:** Utiliza medidas de similitud entre pares de puntos hasta que progresivamente se encuentran cadenas de puntos similares.
- **Agglomerative Clustering:** Junta puntos cercanos hasta que se llega al número de clústeres pedidos.
- **Mean Shift:** Trata de encontrar esferas con densidades uniformes. Para ello utiliza centroides que actualiza para que sea la media de todos los puntos de su clúster.
- **Spectral clustering:** Utiliza la búsqueda de vectores propios de una matriz hecha con las distancias entre puntos.
- **DBSCAN:** Busca áreas de alta densidad y las expande hasta que todo punto tenga su clúster. **OPTICS** es una variante de este algoritmo.
- **BIRCH:** Iterativamente crea un árbol e inserta los puntos en los nodos hojas que más se parezcan. Según su parecido y los parámetros del algoritmo se decide si crear un nuevo nodo o no. De la estructura del árbol se sacan los centroides de los clústeres.
- **Gaussian Mixture Model:** Utiliza una distribución gaussiana para aproximar la función de densidad del problema.



# 3

## Estado del Arte

En esta sección se explicará el algoritmo utilizado para los experimentos y alternativas relacionadas como el Maximum Marging Clustering. Además, se discutirá acerca de las variantes de MMC más modernas, alguna de las cuales se utilizará en la parte experimental. También se detallarán distintas experiencias de clustering interactivo, incluyendo los usos más característicos y ventajas de estas técnicas. Por último, se introducirá algunos conceptos importantes del algoritmo del Dr. Manuel Rubio y se explicará con detalle en que consiste. Por último, se resumirán las tecnologías utilizadas para realizar la herramienta.

### 3.1. Clustering basado en separación máxima

En esta sección se explicará el algoritmo Maximum Marging Clustering (MMC) publicado por Xu et al. (2005) [18]. Para comprender el funcionamiento de MMC primero hay que entender cómo funciona la máquina vector soporte.

#### 3.1.1. Maquinas Vectores Soporte

Las máquinas vectores soporte (SVM) son algoritmos para problemas supervisados (con etiquetas) en los que se trata de encontrar el plano que separe dos clases lo máximo posible.

En la Figura 3.1 se puede ver la solución que da una SVM a un problema

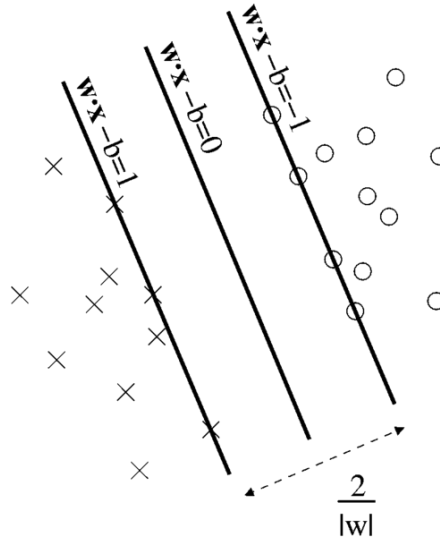


Figura 3.1: Definición de los márgenes en un problema de SVM. (AnAj, 2006) [2].

básico. Se tienen dos grupos de puntos: las cruces y los círculos; dos rectas que los limitan y una línea en el centro, la cual es la solución que se quiere encontrar ya que es la que separa ambos grupos de puntos con un mayor margen. Para encontrarla, se trata de maximizar la distancia que hay entre una de las líneas que limitan las clases con la central:

$$d(l_r, s) = \frac{|w \cdot x + b|}{w} = \frac{1}{w} \rightarrow \max \frac{1}{w} \rightarrow \frac{1}{2} \min w^2$$

Donde se ha elevado al cuadrado y multiplicado por  $\frac{1}{2}$  para simplificar las operaciones. Además, se debe añadir la siguiente restricción para no pasarse de los límites:

$$y_i(x_i \cdot w) \geq -1$$

Donde  $y_i$  será 1 o -1 dependiendo del grupo representado por  $i$ . Este problema se resuelve mediante métodos de programación no lineal.

Las maquinas vectores soporte solo permiten hacer cortes rectos, lo que limita su alcance para separar figuras complejas. Para solucionar esto, existen los kernels, los cuales se basan en la idea de inventar nuevas dimensiones según la distribución de los datos. Esto permite hacer cortes lineales en la nueva dimensión que separen los datos adecuadamente. En la Figura 3.2 se puede observar un ejemplo de cómo al aplicar un kernel a un conjunto que solo se podía separar mediante una división en forma de circunferencia, algo que no puede hacer una SVM, se transforma en un problema divisible mediante una recta, lo cual sí puede resolverse con máquinas

de vectores soporte.

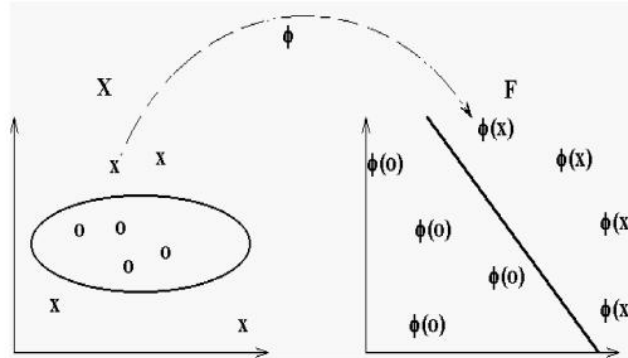


Figura 3.2: Ejemplo de aplicación de kernel en SVM (Elisfm, n.d.) [3].

### 3.1.2. Maximum Margin Clustering

Maximum Margin Clustering es otro método de clustering. Se basa en la idea de probar todas las posibles combinaciones de particiones de los datos y coger la que más separe ambas clases. Su funcionamiento es parecido a ejecutar máquinas de vectores soporte a cada posible partición. En las imágenes de la Figura 3.3 se puede ver cómo funciona esto. En la primera foto este margen es minúsculo, en la segunda es cero, y es en la tercera donde el margen es máximo. Este método solo puede separar los datos en dos clases. Para resolver este algoritmo haría falta probar con todas las posibles separaciones de los datos, lo que hace de este algoritmo un problema exponencial, y solo válido para problemas muy simples. Esto se puede acelerar añadiendo la búsqueda de la etiqueta o la pertenencia de la clase a la fórmula de la SVM. Además, se añade una restricción al problema de programación no lineal para que no vayan todas las instancias a un solo grupo. Aun así, sigue siendo demasiado lento para conjuntos de datos de tamaño medio.

A partir de aquí se proponen varias soluciones que relajan el problema presentado. Algunos algoritmos relajan la restricción de la convexidad para aumentar la velocidad. Esto provoca que se puedan estancar en mínimos locales y obliga a ejecutar el algoritmo varias veces para tratar de encontrar el mínimo absoluto. Un ejemplo de este tipo es el que se propone en el propio artículo en el que se publicó por primera vez MMC (Xu et al., 2005) [18]. Mientras que otros algoritmos tratan de mejorar el tiempo del algoritmo original utilizando otras técnicas matemáticas y reduciendo el espacio de búsqueda.

De este último tipo, es el algoritmo propuesto por Li et al. (2009) [1] que se utilizará en la parte experimental. En esta propuesta se divide el problema de minimización en subproblemas más pequeños que resuelven con máquinas de vector soporte. Además, el código del algoritmo es de uso libre para propósitos



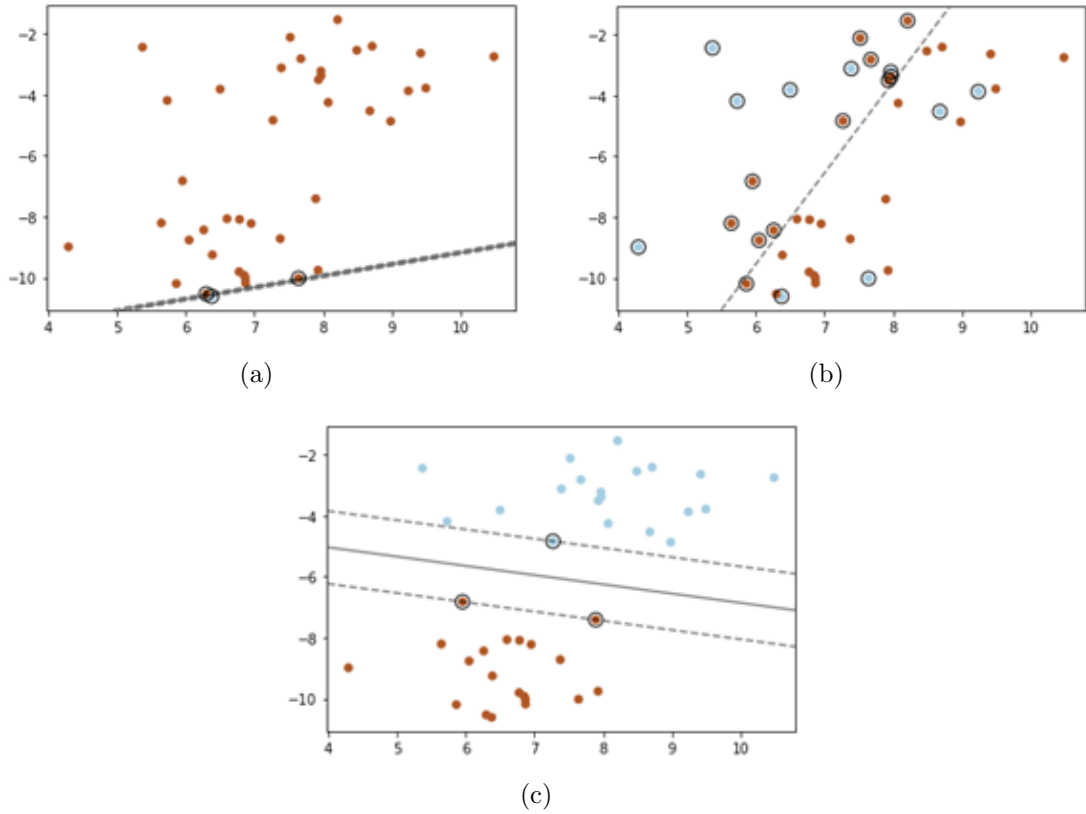


Figura 3.3: Ejemplo de Maximum Margin Clustering.

académicos y se puede encontrar en la página web del grupo de investigación de uno de los autores del artículo (Li, n.d.) [4].

## 3.2. Clustering mediante cortes con hiperplanos

Esta sección describirá el algoritmo de publicado por Rahimi & Recht (2004) [19]. Este algoritmo está relacionado tanto con el Maximum Margin Clustering, ya que trata de encontrar la máxima separación entre dos nubes de puntos y el subspace clustering ya que lo hace dividiendo el espacio con hiperplanos (en el Apéndice A.1 se explica el término *hiperplano*) que pasen por el origen, es decir, subespacios. De forma intuitiva, este algoritmo trata de encontrar el hiperplano que maximice la distancia de todo punto al hiperplano. Para que el hiperplano pase a través de los puntos se estipula que la suma de las distancias de cada punto al plano sea 0, de esta manera se compensaran los valores a la derecha del plano con los de la izquierda.

En la figura 3.4 se observa un ejemplo del algoritmo. Se ve como se calculan las distancias del punto  $x_i \in X$  al hiperplano,  $\pi$ , definido por su normal,  $\vec{n}$ , con

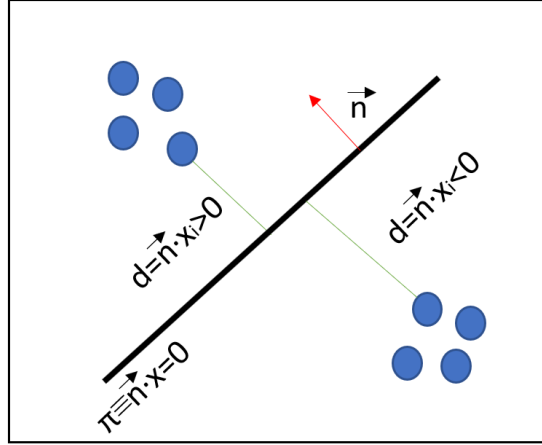


Figura 3.4: Ejemplo de separación mediante corte con hiperplano en 2 dimensiones.

la fórmula  $d(x_i, \pi) = \vec{n}^T \cdot \mathbf{x}_i$ . Por lo que el objetivo es:

$$\text{máx} \sum_{i=1}^N \vec{n}^T \cdot \mathbf{x}_i$$

A esta fórmula, se le añaden las restricciones antes mencionadas y elementos que permiten que el cálculo con derivadas para el problema de optimización sea más sencillo:

$$\begin{aligned} \text{máx} \frac{1}{N} \sum_{i=1}^N \frac{1}{\cos \theta_i} (\vec{n}^T \cdot \mathbf{x}_i)^2 \\ \text{s.a.} \quad \|\vec{n}\| = 1, \\ \vec{n}^T \cdot \mathbf{x}_m = 0 \end{aligned} \tag{3.1}$$

Donde  $\theta_i$  es el ángulo formado por el vector  $\mathbf{x}_i$  y el vector medio (de todos los vectores de  $\mathbf{X}$ ),  $\mathbf{x}_m$ .

En el propio artículo del algoritmo, se preguntan como de bien funcionará, teniendo en cuenta sus limitaciones. Limitaciones como que no todo conjunto de datos es divisible por un hiperplano o que el hiperplano debe pasar por el origen. A lo que ellos mismos se contestan con que al trabajar con espacios de alta dimensión, las divisiones no son rectas o planos, como se podría pensar en clave de dos y tres dimensiones. Cuando se amplían los parámetros, los hiperplanos pueden separar grandes conjuntos de datos con formas que en dos o tres dimensiones se catalogarían como complejas.

### 3.3. Clustering Interactivo

Se hará un breve resumen de la encuesta realizada por Juhee et al. (2020) [20] en la que revisan 105 artículos científicos sobre algoritmos de clustering que tienen algún tipo de interacción con un humano. En el texto se pone de manifiesto algunas de las razones por las que se añaden estas partes interactivas a los algoritmos de clustering:

**Mejora de la calidad de la agrupación:** La agrupación es un problema muy subjetivo y complicado por lo que para encontrar soluciones adecuadas que satisfagan al usuario es necesaria la combinación de las fortalezas humanas y automáticas. Este tipo de herramientas permiten, a través de una interfaz gráfica, modificar los parámetros iterativamente, seleccionar funciones de distancia a minimizar o dar más importancia a ciertos atributos. Entre este tipo de algoritmos se encuentra una herramienta en forma de árbol que describen en el artículo como “ [...] una representación visual del árbol de agrupamiento con fines de refinamiento y validación, ya que, según su conocimiento, los usuarios aceptan o rechazan particiones ” (p. 5). Algo parecido a lo que se mostrará en el algoritmo del Dr. Manuel Rubio en la próxima sección. Estas mejoras dan por hecho que la interacción de un experto puede mejorar la agrupación y suelen ser útiles cuando los problemas son muy complicados.

**Comprensión de la división:** En algunos casos es más importante saber por qué se ha hecho una división que la propia separación. Estas herramientas tratan de mostrar cómo se han partido los datos. Una parte importante de este tipo de herramientas son las de visualización de la solución, representando con distintos colores los clústeres, reduciendo la dimensionalidad para poder pintar en 2-D o 3-D... También, se pueden hacer varias divisiones y dejar que sea el usuario decida cual se ajusta más a sus intereses o utilizar varias técnicas estadísticas que permitan entender la división hecha.

Se dan dos puntos principales para la interacción humana, dentro de los algoritmos de clustering: La que se da **Al terminar el algoritmo**, a modo de corrección o de entendimiento de los resultados. Y la que se da para la **Selección de parámetros**. La mayor parte de los algoritmos de clustering tienen parámetros. Seleccionarlos mediante la comprensión de los datos y del algoritmo y no a través de prueba y error, hace que se llegue más rápido a las soluciones óptimas lo cual se puede hacer a través de herramientas interactivas.

## 3.4. Algoritmo Interactivo de Clustering Jerárquico de Manuel Rubio

### 3.4.1. Medidas de Bimodalidad

El algoritmo se basa en las medidas de bimodalidad inventadas por el Dr. Manuel Rubio. Durante el texto se referirá a ellas como  $\phi$  (*phi*) y  $\psi$  (*psi*). No se hablará sobre ellas ya que aún no están publicadas. Además, se utilizará una medida llamada Bimodality Coefficient cuyo código se puede encontrar en el artículo de Freeman y Dale (2013) [21] y durante el texto se nombrará como  $\beta$  (*beta*). Estas medidas son funciones de momentos estadísticos que aportan valores numéricos dentro de un rango en el que uno de los extremos será la bimodalidad perfecta (dos valores) y en el otro una distribución con un único valor.

### 3.4.2. Algoritmo

El algoritmo propuesto por el Dr. Manuel Rubio, divide en dos los datos. El procedimiento es encontrar una perspectiva, representada por un vector dirección, el cual debe pasar por el origen, que al proyectar los puntos sobre el vector la bimodalidad de la distribución que se forma sea máxima. En el Apéndice A.1 se explica qué es una proyección. Al no ser convexas las funciones de bimodalidad se puede optimizar la medida mediante la iteración de ejecuciones del algoritmo de descenso o ascenso de gradiente (que se explica en el Apéndice A.2.1). En el momento en el que se pare la búsqueda, se mostrará al usuario un histograma (Apéndice A.3.2) con la distribución de los datos proyectados sobre la dirección que tuviese la medida máxima. Será el usuario el que parta en dos, la distribución.

La dirección seleccionada da mucha información interesante sobre la separación ya que marca la perspectiva en la que se están mirando los datos. De esta manera se puede saber cuáles son los atributos más importantes para una separación, pudiendo conseguir información valiosa y de mayor fiabilidad para los análisis subjetivos.

En la Figura 3.5 se puede ver la representación del algoritmo. En (a) se puede ver un conjunto de datos de ejemplo. En (b) se elige una dirección aleatoria y se proyectan los datos sobre esta dirección (c). Luego se calcula el histograma de la distribución en (d). Como se puede ver el valor de la bimodalidad es muy bajo. El algoritmo continúa avanzando un paso determinado hacia donde se reduce la bimodalidad en (e) y se vuelve a calcular el histograma en (f). La bimodalidad es muy alta y en la próxima iteración es menor por lo que el algoritmo de descenso de gradiente termina. El usuario separa en dos el histograma en (g) y se separa el conjunto de datos con una recta perpendicular a la dirección que pasa sobre el

punto seleccionado en (h). La dirección será el vector normal de esta recta, o del plano en el caso de que fuese 3D.

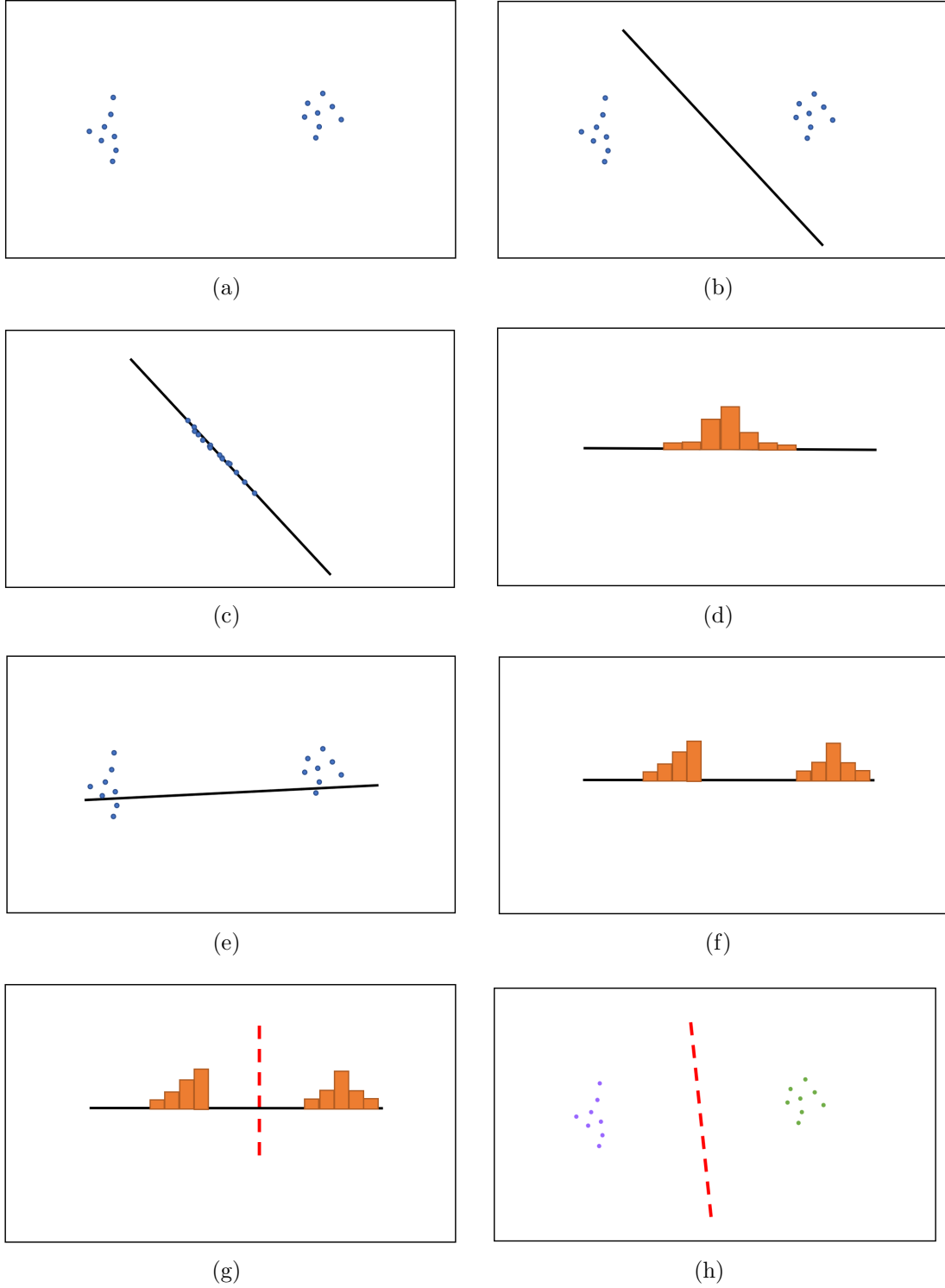


Figura 3.5: Representación del Algoritmo.

### 3.4.3. Comparación con otros algoritmos

El plano que busca Maximum Marging Clustering tendrá medidas de bimodalidad serán muy altas por lo que también lo buscará el algoritmo del Dr. Manuel Rubio. Aunque también permite soluciones donde ambas distribuciones de clases se encuentren superpuestas (muchas de las implementaciones del MMC lo permiten también). Frente a las soluciones convexas, este algoritmo hace una búsqueda iterativa que puede no llegar a ser nunca el mínimo absoluto. Aunque, frente a esto, está abierto a un mayor número de posibles soluciones y es una solución más rápida permitiendo ejecutarse muchas veces en un tiempo proporcional al tamaño del conjunto de datos.

Comparado con el algoritmo de corte con hiperplanos, se consigue liberar casi todas sus restricciones, como que la distancia de los puntos al hiperplano deba ser uno para que el hiperplano no quede fuera del conjunto de datos. Simplemente, no se le permite al usuario encontrar una recta fuera del histograma. Por otro lado, los hiperplanos no deben ser subespacios (solo las direcciones), por lo que la separación no tiene que ser a través del punto de corte.

### 3.4.4. Varianza

Si se busca separar solo en base a la bimodalidad se puede caer en elegir direcciones en las cuales los datos estén muy juntos y haya un salto no representativo entre dos nubes de puntos. Por eso es importante añadir la varianza (término descrito en el Apéndice [A.3.1](#)) al análisis.

### 3.4.5. Interactividad

La interactividad ya está presente en el algoritmo en el corte del histograma. Aunque se pueden añadir otras herramientas interactivas que faciliten la utilización del algoritmo por parte del usuario. Por ejemplo, para separar más de un clúster, se añade un árbol que permite una partición iterativa y jerarquizada de los datos. Así como, una frontera de Pareto que tenga en cuenta en la optimización tanto la medida de bimodalidad como la varianza a la hora de separar los datos. Incluso se puede introducir la dirección con más varianza para completar la frontera de Pareto. Esto se puede lograr con los valores propios de la matriz de covarianza que describe la varianza entre los elementos del conjunto de datos.

En total, hay tres interacciones con el usuario. Primero, el corte del histograma, donde la interacción no ocurre al terminar el algoritmo, se hace dentro de él, y tampoco es una selección de parámetros, por lo que queda fuera de la clasificación realizada por Juhee et al. (2020) [20]. El árbol y la selección del punto de la Frontera de Pareto permiten al usuario modificar los parámetros del algoritmo

de manera intuitiva y visual, el primero aumentando el número de clústeres o encontrando divisiones hasta que tengan un número mínimo de elementos; y el segundo permite al usuario elegir las condiciones de la división según la varianza y la medida de bimodalidad.

### 3.4.6. Desventajas

Al proyectar sobre una recta se pierde gran parte de la información geométrica. Además, las divisiones se hacen mediante hiperplanos por lo que no es un buen método para dividir distribuciones complejas. Otro aspecto negativo es la dificultad de kernelizar el método ya que al proyectar sobre el espacio kernelizado se perdería información del espacio original.

## 3.5. Tecnologías

En esta sección se resumirán las tecnologías utilizadas para realizar la herramienta web asociada al algoritmo de la sección anterior.

- **Python** es un lenguaje interpretado, multiparadigma y centrado en la legibilidad del código frente a la eficiencia. Numpy es uno de sus módulos matemáticos que da soporte al cálculo de matrices y vectores de manera eficiente.
- **Matplotlib** es un paquete de funcionalidades que permiten generar gráficos y tiene soporte para Numpy.
- **TKinter** es una de las bibliotecas para realizar aplicaciones de escritorio básicos en Python con soporte sencillo de Matplotlib, al contrario que sus competidores (Qt o WxWidgets)
- **Numba** es un paquete para Python que convierte el código en instrucciones máquinas acelerando su ejecución.
- **Bokeh** es una librería que genera visualizaciones interactivas utilizando JavaScript y HTML. Bokeh permite embeber los gráficos en páginas web a través de código JavaScript.
- **Plotly Python** es una biblioteca de visualización de gráficos, se parece a Bokeh, pero tiene un mayor soporte y se pueden generar componentes de Plotly Python con componentes React. Está hecha a partir de **Plotly React** y **Plotly.js**, que son bibliotecas de su respectiva plataforma, que permite visualizar los mismos gráficos que en Python. A través del paquete **Dash** se puede conectar Plotly con Flask, para realizar aplicaciones web

con las visualizaciones de Plotly además permite integrar código JavaScript dentro de nuestras aplicaciones Python.

- **Flask** es un framework web para Python que permite realizar aplicaciones web de manera sencilla. No es robusto y no tiene muchas extensiones, pero se puede utilizar para aplicaciones básicas.
- **React** es una biblioteca JavaScript para hacer de manera sencilla aplicaciones de una página. React se basa en componentes facilitando la interacción ellos.





# 4

## Descripción Informática

Como el algoritmo es interactivo, será necesario una interfaz que permita al usuario interactuar con la herramienta y llevar a cabo las acciones descritas en la Sección 3.4. Cuando existe interacción con el usuario es adecuado tener en cuenta la usabilidad y la accesibilidad, teniendo un desarrollo centrado en el usuario, por ejemplo, utilizando estándares, añadiendo guías y distintas gamas de colores o haciendo entrevistas con usuarios. Como se mencionará en [Conclusiones y trabajos futuros](#), este aspecto quedará como línea futura, centrando este trabajo en los requisitos funcionales. A pesar de ello, se seguirá una metodología basada en la descrita por Smith (2017) [22], para la realización de la herramienta. Esta metodología es incremental y se basa en componentes, lo que facilita los desarrollos que se puedan dividir en partes definidas y permiten la reutilización de código y el escalado de la herramienta.

En primer lugar, se buscarán los requisitos que emanan del propio algoritmo y de las recomendaciones del tutor. Se seguirá con la etapa de análisis donde se hará la especificación de estos requisitos y su descripción a través de casos de uso. Luego se harán distintos prototipos en los que se verá de forma general el aspecto de la herramienta que se desea implementar a través de mocks, y se elegirá la tecnología a usar en el desarrollo mediante la realización de prototipos de partes de la herramienta con las distintas posibilidades mostrando ventajas y desventajas de las tecnologías.

En la etapa de diseño se mostrará la dinámica de la ejecución del algoritmo interactivo a través de un diagrama de actividad. Se hará un diseño centrado en componentes. Se describirá la aplicación a través de ellos. Por último, se enume-

rarán los servicios que ofrece la API Flask.

Para terminar, se detallará el funcionamiento del prototipo final a través de un ejemplo de uso de la herramienta. Aunque no se detallará en las siguientes secciones, el proceso para llegar al diseño final se hizo de manera incremental a través de la realización de prototipos y de evaluaciones periódicas con el tutor. Después de estas evaluaciones se añadieron nuevos requisitos, que se incluyeron en su mayoría en el prototipo final. Otros se han dejado como línea futura y se recogen en el Capítulo 6.

## 4.1. Estudio Previo

El algoritmo estaba desarrollado en Matlab por el Dr. Manuel Rubio. En este apartado se describirá el código entregado por el tutor. También se repasarán las componentes principales que deben añadirse para realizar la herramienta interactiva asociada al algoritmo.

En primer lugar, se entregó el código necesario para calcular las medidas de bimodalidad y sus respectivos algoritmos de ascenso o descenso de gradiente. Para que el vector dirección vaya hacia un óptimo se utiliza la búsqueda en línea que da pasos en la dirección que devuelve el gradiente. En este caso da pasos cada vez más lentos. Por otro lado, para actualizar los valores aleatorios se utiliza el algoritmo de búsqueda de variables vecinas. Este algoritmo cambia cada vez más partes del vector objetivo por valores aleatorios lo que permite dar saltos cada vez más grandes y alejados del punto original en búsqueda del óptimo absoluto.

Para desarrollar la herramienta interactiva es necesario recubrir el algoritmo desarrollado por el tutor con distintos componentes:

- **Carga de datos:** El primer paso es cargar los datos. Serán necesarios conocer ciertas propiedades para adaptar el algoritmo al conjunto de datos. Por ejemplo, si tiene información de la clase habría que quitarla antes de introducir el conjunto en el algoritmo. También hay que normalizar los datos y adaptar la interfaz a la cabecera.
- **Frontera de Pareto:** Para hacer una división de unos datos se necesita una gráfica que permita al usuario observar la frontera de Pareto de los puntos que genere el algoritmo y seleccionarlos en base a la varianza y la medida de bimodalidad. El algoritmo creará nuevos puntos hasta que el usuario decida pararlo con el botón de pausa. Estos puntos representan cada uno un hiperplano. Para iniciar el algoritmo será necesario conocer la medida de bimodalidad. Además, se debe poder cambiar la varianza por otra de las medidas de bimodalidad una vez el algoritmo termine.

- **Lista de Histogramas:** Cuando se seleccionan un punto se añade a una lista de gráficas el histograma de la proyección de los puntos sobre la dirección elegida. Para que el usuario pueda comparar los histogramas se debe poder ordenar la lista para juntar los elementos que más le interesen. Una vez haya decidido el usuario debe seleccionar uno de los gráficos, se le llevará a otra ventana que le permitirá separar los datos del mismo histograma.
- **Árbol:** Para que el usuario pueda hacer más de una división se le proporciona un árbol que permita crear una nueva partición cada vez que pulsa sobre un nodo. Una vez la división se haya llevado a cabo se marca como separado el nodo partido y se crean dos nuevos nodos no visitados, en los que si se el usuario los selecciona podrá separar una de las mitades separadas. Cuando el usuario lo pulsa se vuelve a ejecutar el algoritmo sobre esa porción de los puntos.

## 4.2. Análisis de Requisitos

### 4.2.1. Identificación de Requisitos Funcionales

#### *Requisitos generales:*

- RF1. El software debe permitir subir archivos .csv.
- RF2. El software debe preguntar al usuario si el csv tiene información de clase.
- RF3. El software debe preguntar en que columna está la información de clase.
- RF4. El software debe preguntar al usuario acerca de si el csv tiene cabecera.
- RF5. El software debe permitir al usuario escoger el método de normalización que se va a utilizar.

RF6. El software debe ser capaz de procesar el archivo csv cargado y normalizarlo.

RF7. El software debe poder descargar la solución en cualquier momento.

#### *Requisitos del componente árbol:*

- RF8. El software debe permitir al usuario presionar sobre los nodos hoja.
- RF9. El software debe permitir elegir al usuario la medida a optimizar.
- RF10. El software debe permitir elegir al usuario las medidas a calcular.
- RF11. El software puede iniciar el algoritmo.
- RF12. El software debe permitir al usuario observar la información del histograma usado en la separación de los nodos internos.

RF13. El software debe permitir crear dos nodos hijos cuando se separe un nodo.

*Requisitos del componente frontera de Pareto:*

RF22. El software debe poder pintar puntos de la frontera de Pareto.

RF14. El software debe permitir parar el algoritmo.

RF15. El software debe permitir al usuario elegir entre las distintas medidas ( $\phi$ ,  $\psi$ ,  $\beta$  y  $\sigma$ ) seleccionadas para calcular y representarlas en el eje y.

RF16. El software debe permitir al usuario presionar sobre los puntos generados por el algoritmo una vez haya terminado.

*Requisitos del componente lista de histogramas:*

RF17. El software debe ser capaz de generar histogramas a partir de los puntos de la frontera de Pareto.

RF18. El software debe permitir al usuario arrastrar los histogramas y moverlos de posición.

RF19. El software debe permitir al usuario eliminar los histogramas que desee.

RF20. El software debe permitir al usuario seleccionar histograma.

RF21. El software debe permitir al usuario separar el histograma que seleccione.

### 4.2.2. Casos de uso

#### Preparar Datos

En la Figura 4.1 queda representado el caso de uso de preparación del conjunto de datos. Que se descompone a su vez en los casos de uso del usuario en los que introduce si los datos tienen cabecera y solución y en que columna se encuentra. También se pedirá al usuario la normalización que desea hacer a los datos. Por su lado, el sistema procesará los datos antes de que se termine el caso de uso.

#### Ejecutar algoritmo

En la Figura 4.2 se pueden observar los casos de uso de ejecutar el algoritmo. El cual se compone de otros casos de uso: Seleccionar medida a optimizar, seleccionar medidas a calcula y pulsar sobre nodo hoja, iniciar el algoritmo y pintar punto.

#### Separación y nuevo histograma

La Figura 4.3 representa los casos de uso del sistema: generar histograma el cual incluye la selección de uno de los puntos de la frontera de Pareto. Por último, se permite

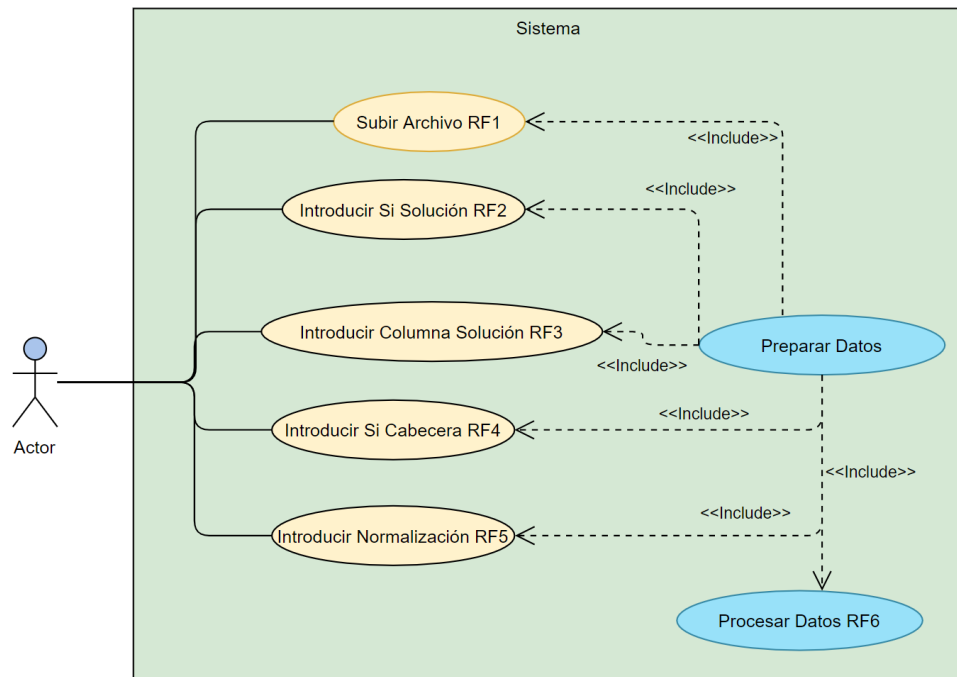


Figura 4.1: Diagrama de caso de uso de preparación de los datos.

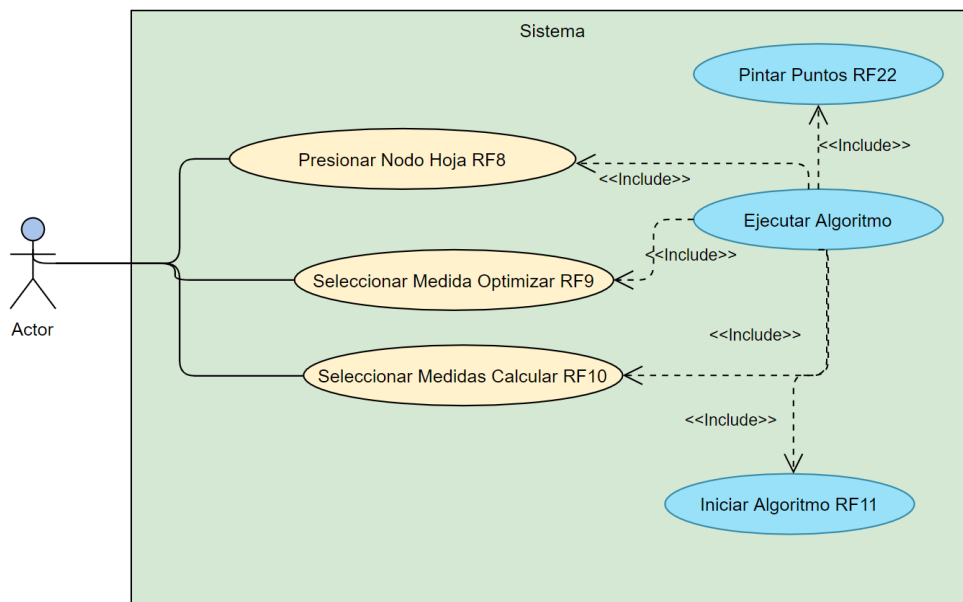


Figura 4.2: Diagrama de caso de uso ejecución del algoritmo.

la separación de los datos compuesta de seleccionar el histograma, que el usuario divida el histograma y se generen dos nodos hijos en el nodo de la separación.

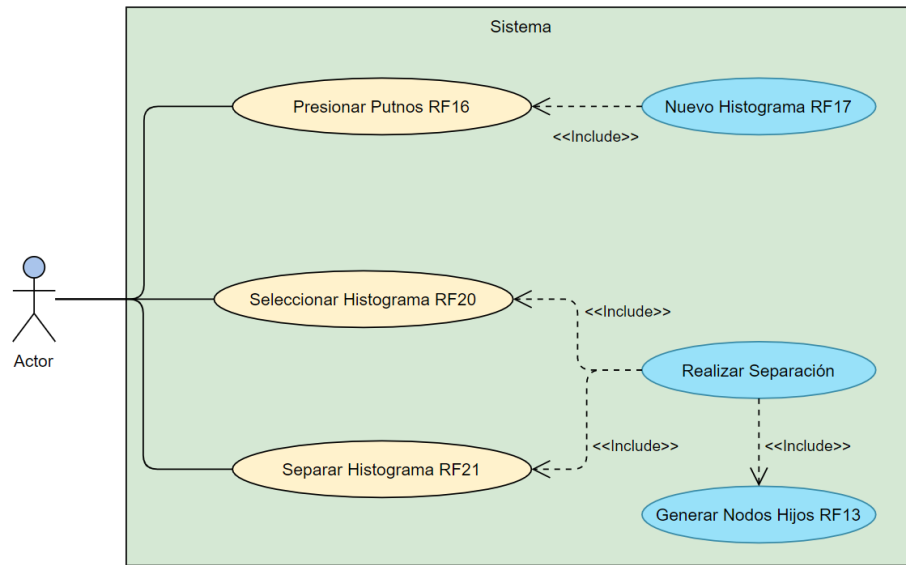


Figura 4.3: Diagrama de casos de uso de crear nuevo histograma y realizar separación.

### Otros

Por último, en la Figura 4.4 se puede ver que se permite al usuario arrastrar y eliminar histogramas, descargar soluciones y ver la información de los nodos internos.

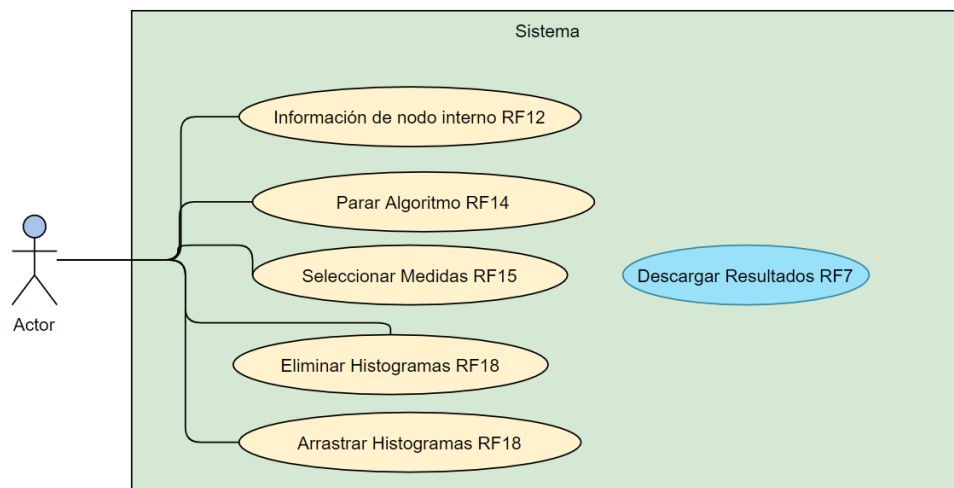
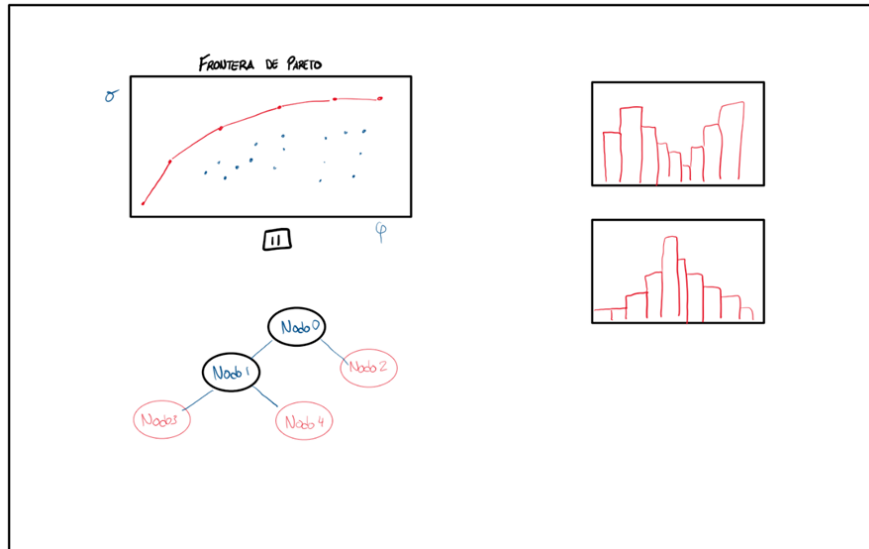


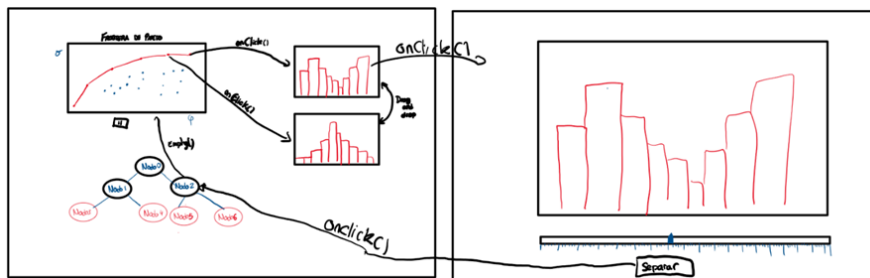
Figura 4.4: Otros casos de uso.

### 4.2.3. Prototipos

#### Mocks



(a)



(b)

Figura 4.5: Mocks de la herramienta (Fase de diseño).



## Prototipos de Partes de la Herramienta

Primero se intentó realizar la frontera de Pareto con **TKinter** y **Matplotlib** la cual se puede ver en la Figura 4.6 (a). Cuando se trato de realizar la lista de histogramas, surgieron problemas al no poder añadir gráficas de Matplotlib a las listas interactivas de TKinter. Se trató de añadir imágenes de los gráficos en su lugar pero el resultado no era muy satisfactorio (Figura 4.6 (b)).

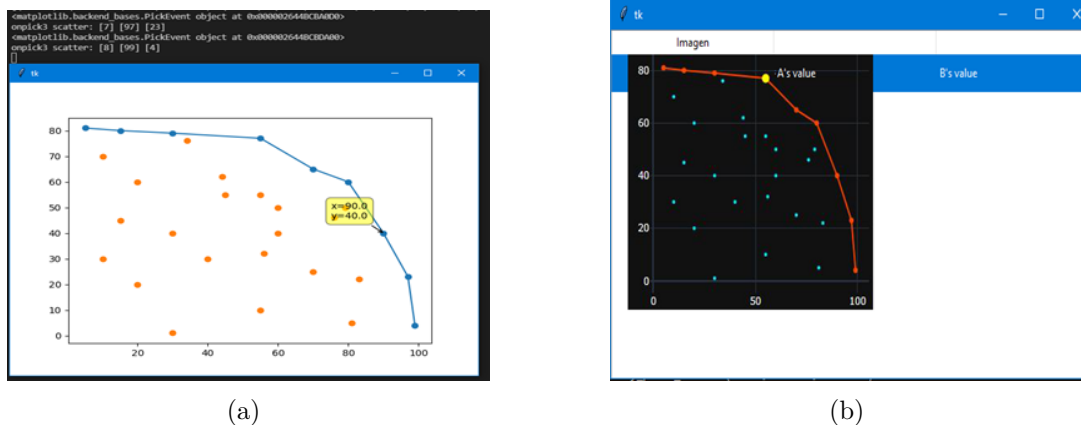


Figura 4.6: Prototipo en TKinter.

Una vez visto que los paquetes para desarrollo de aplicaciones de escritorio de Python no se ajustaban al desarrollo de la herramienta, se decidió probar con otras librerías que permitiesen pintar gráficas, pero que utilizarasen tecnología web por debajo. Se comenzó con **Bokeh** donde se realizó un prototipo que contaba con la frontera de Pareto, incluyendo interactividad al pulsar los puntos, alejar y acercar la gráfica, y presionar un botón, que sería el que pulsaría el usuario para pausar el algoritmo (Figura 4.7). Pronto se descubrió que **Plotly**, era una herramienta parecida pero tenía más posibilidades para añadir elementos web externos a las gráficas, lo que facilitaría la lista de histogramas y el árbol.

**Plotly** proporciona una aplicación web que utiliza React con un servidor Flask. A través de código Python se crean componentes React como gráficas o slidebars. Además, permite utilizar código de JavaScript para añadir algunas funcionalidades a la página web y crear componentes React propios para utilizarlos a través de Python. Como en los anteriores prototipos se realizó la frontera de Pareto. A través de código JavaScript se consiguió realizar un componente de carga de archivos y la lista de histogramas (Figura 4.8). En la realización de estas tareas se observó que tanto conectar el código de JavaScript con Python, como hacer componentes propios era demasiado costoso. Como se ha comentado anteriormente Plotly Python está hecho con un servidor web Flask conectado con React Plotly. La solución más factible, ya que las anteriores propuestas no habían sido lo suficiente permisivas para realizar la herramienta, era bajar un peldaño, y utilizar las tecnologías de Plotly React y Flask para hacer las conexiones entre Python y el frontend más cómodas.

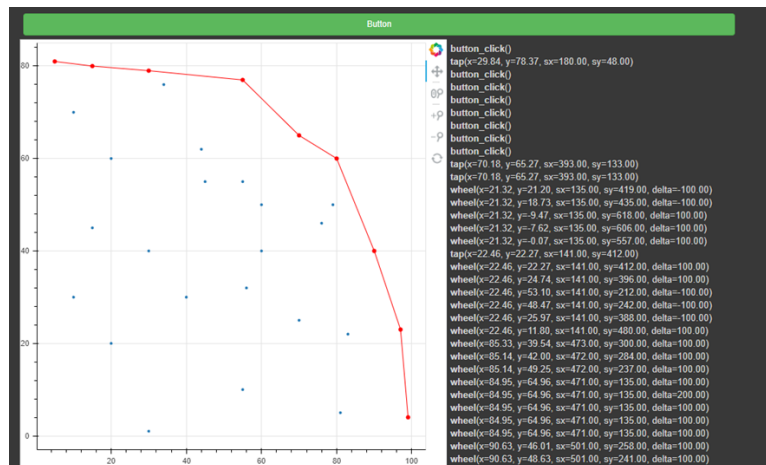


Figura 4.7: Prototipo en Bokeh.

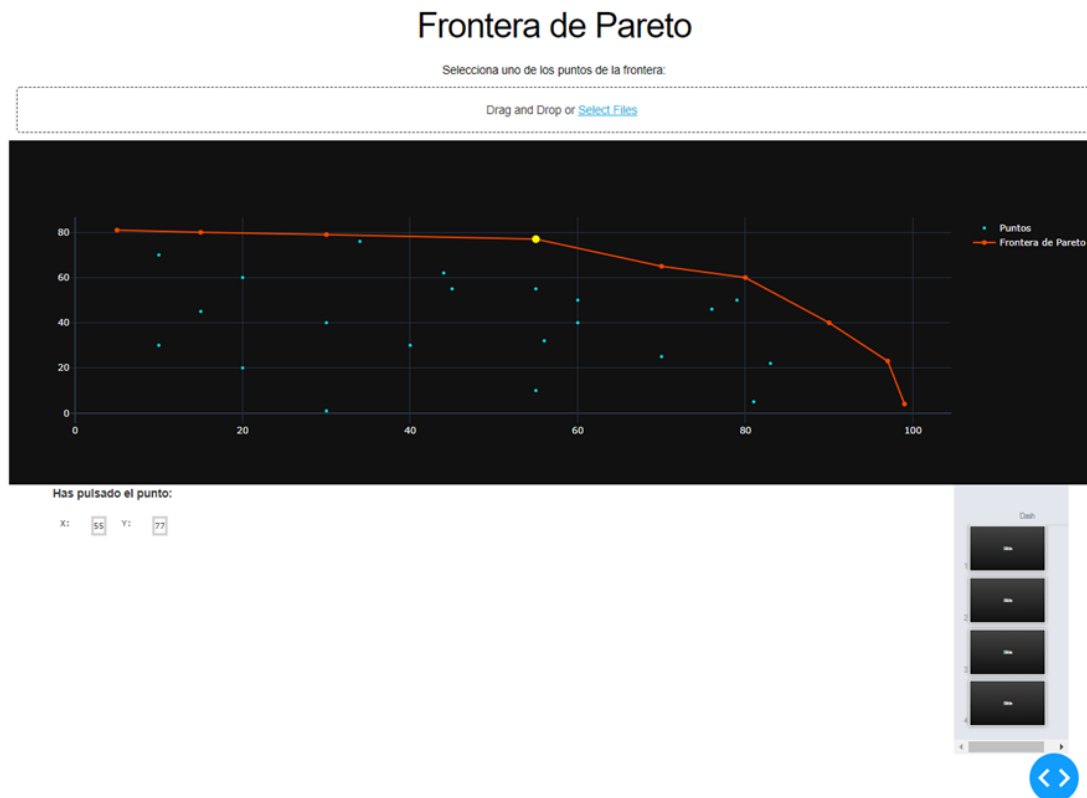


Figura 4.8: Prototipo Plotly.

## 4.3. Diseño

En esta sección se detallará la arquitectura software del prototipo final.

### 4.3.1. Diagrama de Actividad

El diagrama de la Figura 4.9 muestra el flujo de ejecución a alto nivel de la ejecución del algoritmo interactivo. De esta forma se puede observar cómo es el proceso dinámico de la ejecución de una parte de la herramienta.

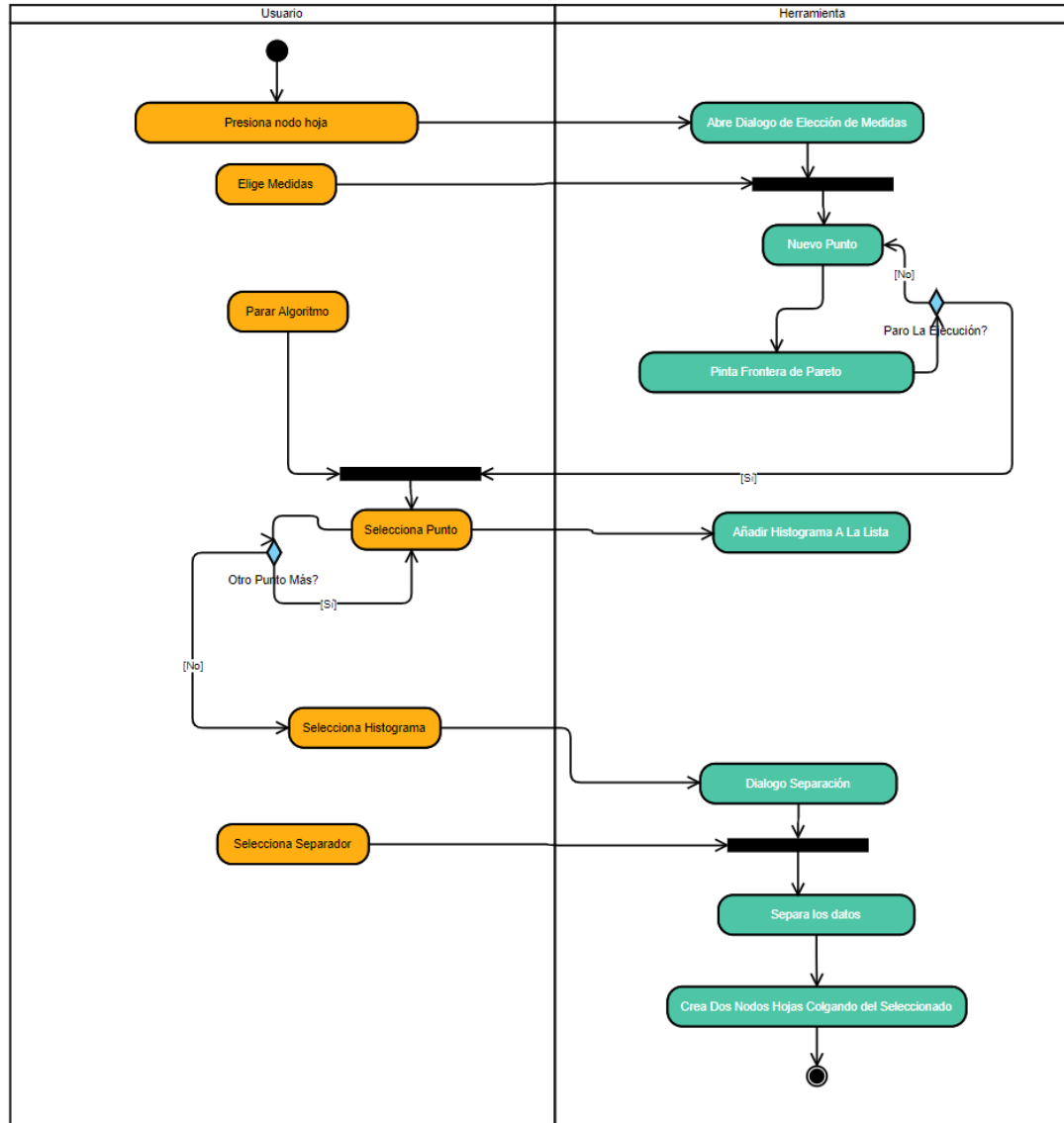


Figura 4.9: Diagrama de Actividad del algoritmo interactivo.

### 4.3.2. Diagrama de Componentes

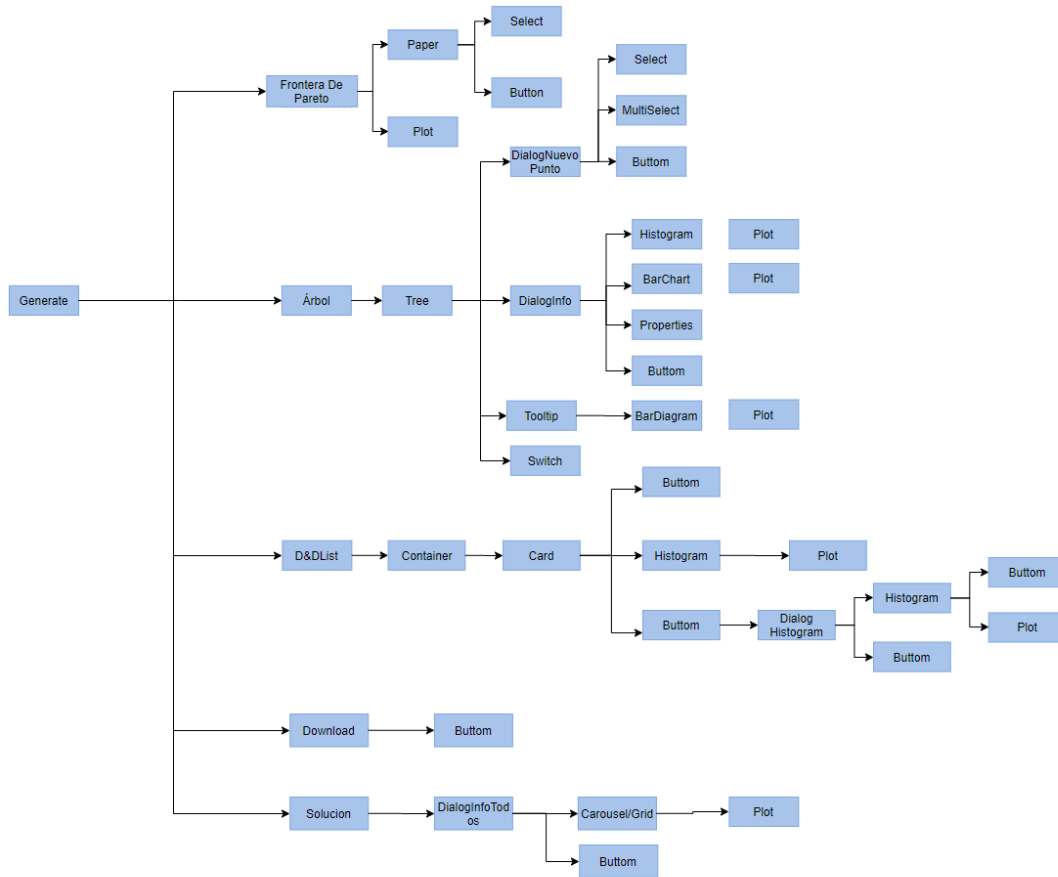


Figura 4.10: Diagrama de Árbol de Componentes de la Ventana del Algoritmo.

Se utilizarán componentes con licencia MIT para agilizar el desarrollo de la herramienta. En la ventana de carga de datos se ha utilizado MaterialUI para el formulario y React Dropzone Uploader para subir el archivo y enviarlo al servidor. Esta ventana está enrutada con Generate la ventana principal del algoritmo cuyos componentes están representada en la Figura 4.11. El árbol se ha creado con el componente React D3 Tree. Se han modificado sus nodos para incluir tooltips mediante el componente rc-tooltip y generar ventanas flotantes. En cuanto a los gráficos, todos se han realizado con React Plotly. La lista de histogramas se realiza con React DnD. Cada histograma tendrá un botón para eliminarlo y otro para abrir un cuadro de dialogo. El cuadro de dialogo tendrá el mismo histograma con un slider para separar los datos, un botón para confirmar la separación y otro para cerrar la ventana. Por último, en la parte inferior, se encuentran los botones de descarga, el cual envía al usuario el archivo con las soluciones del algoritmo, y de información de la separaciones, el cual abre una ventanas flotante

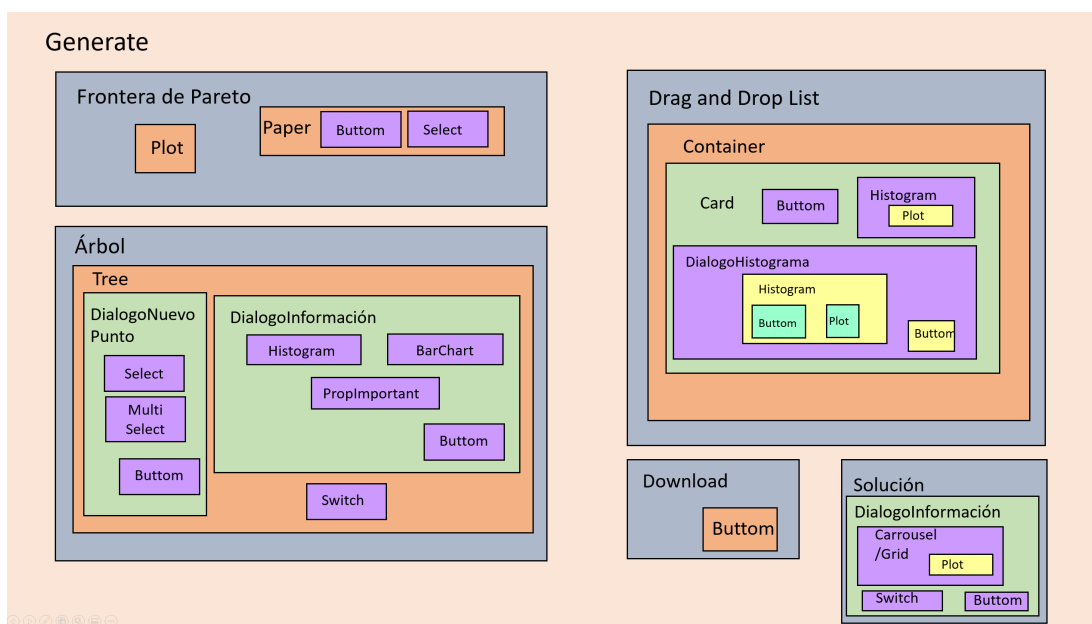


Figura 4.11: Componentes de la Ventana del Algoritmo.

que según presiones un switch muestra un grupo de gráficos en forma de cuadrícula o en forma de Carrousel. En la Figura 4.10 se puede ver un diagrama en forma de árbol de las dependencias de los componentes.

### 4.3.3. Representación de Interacción con la API mediante Diagramas de Secuencia

El servidor Flask funciona como una API. Desde React se hacen peticiones y se devuelven datos en formatos JSON:

- **uploadDB y iniciar:** Estas funciones se encargan de procesar el archivo subido por el usuario. Se espera que el archivo sea .csv separado por comas. También guarda la información introducida por el usuario en el formulario como si tiene cabeceras o la columna con información de clase. Por último, se encarga de normalizar los datos con el método elegido por el usuario. Funciones representadas en la Figura 4.12.
- **cargaDatos:** Devuelve información cuando se carga la ventana del algoritmo: el nombre del archivo, los headers o la columna con información sobre las clases. Función representada en la Figura 4.12.
- **nuevoPunto:** Esta función es la encargada de crear nuevos puntos ejecutando el algoritmo (el cual está acelerado con Numba). Con esos puntos encuentra la frontera de Pareto para encontrar un consenso entre la varianza y la medida seleccionada en la entrada. Función representada en la Figura 4.13 (a).

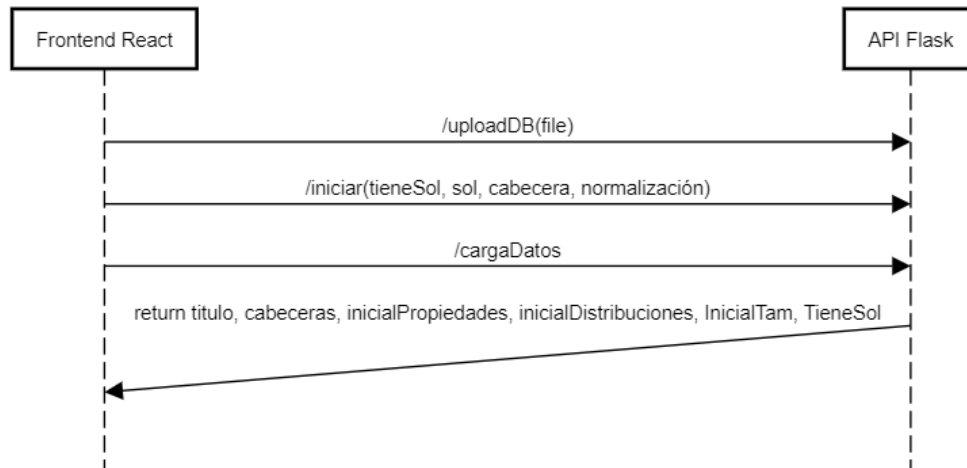


Figura 4.12: Diagrama de secuencia de la interacción entre frontend y backend al inicio de la aplicación (cargaDatos, uploadDB y iniciar).

- **CambioDeMedida:** Según la medida elegida se encarga de cambiar el eje coordenado de la y, entre la varianza y una de las medidas de bimodalidad calculando de nuevo la frontera de Pareto. Función representada en la Figura 4.13 (a).
- **nuevoHistograma:** Se devuelven los datos de la proyección correspondiente al punto que se selecciona en la frontera de Pareto. Función representada en la Figura 4.13 (a).
- **separacion:** Esta función es la encargada de separar los datos. Recibe el punto de corte seleccionado por el usuario en el histograma y la dirección. Función representada en la Figura 4.13 (a).
- **juntar:** Esta función es para eliminar parte del árbol cuando el usuario presione sobre la cruz en la parte derecha del nodo desde el que se quiera comenzar la eliminación recursiva. Función representada en la Figura 4.13 (b).
- **divisorAutomatico:** Esta función es para experimentación. Su objetivo es el de separar en dos un histograma automáticamente. Función representada en la Figura 4.13 (b).
- **resultadoNodoSolucion y resultadoSolucion:** Estas dos funciones se encargan de generar los gráficos de barras que evalúan las particiones si se cuentan con la información de clase. Funciones representadas en la Figura 4.13 (b).
- **download:** Se crea un archivo .csv con los datos normalizados y la columna de soluciones reales (si las tiene) y la columna de las soluciones predichas. Se ejecuta cuando se presiona el botón de descarga. Función representada en la Figura 4.13 (b).

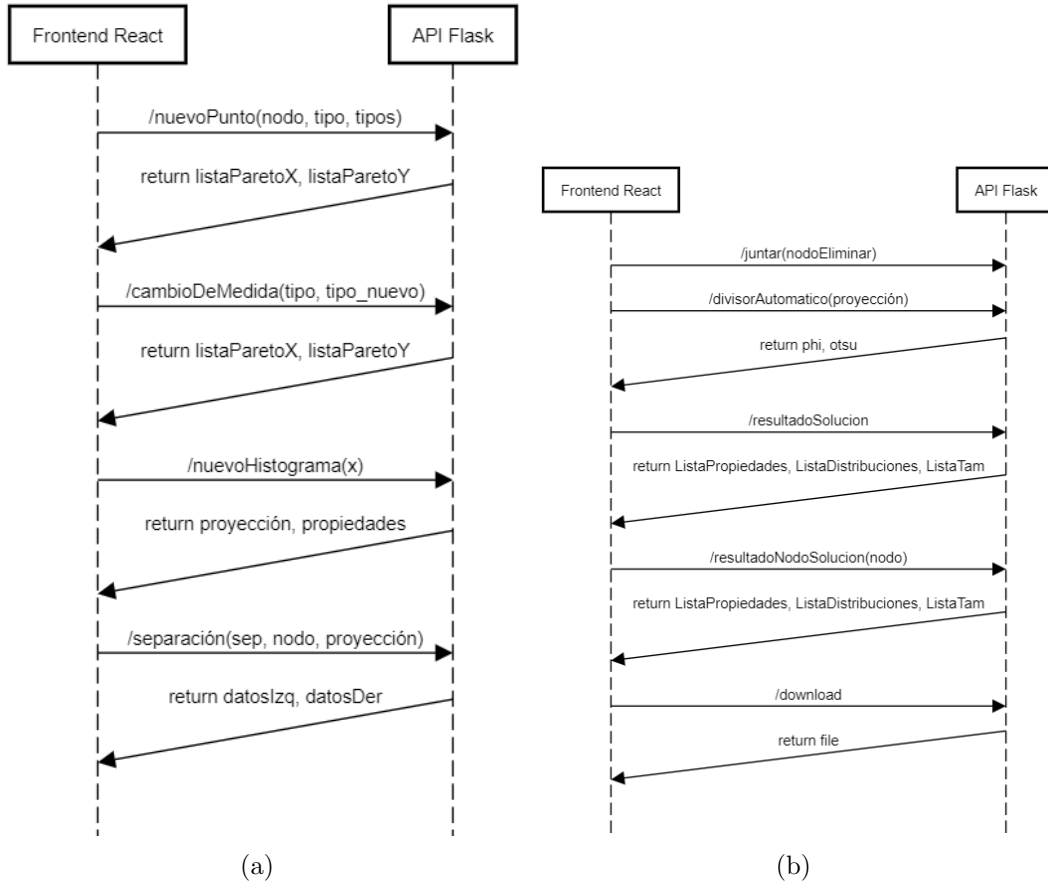


Figura 4.13: Diagrama de secuencia de la interacción entre frontend y backend: (a) ejecución del algoritmo (nuevoPunto, CambioDeMedida, nuevoHistograma y separación); (b) funciones auxiliares de la API (juntar, divisorAutomatico, resultadoNodoSolucion, resultadoSolucion y download).

## 4.4. Prototipo Final: Experiencia de usuario

Para iniciar la herramienta primero se debe iniciar el servidor Flask. Esto se puede hacer posicionándose sobre la carpeta *flaskServer* y ejecutando los comandos *virtualenv venv*, *venv/Scripts/activate* y *pip install -r requirements.txt*. Posteriormente, solo hay que ejecutar el programa *app.py*. La aplicación web React se puede cargar posicionándose sobre la carpeta inicial del proyecto y ejecutando los comandos *npm install* y *npm start*. Se podrá acceder a la herramienta a través de la url <http://localhost:3000/>.

Cuando inicie la aplicación el usuario se encontrará en la pantalla de carga (Figura 4.14). Donde se le pedirá que inserte un archivo .csv. También tendrá que rellenar un formulario con ciertos datos para adaptar la interfaz a la base de datos: si cuenta con cabecera, si tiene solución, en caso de tenerla en que columna se encuentra y finalmente, que normalización se utilizará entre las vistas en la sección 2.3.1. Una vez rellenos los campos y subido el archivo se puede continuar presionando el botón cargar (Figura 4.15).

Figura 4.14: Pantalla de carga de la herramienta.

Figura 4.15: Carga de Archivos de la herramienta.

Una vez subido el archivo, se redirigirá al usuario a la ventana del algoritmo (Figura 4.16). En la parte superior se observa información sobre el archivo: su nombre, el número de parámetros y elementos, así como cuantas particiones se han hecho hasta el momento. En la parte inferior derecha se encuentra un botón de descarga que ofrecerá al usuario un archivo .csv con el resultado hasta ese momento. Si tenía soluciones, se encontrarán en la penúltima columna, y las soluciones predichas en la última. También se devolverán los datos normalizados. Al lado se encuentra el botón de resultados, el cual solo aparece si los datos tienen información sobre la clase. Este botón ofrece la evaluación de las divisiones hechas hasta el momento a través de diagramas de barras. Se mostrarán en forma de carrusel o Grid según se presione el Switch de la parte inferior (Figura 4.17).

En la parte inferior se encuentra el **árbol** binario que se componen de nodos internos, con dos hijos, y los nodos hojas que representan las particiones hechas hasta el momento (Figura 4.18). Los nodos tienen 4 estados que se identifican con su texto de ayuda y su color. Los nodos hojas son azules y tienen el texto: “Empezar Partición”. Cuando se presionan se iniciará su separación mediante el algoritmo. Si el nodo es rojo



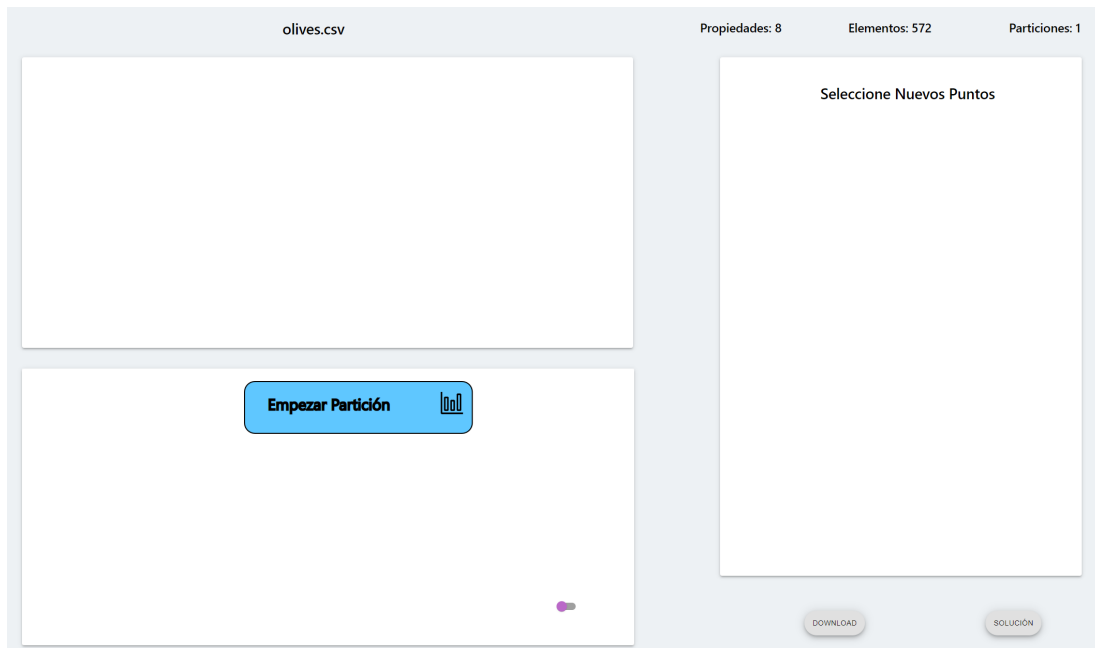


Figura 4.16: Ventana principal del algoritmo.

es que ya se ha separado y se puede eliminar pulsando a la flecha que hay en el lado derecho. En este tipo de nodo aparecen dos números, que corresponden al número de datos que se ha ido por cada lado. Cuando está amarillo y con el texto “*Pulsa OFF*”, es que el algoritmo se está ejecutando, aunque cuando el usuario decide parar de crear puntos, el nodo cambia su texto a “*Presiona Para Deseleccionar*”, y permite al usuario no seguir avanzando en la separación de ese nodo. Esto permite utilizar otra de las medidas o seguir con las divisiones con otro nodo. Cuando se pulsa sobre un nodo hoja se abrirá una ventana para seleccionar la medida que el usuario quiere utilizar para minimizar y cuales se tienen que calcular durante el algoritmo para comparaciones posteriores.

En la parte superior se encuentra la **Frontera de Pareto** (Figura 4.19). Al iniciar la partición, se empezarán a pintar los puntos pertenecientes a la frontera que se descubran en cada iteración del algoritmo. Solo se pintarán los que forman frontera de Pareto. Cuando el usuario decida terminar el algoritmo, presionará el botón de OFF y se le permitirá pulsar sobre los puntos del grafo para generar el histograma correspondiente a la dirección seleccionada. Además, se podrá cambiar la varianza por cualquiera de las medidas que el usuario seleccionó en el paso anterior.

En la derecha de la ventana se encuentra una **lista** que contendrá todos los histogramas seleccionados. Se pueden eliminar o mover de ubicación para ayudar en su comparación. Para que se pueda distinguir la varianza de los datos, todos comparten la misma porción del eje x. Es decir, los segmentos representados en los histogramas tienen la misma longitud en todas las direcciones seleccionadas. Cuando se añaden histogramas a la lista la ventana se parecerá a la representada en la Figura 4.20.

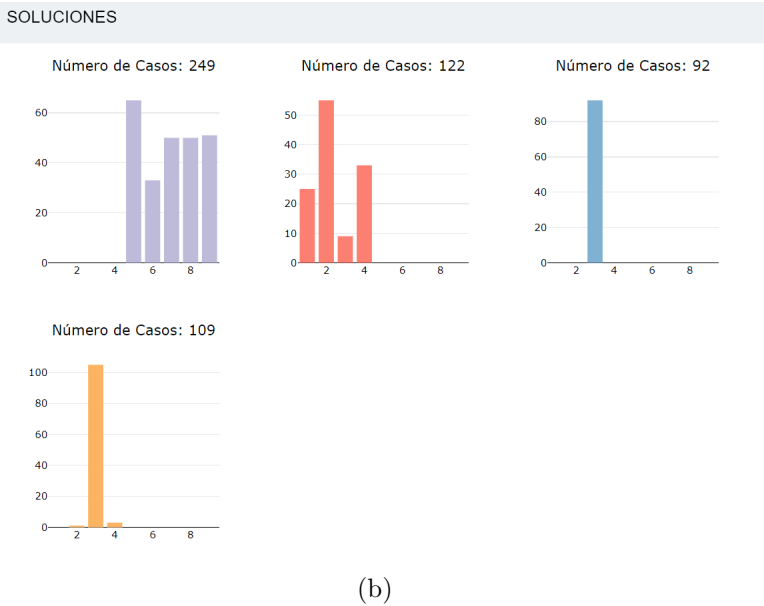
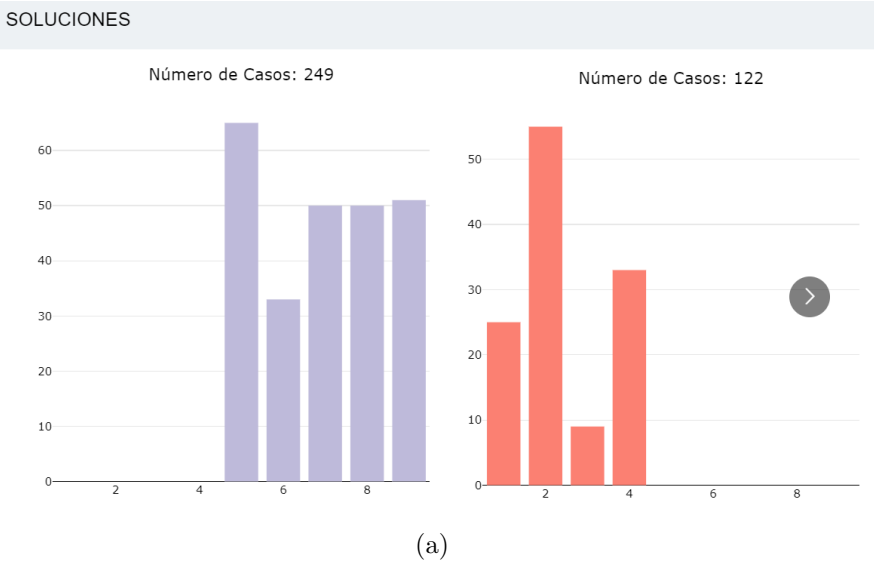


Figura 4.17: Información sobre las divisiones: (a) Carrusel, (b) Grid.

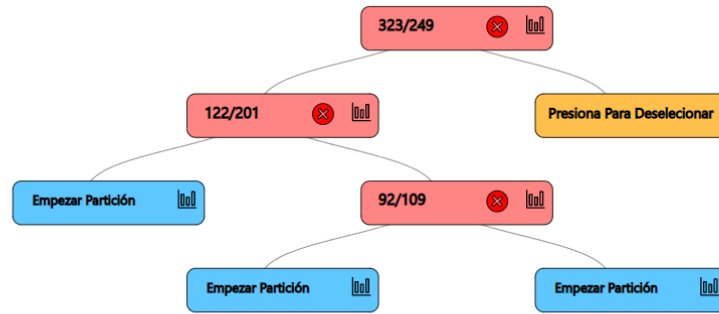


Figura 4.18: Árbol en estado de partición (nodo amarillo), con tres particiones ya hechas (nodos azules).

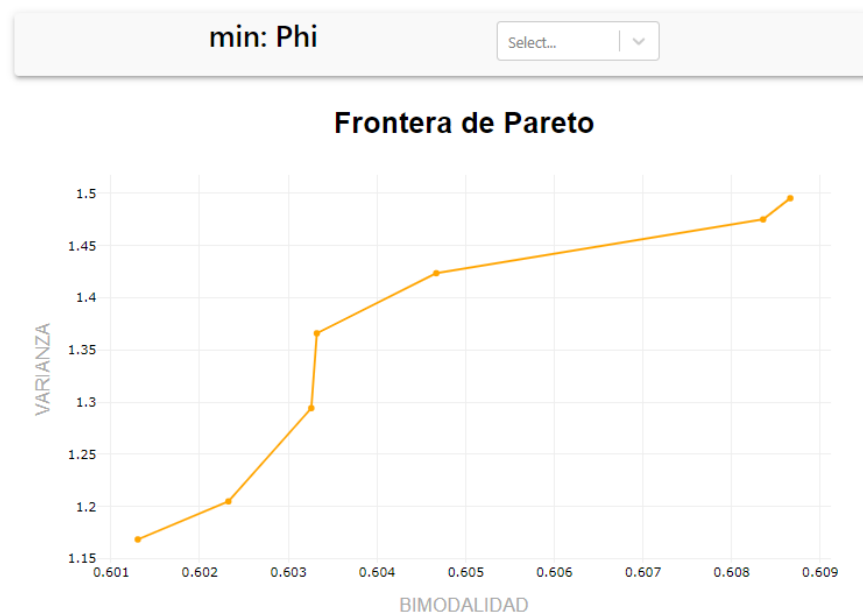


Figura 4.19: Frontera de Pareto al terminar de ejecutar el algoritmo.

El último paso es el de la selección del plano de separación (Figura 4.21). Se hará mediante un slider que marcará el punto de separación. Además, existe la opción de hacer una búsqueda automática del punto de corte (se estudiará la viabilidad de esta solución en la sección 5.6).

Si la base de datos seleccionada tiene solución se ofrecen varias herramientas visuales para evaluar los resultados de la evaluación. Estas evaluaciones se basan en histogramas que representan los elementos de cada clase que se hay en una determinada partición. Los histogramas de todas las separaciones se encuentran al pulsar en el botón *solución* en la parte inferior derecha y que se ha descrito anteriormente (Figura 4.17). Además, también se pueden encontrar esta información en los nodos hojas del árbol. Se pueden ver en formato tooltip o como ampliación del propio nodo según el estado del Switch

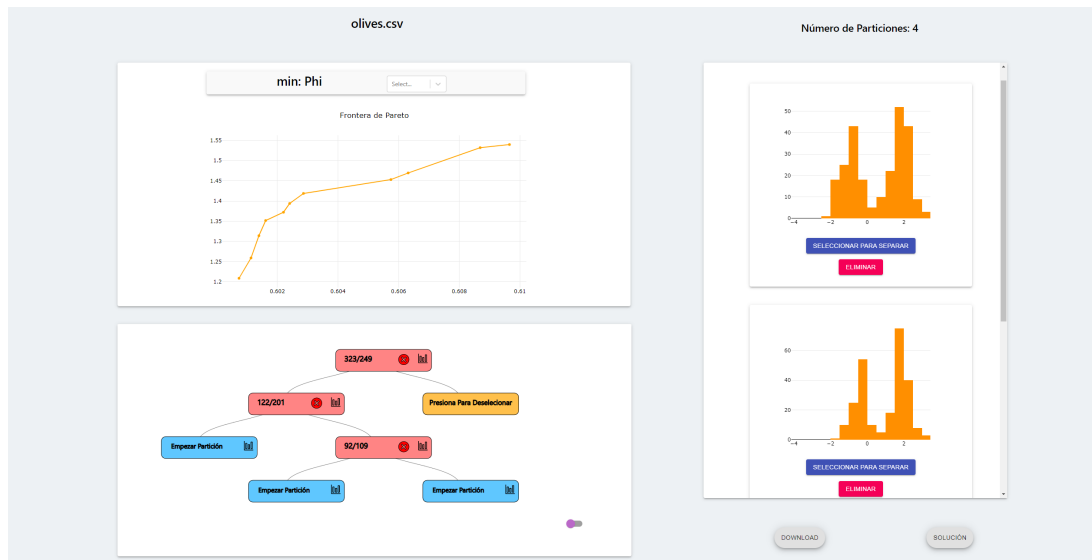


Figura 4.20: Prototipo en el estado de selección de los histogramas.

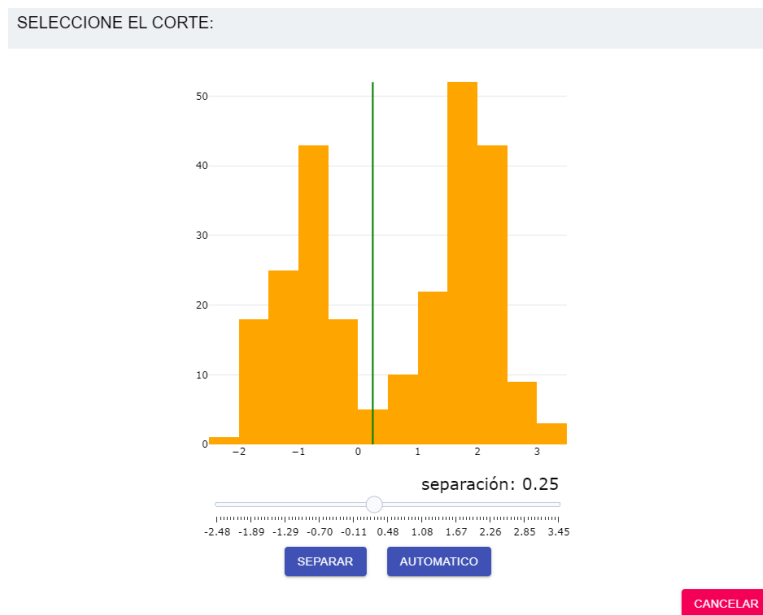


Figura 4.21: Separación del histograma.

de la parte inferior derecha del árbol (Figura 4.22).

En los nodos internos (los ya separados) hay información de la separación (Figura 4.23): El histograma de la dirección utilizada, la importancia de las propiedades en la separación y una gráfica que representa con colores los datos de cada clase real que se han ido a cada lado de la división.

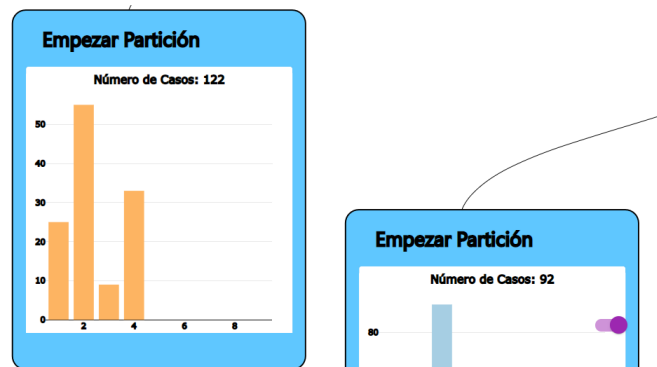


Figura 4.22: Información de las particiones en los nodos hoja.



Figura 4.23: Información de la separación en los nodos internos.

# 5

## Experimentos

En este capítulo se medirá el rendimiento del método de clustering del Dr. Manuel Rubio, el cual se denominará como Hierarchical Interactive Clustering (HIC). Se utilizarán alguna de las medidas vistas en la sección 2.2.5. Normalmente se usará el número de aciertos tomando la clase real máxima de cada separación como los elementos correctos, y el resto como incorrectos. Cuando sean únicamente 2 clases, se evitará que ambas separaciones puedan ser de una misma clase.

Una medida importante en la comparación de métodos de clustering es el tiempo. Al requerir, el algoritmo, de la intervención humana, el tiempo puede ser subjetivo. Dependiendo de la retroalimentación de la herramienta el usuario puede decidir estar más o menos tiempo buscando la mejor separación. Otra cuestión es que la herramienta se aprovecha mejor siendo experto en los datos: conociendo una aproximación de los números de elementos por clúster, conociendo el número de separaciones, sabiendo si los datos son más o menos separables... Durante el capítulo se tendrán en cuenta ambas cuestiones.

Se comenzará con un ejemplo, ejecutando el algoritmo sobre una base de datos con distintos tipos de aceites de oliva (Sección 5.1). De esta manera el lector podrá entender el método seguido en el resto de los experimentos. A continuación, se procederá a comparar las distintas medidas de bimodalidad, ejecutando el algoritmo con cada medida sobre la base de datos de aceites de oliva, y dando algunas claves descubiertas con las pruebas realizadas en el resto del capítulo (Sección 5.2). En las secciones 5.3 y 5.4 se comparará el algoritmo con los explicados en la sección 2.4.2 en bases de datos en 2D y en bases de datos reales, respectivamente. En la sección 5.5 se comparará con el método de MMC mencionado en la sección 3.1.2. Por último, se discutirá sobre la posibilidad de automatizar el algoritmo, y se medirá el rendimiento de la automatización de la división del histograma (Sección 5.6).

## 5.1. Prueba Inicial: Separación de Aceites de Oliva

Primero, se probará el algoritmo sobre la base de datos de aceites de oliva para dar al lector una idea del funcionamiento de la herramienta y de cómo se hicieron el resto de los experimentos. El primer paso en cualquier problema en el que se involucren datos es entenderlos y prepararlos para procesarlos. Se puede utilizar cualquier herramienta, aunque en este caso se utilizará distintas funcionalidades de los paquetes de Python: Numpy, Pandas y Matplotlib.

Primero, es conveniente observar ciertos aspectos como la cabecera, el número de datos o el tipo de información de cada propiedad. En la Figura 5.1 (a) se describe la base de datos mediante estas características sacadas con el método *info()* de los objetos Pandas. Se puede observar que la primera columna guarda información sobre la clase y que el resto de las columnas son numéricas. También, que hay 8 propiedades y 572 elementos.

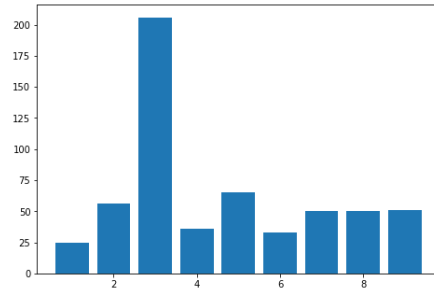
Al contar, el conjunto de datos, con las etiquetas y al ser el algoritmo un método de aprendizaje no supervisado, no se incluirá esta información en el análisis hasta que se haya realizado toda división, aunque se conocerá la información de la Figura 5.1 (b): un gráfico de barras que muestra el número de clases y el número de cada una, lo cual puede simular el conocimiento del experto del dominio. El archivo debe ser un .csv separado por comas, lo cual se puede conseguir con la función de pandas *to\_csv()*. Con esta información, junto a la elección de una técnica de normalización (*zscore* en este caso), se puede completar los datos pedidos en la pantalla de carga tal como se ve en la Figura 4.14.

```

RangeIndex: 572 entries, 0 to 571
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    sol         572 non-null    int64
1    palmitic    572 non-null    int64
2    palmitoleic 572 non-null    int64
3    stearic     572 non-null    int64
4    oleic       572 non-null    int64
5    linoleic    572 non-null    int64
6    linolenic   572 non-null    int64
7    arachidic   572 non-null    int64
8    eicosenoic  572 non-null    int64

```

(a)



(b)

Figura 5.1: Estudio de los datos: (a) Información de las propiedades, (b) Información de la columna solución.

El segundo paso es seleccionar el único nodo del árbol para comenzar con su partición. Se elige una de las tres medidas (en este caso  $\phi$ ) y se le da al botón de continuar. Aparecerán en la frontera de Pareto varios puntos que representan direcciones, donde el eje de abscisas representa lo bimodal que es la distribución de los datos proyectados sobre esa dirección, mientras que el eje de las ordenadas mide la varianza de los datos. Una vez se decide que es suficiente y se presiona el botón de "parar" se pueden selec-

cionar los puntos de la frontera de Pareto más interesantes. Se debe tener en cuenta que al contrario de lo que ocurre con las otras dos medidas, cuanto más pequeña sea  $\phi$  más bimodal será. En este caso, como se puede observar en la Figura 4.19 el valor de la varianza es bastante alto para los valores más pequeños de  $\phi$ , por lo que se pueden elegir los primeros.



Figura 5.2: Ejemplos de histogramas de la primera separación.

Por cada punto seleccionado aparecerá un histograma en la parte derecha. El usuario deberá seleccionar el que le parece más divisible. En la figura 5.2 se observan el ejemplo de dos histogramas. Se elegirá el primero, ya que hay una separación total. Una vez marcado el punto de corte, el usuario puede darle al botón de *separar* y encontrará que el árbol ha cambiado. Como puede verse en la Figura 5.3, ahora el nodo que seleccionó al principio es de color rojo y han aparecido dos nuevos nodos hojas que serán los próximos en dividirse. Se puede continuar este procedimiento hasta dividir en 9 los

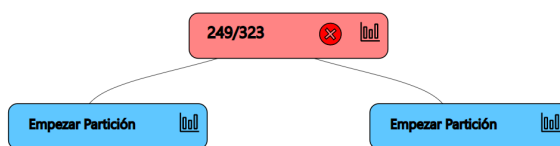


Figura 5.3: Estado del componente árbol cuando la separación ha sido realizada.

datos. En ocasiones las divisiones o la elección del histograma no serán tareas triviales y quedará a cargo del profesional del dominio el escoger de la manera más conveniente para cada problema específico. Esto hace que la herramienta sea muy versátil, pudiendo dividir los datos de muchas maneras diferentes, siendo el usuario el que decide como hacerlo. Esto se puede comparar con los parámetros de otros algoritmos, pero en este se cambian en tiempo real gracias a la interactividad de la herramienta.

Como los datos tienen información de la clase, se pueden ver los resultados pulsando el botón de información de la división. Los datos de la primea separación se pueden ver



en la Figura 4.23. La división entre las primeras cuatro clases del resto ha sido perfecta. En la parte inferior se encuentra la importancia de los atributos, donde se observa que la propiedad *eiconsenoic* ha sido la más importante a la hora de dividir. Esto proporciona información que puede ser valiosa: Las primeras cuatro clases tienen un valor de la propiedad *eiconsenoic* distinto de los del resto. Esto es una información que puede ser muy interesante para las personas del dominio. En el caso de que los datos sean la información clínica de pacientes y algunos de ellos padecen cierta enfermedad, separar los pacientes con la enfermedad de los que no la tienen en base a ciertas propiedades ofrecería una información que puede ayudar a detectar y conocer mejor la enfermedad.

Sobre esto se puede observar en la Figura 5.4, donde se grafican los valores de la propiedad *eiconsenoic* en contraposición a los datos de la clase, que existe una línea horizontal que divide ambos grupos de clases, mientras las 4 primeras están localizadas en los valores de *eiconsenoic* de cero a cinco, el resto de las clases se encuentran por encima del valor 10. De aquí se pueden sacar varias conclusiones. Primero, la división hecha tiene una base real comprobable; además, las particiones proporcionan información sobre los datos; y por último, este conocimiento se ha conseguido a través del propio método de clustering.

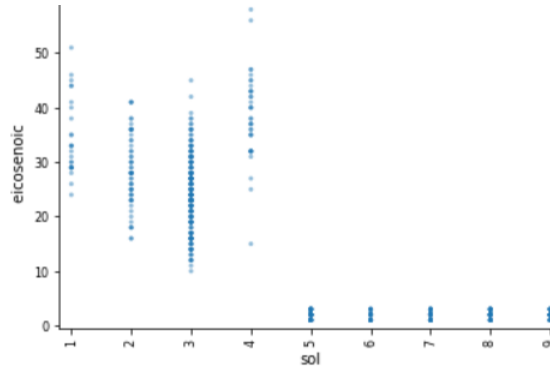


Figura 5.4: Gráfica de *eiconsenoic* frente a las clases.

Al ser la medición de rendimiento una tarea poco visual y que solo sirven para la comparación con otros algoritmos se ha dejado fuera de la herramienta por lo que se puede descargar la solución y utilizar la medida deseada. En este caso se utilizará la explicada en la introducción del capítulo. El resultado de la medida de rendimiento, para esta división de la base de datos de aceites de oliva, es del 92 % de acierto.

## 5.2. Experimento 1: Comparar las distintas medidas

En la tabla 5.1 se comparan los resultados del algoritmo, en la base de datos de aceites de oliva, utilizando las medidas  $\phi$ ,  $\psi$  y  $\beta$ . Se puede observar que los resultados de  $\phi$  y  $\beta$  son casi idénticos y los de  $\psi$  se quedan algo atrás.

| PHI  | BC   | PSI  |
|------|------|------|
| 0.92 | 0.93 | 0.85 |

Tabla 5.1: Porcentaje de acierto en olives con el algoritmo utilizando las distintas medidas de bimodalidad.

Después de los experimentos realizados durante el capítulo se ha comprobado que la medida  $\phi$  encuentra antes o después direcciones similares a las de  $\beta$  y normalmente alguna dirección con varianza más alta, que puede ser interesante. Las distribuciones que encuentran estas dos medidas suelen ser más bimodales que las que encuentra  $\psi$ , que, por su lado, permite encontrar direcciones que, aunque no sean tan bimodales tienen una mayor varianza. Estas direcciones pueden ser adecuadas cuando las direcciones más bimodales que encuentran las variables  $\phi$  y  $\beta$  no son la división adecuada para los datos. En cuanto al tiempo, la variable  $\psi$  es la más rápida,  $\phi$  es algo más lenta y  $\beta$  (al no poder acelerar parte del código con el paquete *Numba*) es la que más despacio va y no es recomendable utilizarla en bases de datos grandes.

### 5.3. Experimento 2: Conjuntos 2D

En esta sección se estudiará el comportamiento del algoritmo con distintas distribuciones de puntos artificiales en dos dimensiones proporcionadas por Scikit Learn developers (n.d.) [23]. En la Figura B.1 se pueden observar los resultados de la ejecución de varios algoritmos de clustering en las bases de datos artificiales, incluyendo el algoritmo del Dr. Manuel Rubio. Como se ha explicado con anterioridad el algoritmo presentado en este TFG está diseñado para datos separables por subespacios, en el caso de 2D solo se pueden separar mediante rectas. Existe la excepción teórica de que si los valores de la dirección son ambos cero el subespacio sería la coordenada en el origen y la proyección movería todos los puntos al (0,0), la división sería aleatoria, pero al no ser una distribución bimodal y tener muy poca varianza siempre tenderá a otros valores.

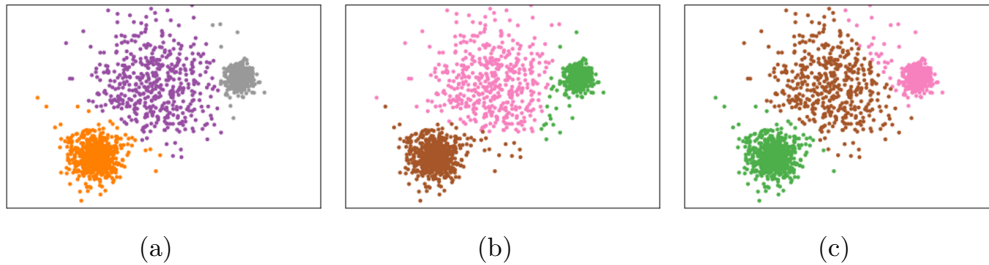


Figura 5.5: Gráficos con la separación de los datos: (a)  $\phi$ , (b)  $\psi$ , (c)  $\beta$ .

En la Figura 5.5, se observa la separación de los datos con las tres medidas, se observa que la de  $\phi$  (a) es la más adecuada, mientras que la de  $\beta$  (c) confunde algunos datos de la distribución central (color marrón), con la distribución superior (color rosa).

## 5.4. Experimento 3: Bases de Datos Reales

En esta sección se comparará el algoritmo iterativo con los algoritmos descritos en la sección 2.4.2. En la tabla 5.2 muestra información de las bases de datos utilizadas (parámetros y elementos totales) y el número de clases de cada una. En la tabla B.1 se encuentran los resultados obtenidos.

|                    | Parámetros | N    | Clases |
|--------------------|------------|------|--------|
| <i>Olives</i>      | 8          | 572  | 9      |
| <i>Iris</i>        | 4          | 150  | 3      |
| <i>Cancer</i>      | 8          | 699  | 2      |
| <i>AutoMPG</i>     | 6          | 392  | 3      |
| <i>Glass</i>       | 9          | 214  | 7      |
| <i>Mnist 2vs7</i>  | 784        | 2060 | 2      |
| <i>Mnist 3vs8</i>  | 784        | 1984 | 2      |
| <i>Echodiagram</i> | 132        | 131  | 2      |
| <i>Ionosphere</i>  | 34         | 351  | 2      |

Tabla 5.2: Características de las Bases de Datos utilizadas.

Se puede comprobar que el funcionamiento del algoritmo está por encima o igualando los mejores resultados de otros algoritmos en la mayor parte de las bases de datos. En las bases de datos de Glass y Autompg el rendimiento, aun siendo superior que en el resto de los algoritmos, no parece que sea una división adecuada. Esto puede deberse a que los datos no son separables, o que existe una división más intuitiva. En el caso del conjunto de datos de cristales, se puede ejecutar distintas medidas, vistas en la sección de 2.2.5, para comprobar la separabilidad de los datos. Por ejemplo, el coeficiente de Hopkins es de 0.1, lo que indica que es muy separable. En la Figura 5.6 (a), que muestra el resultado de aplicar el VAT a los datos, reafirma este resultado. Se distinguen entre 4 o 5 divisiones marcadas en la Figura 5.6 (b).

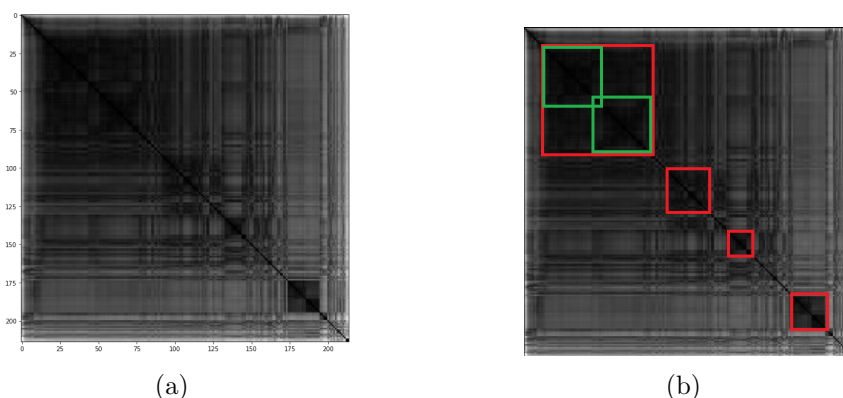


Figura 5.6: VAT de la base de datos de Glass.

Esto quiere decir que, aunque no se ajusten a las etiquetas proporcionada, existe una separación buena de los datos. Y gracias a la herramienta se puede entender porque ha hecho este tipo de división. Para ello, primero hay que entender los datos. El dataset tiene datos de los elementos químicos de 6 clases de cristales: Los 2 primeros tipos

representan cristales de edificios, mientras que el tercero es de vehículos. El 5 es vidrio de envases, el 6 de vajillas y el 7 de lámparas. Los atributos son (1) *índice de refracción*, (2) *Na*, (3) *Mg*, (4) *Al*, (5) *Si*, (6) *K*, (7) *Ca*, (8) *Ba* y (9) *Fe*.

En la Figura 5.7 (a) se observa la primera división, que divide los 3 primeros cristales por un lado y el resto por el otro de forma casi perfecta. Esta división, como se ve en el apartado (b) de la misma figura, se hace principalmente por el nivel de magnesio y en menor medida por el nivel de calcio y sodio. Una segunda división interesante se representa en la Figura 5.8. Donde se divide la vajilla y los envases de las lámparas, teniendo en cuenta el nivel de magnesio, aluminio y bario. La tercera división, representada en la Figura 5.9 solo tiene en cuenta el nivel de hierro para separar los cristales de vehículos, y edificios. Esta división deja de relacionarse con las etiquetas. Parece que existen dos proporciones distintas de hierro, que divide a más o menos un tercio de los cristales del otro. En la realidad existen dos formas de crear ventanas de edificios y vehículos, las que utilizan muy poco hierro y las que utilizan algo más. Los primeros cristales son más caros y seguros por lo que se suelen utilizar en ventanas más pequeñas por lo que cabe intuir que son los de menor número. Mientras, los de mayor número, podrían ser los que utilizan algo más de hierro en su producción. De esta manera, utilizando un razonamiento subjetivo y con ayuda de la información brindada por el algoritmo se pueden observar tendencias reales en los datos. Aunque, ciertamente, este tipo de análisis tiene un valor empírico menor y debería comprobarse con los datos de entrada.

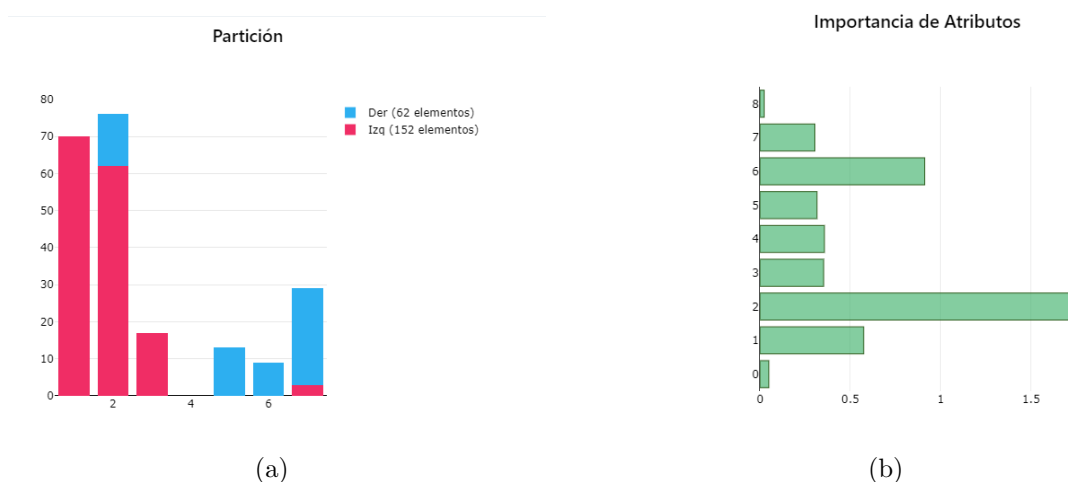


Figura 5.7: Resultado de la primera división de Glass.

También se compara el algoritmo con algoritmos de clustering jerárquico, en este caso con el algoritmo Affinity Propagation que utiliza Ward-Linkage. A pesar de que este algoritmo sea aglomerativo y HIC sea divisivo, se pueden comparar el árbol de la aplicación con el dendrograma del algoritmo si se cortan en el mismo nivel. En la Figura 5.10 se pueden observar ambos gráficos de la base de datos de Olives. (a) es el dendrograma de Ward-Linkage. Entre paréntesis se encuentra el número de elementos de cada clúster y en la parte inferior el identificador del aceite de oliva con mayor número. La misma información se encuentra en el árbol de la herramienta en (b). Se

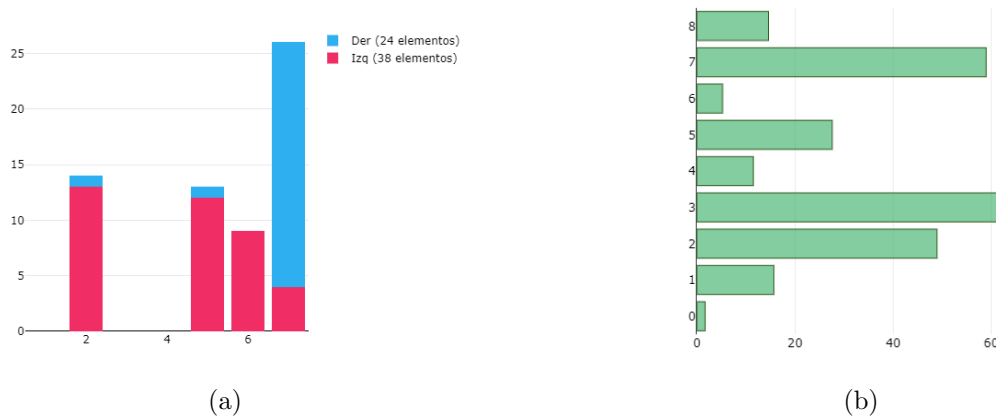


Figura 5.8: Resultado de la segunda división de Glass.

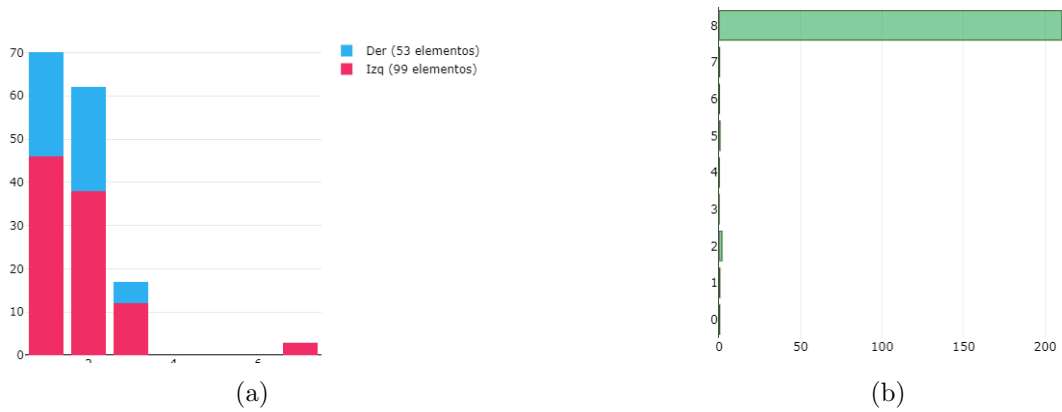
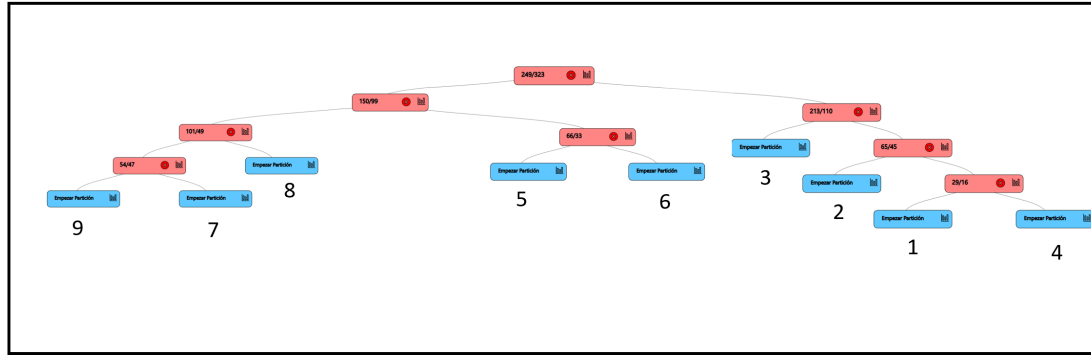


Figura 5.9: Resultado de la tercera división de Glass.

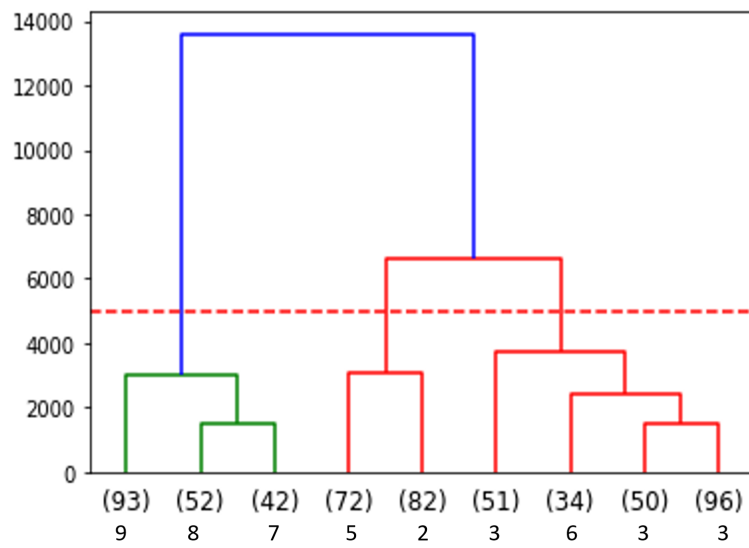
puede apreciar que Affinity Propagation no hace una adecuada separación de los cinco primeros tipos del resto que, como ya se ha visto en anteriores secciones, se una división perfecta. Por otro lado, se repiten varias veces la clase 3. En el resto de los métodos de elección de representante no se encuentran 9 grupos con más de un elemento.

En la Figura 5.11 se puede ver el mismo procedimiento que antes con la base de datos Iris. En este caso se puede ver que el algoritmo del Dr. Manuel Rubio ha tenido mejores resultados, aunque ambos han hecho la misma división: primero han separado *Setosa* del resto. Está vez, dentro de los distintos métodos de los algoritmos de clustering jerárquicos los mejores resultados han sido los de Average y Ward (b). Single-Linkage en (c) ha dividido mayoritariamente en dos el clúster, y Complete-Linkage en (d) son algo peores que los dos primeros.

También se pueden realizar mediciones sin información de clase. Por ejemplo, en la base de datos de aceites de oliva, la media de valores Silhouette de todos los puntos da 0.46. Para poner en contexto este dato, otros algoritmos como el k-Means y el Clustering Aglomerativo con distancia euclídea y Linkage-Ward dan 0.33 y 0.29 respectivamente. Todos los resultados están por encima de cero. Esto quiere decir que la media de los



(a)



(b)

Figura 5.10: (a) Árbol de la herramienta (b) Dendrograma con Ward-Linkage.

puntos ha acertado su clúster según la concepción de distancia que tiene esta medida la cual tiene en cuenta la distancia de todos los elementos al punto medio del clúster al que pertenecen y del resto de agrupaciones. Es paradójico que el algoritmo del Dr. Manuel Rubio tenga mejores resultados ya que K-Means y Clustering Aglomerativo se basan en distancias del espacio de origen al contrario que HIC que utiliza direcciones, proyecciones y densidades. Es decir, lo que un algoritmo considera una buena partición no tiene por qué corresponderse con un buen resultado de la medida Silhouette. Esto es algo que se puede ver en la base de datos Iris en la que k-Means y Clustering Aglomerativo tienen 0.46 y 0.45 respectivamente en Silhouette. En cambio, HIC tiene 0.37 a pesar de tener mejores resultados en la medida con información externa. Como se explicó en la Sección 2.2.5, este problema hace que sea complicado la evaluación de clustering sin información externa ya que cada método tiene su forma de dividir.

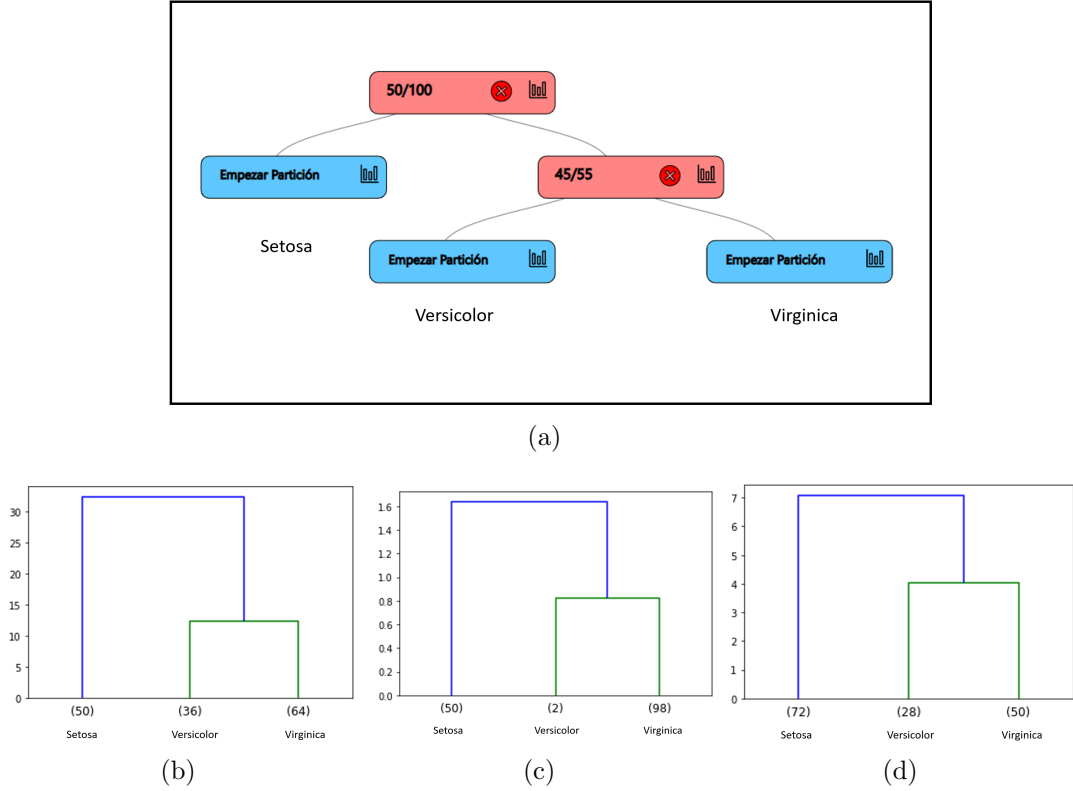


Figura 5.11: (a) Árbol de la herramienta (b) Dendrograma con Ward y Average-Linkage, (c) Dendrograma con Single-Linkage, (d) Dendrograma con Complete-Linkage.

## 5.5. Experimento 4: Algoritmos MMC

En esta sección se utilizará la implementación en Matlab del algoritmo de MMC de Li et co. (2009) [1]. En el artículo se puede ver una tabla con los resultados en distintos experimentos del algoritmo frente a otros métodos como el k-means y otros relacionados con el Maximum Marging Clustering. Se utilizan dos kernels, el lineal y el rbf. La tasa de error en el artículo es la mínima de ambas. El algoritmo propuesto en este TFG es lineal, por lo que si hay una mejor división que no sea divisible mediante hiperplanos, la implementación de MMC funcionará con un menor error. En la Tabla 5.3 se encuentran los resultados del algoritmo interactivo en 10 de las bases de datos del artículo frente a los resultados ofrecidos en el artículo.

En la Tabla 5.3 se puede observar cómo ambos algoritmos se reparten los mejores resultados, aunque en el caso del algoritmo del Dr. Manuel Rubio se consigue el estado del arte en más bases de datos que el algoritmo de MMC, como por ejemplo las divisiones de MNIST (las bases de datos con mayor dimensión). Un problema detectado en esta forma de medir del rendimiento es que, en clases descompensadas, se premia reducir el número de errores totales y no los errores por clase. Este problema se aprecia sobre todo en los datos del Echodiagrama, donde se puede alcanzar los resultados de

|                       | HIC           | MMC           |
|-----------------------|---------------|---------------|
| <i>Echocardiogram</i> | <b>17.42*</b> | 18.18         |
| <i>Heart stalog</i>   | <b>20.03*</b> | 24.81         |
| <i>Haberman</i>       | 44.34         | <b>25.82*</b> |
| <i>lifeDisorder</i>   | -             | <b>34.49*</b> |
| <i>Spectf</i>         | 26.25         | <b>25.70</b>  |
| <i>HouseVotes</i>     | <b>12.64*</b> | 12.87         |
| <i>Clean1</i>         | 34.87         | <b>31.72*</b> |
| <i>Ionosphere</i>     | 27.92         | <b>24.70</b>  |
| <i>MNIST3vs8</i>      | <b>17.52*</b> | 18.12         |
| <i>MNIST1vs7</i>      | <b>1.12*</b>  | 2.30          |

Tabla 5.3: Comparación entre el algoritmo propuesto, Hierarchical Interactive Clustering (HIC) y la implementación del algoritmo MMC (Los resultados marcados con (\*)) son los mejores en esa base de datos con la comparación hecha en el artículo de Li et co. (2009) [1]).

MMC poniendo todos los valores a un lado.

Por otro lado, se han probado tres bases de datos distintas en la aplicación de Matlab en la que está desarrollado el algoritmo y que se puede utilizar para propósitos académicos (Li, n.d.) [4]. En la Figura 5.12 se puede observar los resultados de ejecutar el algoritmo sobre la base de datos de la ionosfera. Mientras, en la Tabla 5.4 se presentan los resultados de la base de datos del Echodiagrama, de las olivas (marcando las primeras cuatro clases como una de las agrupaciones y el resto en otra) y los datos de la ionosfera.

```

-----
Iter | Obj.      | DiffBetas | DualGap | KKT C.  |
-----
1   | -2232.9164 | 0.0000    | -0.0399 | 0.1725  |
2   | -2232.9164 | 0.0000    | -3.7240 | 0.0470  |
3   | -2244.6193 | 0.0065    | -1.6239 | 0.7814  |
4   | -2248.1665 | 0.0023    | 0.0848  | 0.0255  |
5   | -2248.1956 | 0.0011    | 0.0487  | 0.0563  |
maximum number of iterations reached
Output the accuract and the time..
LG_MMC_ACCURACY (linear kernel): 0.70085
LG_MMC_CPUTIME (linear kernel): 4.3906
LG_MMC_ACCURACY (rbf kernel): 0.58689
LG_MMC_CPUTIME (rbf kernel): 2.3594

```

Figura 5.12: Salida del programa de Li (n.d.) [4] donde se implementa la versión descrita en esta sección del algoritmo MMC en Matlab.

Se puede observar como el algoritmo interactivo supera en todos los conjuntos al



|                           | HIC  | Lineal | Rbf  |
|---------------------------|------|--------|------|
| <i>Electrocardiograma</i> | 0.83 | 0.76   | 0.82 |
| <i>Ionosfera</i>          | 0.72 | 0.70   | 0.59 |
| <i>Olivas</i>             | 1.00 | 0.98   | 0.88 |

Tabla 5.4: Resultados de ejecutar el algoritmo de MMC con sus dos kernel en bases de datos, frente a los resultados obtenidos por el propuesto en este TFG, Hierarchical Interactive Clustering (HIC).

MMC con sus dos kernel. El resultado de la ionosfera dista bastante del publicado en el artículo, pudiendo deberse a que se utilizó otra normalización a la que se da por defecto en el programa ofrecido. En cualquier caso, el algoritmo interactivo (utilizando esa misma normalización) consigue superar al kernel lineal y rbf en los tres conjuntos de datos. Se puede observar que según aumenta el número de parámetros el rbf empeora los datos.

Por último, se tratará los tiempos de ambos algoritmos. Se utiliza como referencia los vistos en el artículo, aunque distan bastante de los experimentados con el código de Matlab, debido seguramente a los parámetros, el hardware y la normalización utilizados. Se puede destacar que en el tiempo que se buscaba la solución de MNIST 3vs8 se calcularon 10 puntos con la medida  $\phi$ . Mientras que en MNIST 7vs1 se calcularon 31 puntos con la medida  $\psi$ . Siendo este un método no convexo, es normal que supere en tiempo al algoritmo convexo. Aunque, el beneficio de este último es que obtiene mejores resultados. Se puede observar con los resultados obtenidos, que, en varias bases de datos, esto no es así.

## 5.6. Experimento 5: Automatización del Algoritmo

En los últimos capítulos se ha demostrado que el algoritmo estudiado es un método prometedor, pero la interacción humana en el algoritmo puede hacer de él un método tedioso y complicado de reproducir, ya que habría que volver a construir la herramienta descrita en el actual TFG. Cabe preguntarse si es realmente necesaria la interactividad en el algoritmo y si se puede sustituir las partes con interacción humana por procesos automáticos. Además de responder a estas preguntas se tratará de entender los beneficios de la interactividad. La interacción con el usuario se da, principalmente, en tres partes de la herramienta:

En primer lugar, está el árbol, donde el usuario tiene la tarea de decidir si separa un nodo. Las ventajas de que sea el propio usuario el que decida esto, es que no solo utiliza datos empíricos, como los valores de bimodalidad de las primeras direcciones encontradas o el número de datos a separar, sino que, puede utilizar la intuición como experto del dominio y los datos de separaciones previas, para decidir si separa un nodo o no. Para automatizarlo, es necesario un parámetro de parada, pudiendo ser el número

de clúster a separar, el número de elementos mínimos o la bimodalidad mínima que se exigirá para separar. Otro aspecto es el orden en el que se elige separar, ya que, si el número de clúster está limitado, el seleccionar que nodos separar primero es un aspecto determinante en la división final de los datos. Dependiendo del problema se puede elegir entre estas tres soluciones para automatizar el proceso de buscar el nodo a separar:

- Expandir el árbol con una búsqueda en anchura: la solución más rápida,  $\mathcal{O}(1)$ , pero es la menos inteligente. Será útil en casos en los que el número de clúster y el número mínimo de elementos por clúster no sea determinante.
- Expandir el nodo con más elementos: una solución que crece de manera exponencial con el nivel del árbol,  $l$ . Su complejidad será de  $\mathcal{O}(2^l)$ . Este es un buen método para problemas en los que el número de clúster no sea determinante.
- Ejecutar  $N$  iteraciones del algoritmo en cada nodo y expandir el que tenga un valor de bimodalidad más alto: la complejidad de esta solución será la misma que en el anterior método pero multiplicado por las  $N$  iteraciones del algoritmo. Esta solución es adecuada para problemas con un número reducido de clústeres.

Si se elimina el árbol de la herramienta se perderá la información de la importancia de los atributos en cada separación. Además, en el caso de que alguna de las divisiones se haya hecho bajo una distribución muy pobre, el usuario no sabrá en que parte del árbol se ha dado. Para solucionar esto, se podría continuar mostrando la información del árbol, al terminar de ejecutar el algoritmo.

En segundo lugar, está la Frontera de Parteo donde el usuario tiene que elegir uno de los puntos. Una posible solución es llegar a un compromiso previo entre la varianza y la medida de bimodalidad en forma de parámetro. Una forma de hacer esto es la utilización de métodos de combinación de objetivos, tal como sugiere el Dr. Manuel Rubio. Donde se convierte el problema en una optimización de un único objetivo, añadiendo la varianza a la fórmula de maximización de la bimodalidad, dando a ambas un peso determinado a través, por ejemplo, de un parámetro. La automatización de este proceso hará que sea complicada la elección de la mejor dirección, ya que habría que probar a ejecutar el algoritmo con distintos parámetros, para encontrar el que mejor funcione. Además de que puede que el compromiso alcanzado para una división no valga para la siguiente. Esto llevará a un descenso del espacio de búsqueda de la solución.

La última tarea interactiva del algoritmo es la división del histograma. Su automatización está implementada en la propia herramienta como ayuda al usuario. Después de los resultados obtenidos durante el capítulo se ha visto que las medidas de bimodalidad son excelentes en la tarea de detectar distribuciones bimodales, pero ¿son buenas en lo inverso?, es decir, ¿cuánto más bajo es el valor de bimodalidad más unimodal es la distribución? Si es así, se podría seleccionar el corte de la partición que haga mínima la suma de los valores de bimodalidad. El planteamiento queda resumido en la Figura 5.13 que corresponde a la primera división de la base de datos de olivas. La división azul tendrá un valor muy bajo de bimodalidad en el lado izquierdo de la división, ya

que solo hay una montaña. En cambio, en el lado derecho, al haber dos montañas, deberá haber un valor muy alto de bimodalidad. En cuanto a la división verde, en el lado derecho, no debería haber apenas bimodalidad, pero en la otra parte, al haber una pequeña montaña a la derecha, debería de tener un valor mayor que la división verde, donde ambos lados tienen bimodalidades muy bajas.

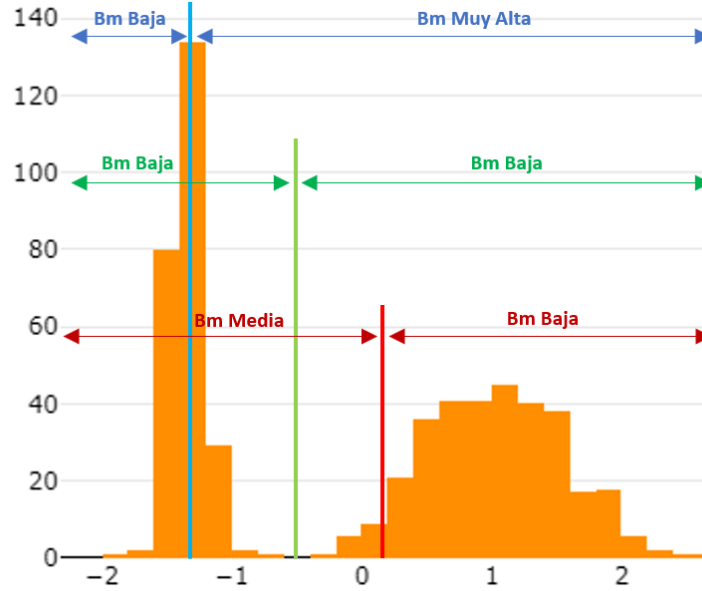


Figura 5.13: Concepto de separación automática.

Para medir el rendimiento de las medidas de bimodalidad en la medición de distribuciones unimodales se utilizará la herramienta de división automática con los valores de  $\phi$ , que es la medida que ha dado mejores resultados, y con el método de *Otsu* (1979) [24], una técnica de umbralizado automático para histogramas de imágenes en escalas de grises. Esta técnica utiliza la varianza de la distribución para dividir histogramas bimodales.

En la tabla 5.5 se pueden ver los resultados de este experimento frente a las separaciones que haría un humano y los límites tolerables de la división (la anchura de la barra del histograma seleccionada por el humano para la separación). Puede verse como los valores de  $\phi$  se distancian bastante de la división que haría un humano. Esto se debe a que la medida no distingue cual es la división menos bimodal. Las divisiones que no son totalmente bimodales tienen valores muy bajos diferenciados entre ellos por muy pocos decimales que no siguen un orden lógico. Por lo que se puede concluir que estas medidas no son las adecuadas para buscar la distribución más unimodal.

En cuanto al método de *Otsu*, parece que se encuentra dentro de los límites en casi todas las ocasiones, aunque, a veces, los resultados se distancian de los que elegiría un humano. Estos resultados pueden variar las soluciones en gran medida o no. Por ejemplo en la primera división ambos se sobrepasan los límites, aunque el método de *Otsu* confunde un único elemento. Mientras el de  $\phi$  confunde 6 (de 572 elementos totales).

|                     | Usuario | Mínimo<br>Tolerable | Máximo<br>Tolerable | Otsu  | Phi   |
|---------------------|---------|---------------------|---------------------|-------|-------|
| <i>Separación 1</i> | 0.5     | -0.6                | -0.38               | -0.18 | -0.03 |
| <i>Separación 2</i> | 0       | -0.5                | 0.5                 | -0.43 | -0.78 |
| <i>Separación 3</i> | -0.22   | -0.47               | 0                   | -0.23 | -0.74 |
| <i>Separación 4</i> | -0.24   | -0.74               | -0.24               | -0.56 | -0.78 |
| <i>Separación 5</i> | -1.98   | -2.2                | -1.79               | -2.02 | -1.76 |
| <i>Separación 6</i> | 0.6     | 0.4                 | 0.8                 | 0.4   | 0.67  |
| <i>Separación 7</i> | -1.22   | -1.5                | -1                  | -1.11 | -1.58 |
| <i>Separación 8</i> | -1.46   | -2                  | -0.97               | -1.73 | -0.97 |

Tabla 5.5: Generación automática de la separación con los los métodos de Otsu y  $\phi$  (en verde dentro de los márgenes y en rojo fuera), frente a los elegidos por el usuario.

Si se utilizan los valores de separación de *Otsu* para dividir la base de datos de aceites de oliva, el resultado es muy similar al de un humano, se distancian en unos pocos decimales con los resultados obtenidos por  $\phi$ , dando un 92 % de acierto. En el caso del conjunto de datos Iris, ha tenido incluso mejores resultados, aunque, de nuevo, al redondear, los resultados coinciden en 97 %. En bases de datos donde un pequeño cambio puede significar grandes modificaciones en el resultado, como es el caso de la separación de Mnist de los número 1 y 7, el resultado de *Otsu* ha sido de 97 % mientras que en 3 contra 8, el resultado ha sido del 81 %. Ambos algo por debajo del resultados que se encuentran en la Tabla 5.3. Se puede concluir, que a pesar de que algunas veces la distancia de separación del humano y la realizada por *Otsu* estén distanciadas, los resultados finales son casi idénticos, al menos en las pruebas realizadas.



# 6

## Conclusiones y trabajos futuros

Con el uso de tecnologías web, tales como React y Flask, se ha desarrollado una herramienta que no solo permite la ejecución del algoritmo de clustering proporcionado por el Dr. Manuel Rubio, si no, que incluye herramientas de interactividad que ofrecen un mayor entendimiento de los resultados, más variabilidad en las soluciones y una selección más intuitiva de los parámetros.

Durante el Capítulo 2: [Marco Teórico](#) se explica en que consiste el clustering junto con ejemplos, formas de evaluarlo y los problemas que puede causar una alta dimensionalidad.

En el Capítulo 3: [Estado del Arte](#) se han explicado algoritmos de clustering modernos como el Maximum Margin Clustering y los beneficios en proyectos reales de la interactividad en algoritmos de agrupamiento. Por último, se ha detallado cómo funciona el algoritmo del Dr. Manuel Rubio.

En el Capítulo 4: [Descripción Informática](#) se ha explicado el proceso de selección de las tecnologías: React y Flask. También se explica cómo es la herramienta final y como utilizarla para la realización de los experimentos.

Por último, los resultados obtenidos en el Capítulo 5: [Experimentos](#), permiten sacar algunas conclusiones:

- En primer lugar, queda ampliamente demostrado que debe ser uno de los algoritmos a probar al enfrentarse a un problema de clustering. La escalabilidad del algoritmo, la cual permite utilizarlo en todo tipo de problemas y los resultados obtenidos, alcanzando el estado del arte en varias bases de datos, hacen de este algoritmo un método de agrupamiento prometedor, sobre todo en bases de datos con mayor número de parámetros.

- 
- A pesar de lo anterior, el método tiene algunas desventajas, como su dependencia de la normalización y que no se pueda utilizar para separar distribuciones complejas. Además, los resultados suelen empeorar cuando todas o la mayoría de las columnas son categóricas o distribuciones con poca variabilidad, lo cual se puede comprobar en bases de datos como Haberman o LifeDisorder, donde ambos conjuntos de datos cuentan con parámetros con muy poca diversidad, y los resultados han sido peores que los de sus competidores.
  - Las tres medidas de bimodalidad, son complementarias, y su combinación en las separaciones suele mejorar los resultados.  $\phi$  y  $\beta$  encuentran las medidas más bimodales, normalmente con resultados muy similares, aunque algunas veces una de ellas sobresale respecto a la otra. Mientras,  $\psi$  encuentra otras distribuciones que lo son menos, pero con mayor varianza, lo que puede ser útil en determinados problemas.
  - La comparación con métodos de clustering modernos como el MMC, ofrecen resultados que refuerzan el punto uno. Y se ha podido comprobar que el uso de kernels no es tan importante en bases de datos de alta dimensión, reduciendo la importancia de algunos de los problemas expuestos en el punto dos.
  - El algoritmo destaca, sobre todo, en la interpretabilidad de las soluciones. Tanto en los casos de los conjuntos de aceites de oliva y cristales, se han sacado informaciones y tendencias de las separaciones realizadas que han permitido entender mejor los datos.
  - Con los resultados e ideas expuestas en la Sección 5.6, se puede afirmar que la automatización del método es posible, aunque esto reducirá la variabilidad del método. Dependiendo del tipo de automatización puede aumentar el tiempo de ejecución del algoritmo y el número de parámetros pueden ser demasiados. Según las pruebas realizadas, utilizar *Otsu* para la separación, da casi los mismos resultados que la división humana. Se puede intuir que al reducir la variabilidad y separar el histograma de forma automática, puedan empeorar los resultados obtenidos. Aunque para afirmar esto se debería de realizar experimentos con el algoritmo automatizado.

Al ser un método totalmente nuevo, se abren multitud de puertas a partir de aquí. Entre las líneas futuras, se encuentran la automatización del método y probar a usar distancias en vez de proyecciones lo que podría mejorar el método para conjuntos de datos con pocas dimensiones. En cuanto a las líneas que siguen a la herramienta, se pueden hacer multitud de mejoras. Primero, se deben mejorar algunos aspectos visuales como los formularios. Se debería dar la posibilidad de agrandar las principales partes de la herramienta y añadir más respuestas a las acciones del usuario como alertas al añadir un histograma que ya has añadido o marcando los puntos ya seleccionados de otro color. Para reducir del trabajo de preparación de los datos se podría añadir la posibilidad de utilizar más tipos de bases de datos, por ejemplo, separados por otros símbolos como punto y coma o tabulaciones, y con otras extensiones como .txt o .xls. Para aumentar el entendimiento de una división, se puede incluir una gráfica que dibuje las agrupaciones con distintos colores en las dos dimensiones más altas en la división

y añadir un grupo de gráficas que representen los parámetros más importantes en la división frente a la información de clase o la separación hecha. Además se debería añadir principios de usabilidad y accesibilidad. Otro de los puntos a explorar es la posibilidad de convertir la herramienta en una aplicación de escritorio, con herramientas como Electron facilitando la instalación de la herramienta.

En conclusión, el algoritmo propuesto por el Dr. Manuel Rubio, es un método de clustering escalable, en el que convergen la idea del Maximum Marging Clustering con la interactividad, para encontrar la perspectiva más bimodal y separarla. Los resultados experimentales, muestran que es una prometedora aproximación al problema de agrupamiento de datos, superando los resultados de los algoritmos más utilizados en datos con distinta dimensión y de diferente dominio. Se abren nuevas líneas para mejorar y ampliar el algoritmo y la herramienta desarrollada. Entre estos trabajos futuros se encuentra la automatización de la herramienta, para lo cual se han dado algunas ideas durante este TFG.





# Bibliografía

- [1] Y.-F. Li, I. W. Tsang, J. Kwok, and Z.-H. Zhou, “Tighter and convex maximum margin clustering,” in *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, D. van Dyk and M. Welling, Eds., vol. 5. PMLR, 16–18 Apr 2009, pp. 344–351. [Online]. Available: <https://proceedings.mlr.press/v5/li09c.html>
- [2] AnAj. (2006) Svm margins image de english wikipedia, public domain, via wikimedia commons. [Online]. Available: [https://commons.wikimedia.org/wiki/File:SVM\\_margins.png](https://commons.wikimedia.org/wiki/File:SVM_margins.png)
- [3] Elisfm. Svm 8 polinomial, public domain, via wikimedia commons. [Online]. Available: [https://commons.wikimedia.org/wiki/File:Svm\\_8\\_polinomial.JPG](https://commons.wikimedia.org/wiki/File:Svm_8_polinomial.JPG)
- [4] Y.-F. Li. Lgmmc: Código de tighter and convex maximum margin clustering. [Online]. Available: [http://www.lamda.nju.edu.cn/code\\_LGMMC.ashx?AspxAutoDetectCookieSupport=1](http://www.lamda.nju.edu.cn/code_LGMMC.ashx?AspxAutoDetectCookieSupport=1)
- [5] L. A. Suchman and R. H. Trigg, *Artificial Intelligence as Craftwork*. Chrisley, R., 2000, ch. 68, pp. 34–63.
- [6] J. McCarthy, “What is artificial intelligence?” *Stanford Engineering: Computer Science*, 1998. [Online]. Available: <http://www-formal.stanford.edu/jmc/whatisai/whatisai.html>
- [7] H. E. Driver and A. L. Kroeber, *Quantitative Expression of Cultural Relationship*. University of California Press, 1932, vol. 31, no. 4.
- [8] H. Steinhaus, “Sur la division des corps matériels en parties,” *Bulletin de l’Académie Polonaise des Sciences, Classe III*, vol. 4, no. 12, pp. 801–804, 1956.
- [9] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” *Proc. Fifth Berkeley Symp. on Math. Statist. and Prob.*, vol. 1, pp. 281–297, 1967.
- [10] k-means advantages and disadvantages. en clustering in machine learning del machine learning crash course de google. [Online]. Available: <https://developers.google.com/machine-learning/clustering/algorithm/advantages-disadvantages>
- [11] A. K. Jain, M. N. Murty, and P. J. Flynn, “Data clustering: A review,” vol. 31, no. 3, 1999. [Online]. Available: <https://doi.org/10.1145/331499.331504>
- [12] A. K. Jain, “Data clustering: 50 years beyond k-means,” *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167865509002323>
- [13] Documentación del paquete pyclustertend para python. [Online]. Available: <https://pyclustertend.readthedocs.io/en/latest/>
- [14] J. Amat, “Clustering y heatmaps: aprendizaje no supervisado,” *Attribution 4.0 International (CC BY 4.0)*, 2017. [Online]. Available: [https://www.cienciadedatos.net/documentos/37\\_clustering\\_y\\_heatmaps](https://www.cienciadedatos.net/documentos/37_clustering_y_heatmaps)

- [15] M. Rodriguez, C. Comin, D. Casanova, O. Bruno, D. Amancio, F. Rodrigues, and L. da F. Costa, “Clustering algorithms: A comparative approach,” *PLOS ONE*, vol. 14, 12 2016. [Online]. Available: <https://doi.org/10.1371/journal.pone.0210236>
- [16] V. Estivill-Castro, “Why so many clustering algorithms - a position paper,” *SIGKDD Explorations, Volume 4, Issue 1*, pp. 65–75, 2002.
- [17] Scikit Learn developers. Clustering. [Online]. Available: <https://scikit-learn.org/stable/modules/clustering.html>
- [18] L. Xu, J. Neufeld, B. Larson, and D. Schuurmans, “Maximum margin clustering,” vol. 17, 11 2004.
- [19] A. Rahimi and B. Recht, “Clustering with normalized cuts is clustering with a hyperplane,” 2004.
- [20] J. Bae, T. Helldin, M. Riveiro, S. Nowaczyk, M.-R. Bouguelia, and G. Falkman, “Interactive clustering: A comprehensive review,” *ACM Comput. Surv.*, vol. 53, no. 1, 2020. [Online]. Available: <https://doi.org/10.1145/3340960>
- [21] J. Freeman and R. Dale, “Assessing bimodality to detect the presence of a dual cognitive process,” vol. 45, pp. 83–97, 2013. [Online]. Available: <https://doi.org/10.3758/s13428-012-0225-x>
- [22] A. B. Smith. (2017) Component-based design. [Online]. Available: <https://alanbsmith.medium.com/component-based-design-168811e0c4ab>
- [23] Scikit Learn developers. Comparing different clustering algorithms on toy datasets. [Online]. Available: [https://scikit-learn.org/stable/auto\\_examples/cluster/plot\\_cluster\\_comparison.html](https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html)
- [24] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

# Apéndice





# Conceptos Matemáticos

En el presente apéndice se describirán distintos conceptos matemáticos que son importantes para entender varias secciones del TFG.

## A.1. Conceptos de Álgebra

Un **vector** de  $n$  componentes se define como un conjunto ordenado de  $n$  números,  $v_1 = (x_1, x_2, x_3, \dots, x_n)$ . Representan una dirección, un sentido y una magnitud. En Machine Learning se utilizan los vectores y las matrices (grupos de vectores) para representar información. Cada componente del vector será una propiedad. La **dimensión** del vector vendrá determinada por el número de componentes.

Un **espacio vectorial** es un conjunto de vectores con dos operadores (suma y multiplicación por escalar) que satisfacen unos axiomas, que se pueden recordar en cualquier libro de Álgebra. En la Figura A.1 se observa un espacio de 3 dimensiones que representan el espacio de los coches describiéndolos con 3 propiedades ( $n^o$  de puertas, años del vehículos, ultima reparación), y un coche con unas propiedades definidas (el vector azul).

Un **subespacio vectorial** es un subconjunto de los vectores de un espacio vectorial que cumple las propiedades de que la suma de dos elementos debe estar en el subespacio y la multiplicación por cualquier escalar también. De esto se deduce que todo subespacio debe contener al 0. Un subespacio de  $R^2$  puede ser una recta, en  $R^3$  un plano, y en  $R^4$  un hiperplano. Estos se pueden representar a través de su fórmula implícita como:

$$Ax_1 + Bx_2 + Cx_3 + \dots = 0$$

Por lo que un hiperplano, que es un subconjunto del espacio de propiedades de los datos puede quedar definido a través de una lista de sus constates ( $A, B, C, D, \dots$ ). Siendo un

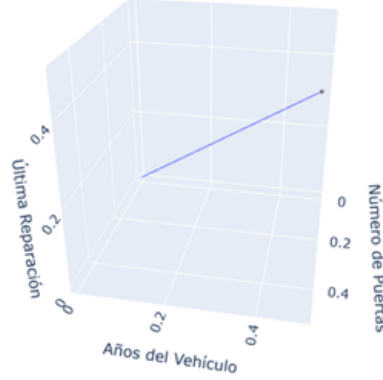


Figura A.1: Ejemplo de espacio vectorial y un vector de ese espacio.

hiperplano del anterior ejemplo:

$$A \cdot \text{n}^{\circ} \text{ de puertas} + B \cdot \text{años del vehículos} + C \cdot \text{ultima reparación} = 0$$

Donde A, B y C son constantes.

Una **aplicación lineal** es una función que dada una entrada en un espacio vectorial da una salida en otro espacio vectorial que cumple que:

$$T(u + v) = Tu + Tv \quad T(\alpha \cdot v) = \alpha Tv$$

Lo que garantiza que para cada vector de entrada solo hay una salida. Una importante aplicación, que no tiene por qué ser lineal a no ser que se utilice en el espacio real, es la **proyección**: dado un punto en un espacio encuentra el punto de un subespacio en el que la distancia sea mínima entre ellos. La componente de la proyección de un vector sobre otro vector se puede calcular mediante la siguiente fórmula:

$$||\text{proy}_{\vec{a}} \vec{b}|| = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}|}$$

## A.2. Algoritmos de optimización

### A.2.1. Descenso de Gradiente

El descenso de gradiente es un algoritmo iterativo que permite encontrar soluciones óptimas a problemas bajo una función de coste  $f(\mathbf{x})$ , la cual recibe como entrada una lista de parámetros,  $\mathbf{x} = x_0, x_1, \dots, x_n$ . El algoritmo se basa en ajustar los parámetros de manera iterativa hasta encontrar un mínimo en la función de coste. El algoritmo comienza escogiendo unos parámetros

aleatorios, y se calcula la dirección en el que la función de coste descienda más. Esto se hace calculando el gradiente de la función:

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f(\mathbf{x})}{\partial x_0} \\ \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_n} \end{pmatrix}$$

Después se actualizan los valores de los parámetros,  $x'$ , en la dirección opuesta al gradiente (que es la dirección de mayor incremento).

$$\mathbf{x}' = \mathbf{x} - \alpha \nabla f(\mathbf{x})$$

El parámetro de aprendizaje,  $\alpha$ , determina el porcentaje que se tendrá en cuenta de lo aprendido. Este parámetro ayuda a una convergencia adecuada. El algoritmo termina cuando llega a un mínimo o un punto muy cercano a él, es decir, cuando el gradiente de la función de coste es cero. Cuantos más parámetros o más compleja sea la función de coste más difícil será encontrar el mínimo absoluto, por lo que se suele ejecutar el algoritmo varias veces con parámetros aleatorios distintos.

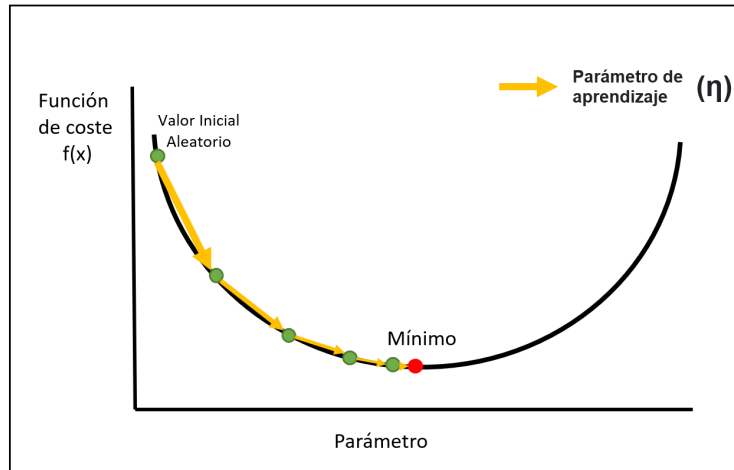


Figura A.2: Simulación de la ejecución del Descenso de Gradiente para un Parámetro.

Se debe tener cuidado con el parámetro de aprendizaje  $\alpha$  ya que si es muy pequeño tardará mucho en encontrar un mínimo y si es un valor grande se pueden saltar los mínimos. Normalmente se utilizan valores que descienden con el tiempo: reduce la velocidad según avanzan las iteraciones del algoritmo. Un ejemplo de cómo sería la ejecución del algoritmo de descenso de gradiente en una función convexa de un único parámetro se puede ver en la Figura A.2.

## A.2.2. Frontera de Pareto

Cuando hay más de un objetivo, ya sea maximizar o minimizar dos funciones sobre las mismas variables se pueden utilizar algoritmos multiobjetivo. Al tener dos o más objetivos, si minimizamos uno puede significar que otro empeore considerablemente. Por eso se necesita



llegar a un consenso entre ambas variables, y eso es lo que hace la frontera de Pareto: Se dice que  $x$  es Pareto óptimo si no hay un  $y$  tal que  $f_i(x) < f_i(y)$  con  $f_i$  todas las funciones que se quieren minimizar. Si se quiere maximizar se cambia el símbolo por un mayor que.

## A.3. Exploración de Datos

Un proceso previo a tomar decisiones sobre los datos es su exploración. Para ello hay técnicas de medidas y herramientas que ayudan a entender los datos.

### A.3.1. Métricas de Variabilidad

La varianza es la suma del cuadrado de las desviaciones de los datos sobre la media, dividida entre el número de elementos menos 1. Mide la variabilidad de una distribución, es decir que será mayor cuanto más alejados estén los datos entre sí.

$$\text{Varianza} = s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}$$

### A.3.2. Exploración de una Distribución

El histograma es un medidor de frecuencia de grupos de valores del eje  $x$ , que mide el número de elementos en ese grupo en el eje  $y$ . No debe confundirse con el diagrama de barras en la que se representan el número de distintas clases. Un ejemplo de histograma se puede observar en la Figura A.3.

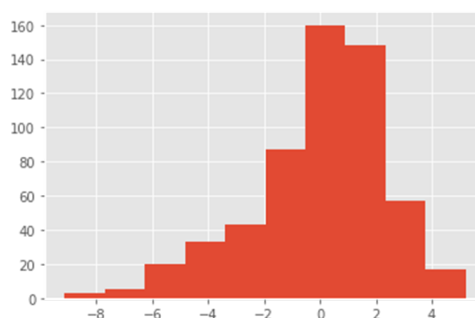


Figura A.3: Ejemplo de Histograma.

B

Gráficas y Tablas Grandes

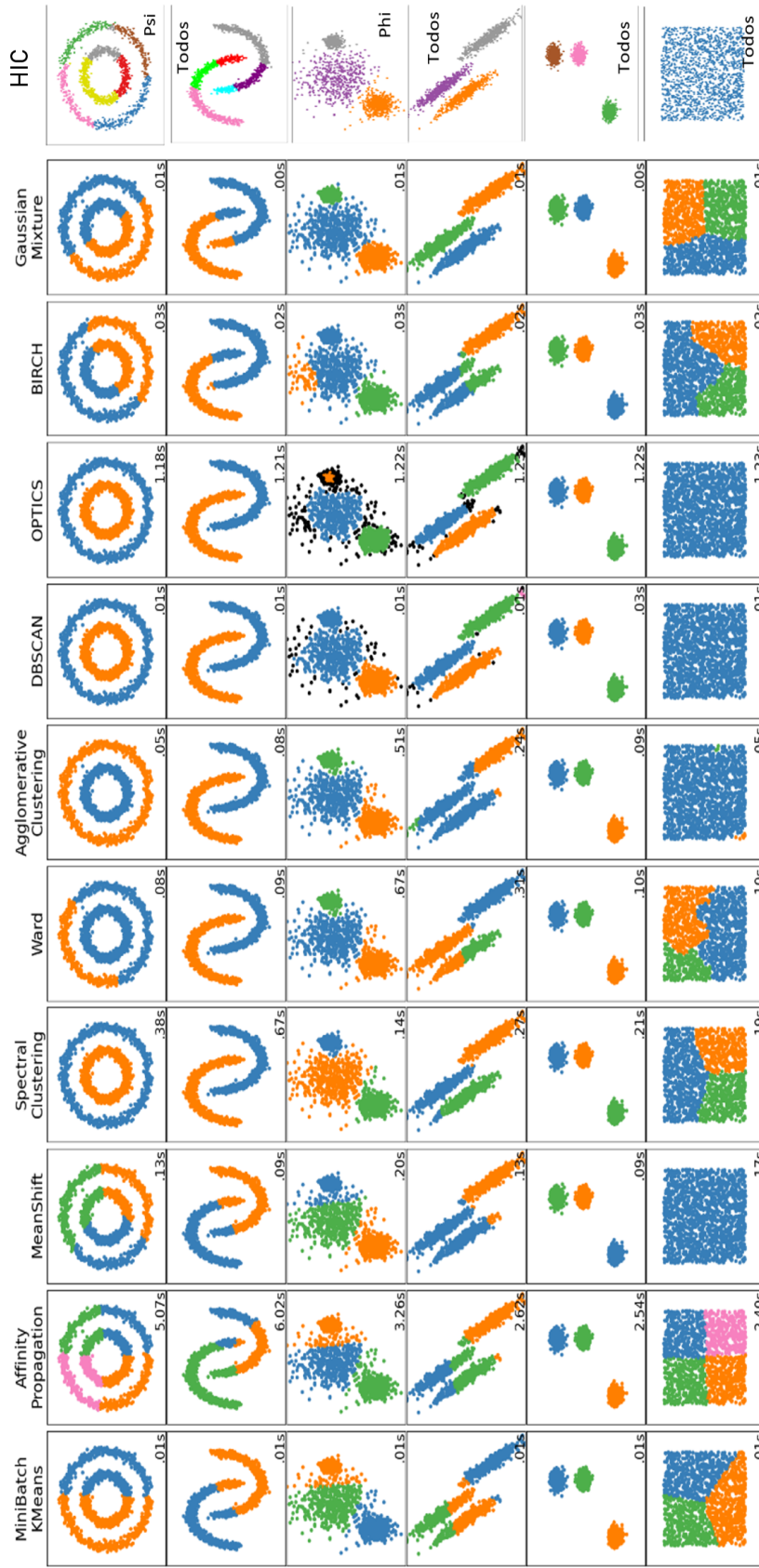


Figura B.1: Resultados de HIC en distribuciones 2D comparado con otros algoritmos de clustering.

|                   | HIC         | KMeans      | DBSCAN      | Affinity<br>Propagation | Agglomerative<br>Clustering | Birch       | MiniBatch<br>KMeans | MeanShift | OPTICS      | Spectral<br>Clustering | Gaussian<br>Mixture |
|-------------------|-------------|-------------|-------------|-------------------------|-----------------------------|-------------|---------------------|-----------|-------------|------------------------|---------------------|
| <i>Olives</i>     | <b>0.93</b> | 0.71        | 0.60        | 0.36                    | 0.65                        | 0.65        | 0.72                | 0.37      | 0.52        | 0.71                   | 0.80                |
| <i>Iris</i>       | <b>0.97</b> | 0.83        | 0.66        | 0.71                    | 0.83                        | 0.83        | 0.84                | 0.67      | 0.67        | 0.85                   | <b>0.97</b>         |
| <i>Cáncer</i>     | <b>0.97</b> | 0.96        | 0.96        | -                       | 0.96                        | <b>0.97</b> | 0.95                | -         | 0.96        | 0.95                   | 0.92                |
| <i>AutoMPG</i>    | <b>0.46</b> | 0.45        | -           | -                       | 0.45                        | 0.45        | 0.45                | -         | -           | 0.45                   | 0.45                |
| <i>Glass</i>      | <b>0.46</b> | 0.45        | -           | -                       | 0.45                        | 0.45        | 0.45                | -         | -           | 0.45                   | 0.45                |
| <i>Mnist</i>      | <b>0.96</b> | <b>0.96</b> | <b>0.96</b> | -                       | <b>0.96</b>                 | <b>0.96</b> | <b>0.96</b>         | -         | <b>0.96</b> | <b>0.96</b>            | <b>0.96</b>         |
| <i>2vs7 Test</i>  |             |             |             |                         |                             |             |                     |           |             |                        |                     |
| <i>Mnist</i>      | <b>0.89</b> | 0.84        | 0.51        | -                       | 0.85                        | 0.85        | 0.87                | 0.51      | -           | -                      | 0.84                |
| <i>3vs8 Test</i>  |             |             |             |                         |                             |             |                     |           |             |                        |                     |
| <i>Echodi</i>     | <b>0.77</b> | <b>0.77</b> | -           | -                       | 0.62                        | 0.62        | <b>0.77</b>         | -         | -           | -                      | -                   |
| <i>agram</i>      |             |             |             |                         |                             |             |                     |           |             |                        |                     |
| <i>Ionosphere</i> | <b>0.77</b> | <b>0.77</b> | -           | -                       | 0.62                        | 0.62        | <b>0.77</b>         | -         | -           | -                      | -                   |

Tabla B.1: Comparación de algoritmos de Clustering con el algoritmo propuesto en el TFG, Hierarchical Interactive Clustering (HIC).